



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria Informatica

Elaborato di Architettura dei Sistemi Digitali

Anno Accademico 2023-24

Roberta Acampora M63001558
Federica Iervolino M63001562
Simone Fuorto M63001564

Indice

1	Reti Combinatorie Elementari	1
1.1	Esercizio 1: Multiplexer 16:1	1
1.1.1	Esercizio 1.1	1
1.1.2	Progetto e Architettura	2
1.1.3	Implementazione	4
1.1.4	Simulazione	6
1.2	Esercizio 1.2	10
1.2.1	Progetto e Architettura	10
1.2.2	Implementazione	11
1.2.3	Simulazione	12
1.3	Esercizio 1.3	15
1.3.1	Progetto e Architettura	15
1.3.2	Implementazione	16
1.3.3	Simulazione	19
1.4	Esercizio 2: Sistema ROM+M	19
1.4.1	Esercizio 2.1	19
1.4.2	Progetto e Architettura	20
1.4.3	Implementazione	22
1.4.4	Simulazione	24

1.4.5	Esercizio 2.2	28
1.4.6	Progetto e Architettura	28
1.4.7	Implementazione	29
1.4.8	Simulazione	30
2	Reti sequenziali elementari	31
2.1	Esercizio 3: Riconoscitore di sequenze	31
2.1.1	Esercizio 3.1	31
2.1.2	Progetto e Architettura	32
2.1.3	Implementazione	33
2.1.4	Simulazione	40
2.2	Esercizio 3.2	44
2.2.1	Progetto e Architettura	44
2.3	Esercizio 4: Shift Register	44
2.3.1	Approccio Comportamentale	45
2.3.2	Approccio strutturale	54
2.4	Esercizio 5: Cronometro	64
2.4.1	Esercizio 5.1	64
2.4.2	Esercizio 5.2	72
2.4.3	Esercizio 5.3	75
2.5	Esercizio 6: Sistema di letturaelaborazione scrittura PO_PC .	79
2.5.1	Esercizio 6.1	79
2.5.2	Esercizio 6.2	97
3	Macchine Aritmetiche	98
3.1	Esercizio 7: Moltiplicatore di Booth	98
3.1.1	Esercizio 7.1	98

3.1.2	Esercizio 7.2	116
3.2	Esercizio 7_bis: Divisore Non—Restoring	123
3.2.1	Esercizio 7BIS.1	123
3.2.2	Esercizio 7BIS.2	141
4	Comunicazione con Handshaking	157
4.1	Esercizio 8:comunicazione con handshaking	157
4.1.1	Esercizio 8.1	157
5	Esercizio 9	183
5.0.1	Esercizio 9 a	183
5.0.2	Esercizio 9 b	191

Capitolo 1

Reti Combinatorie Elementari

1.1 Esercizio 1: Multiplexer 16:1

1.1.1 Esercizio 1.1

Progettare, implementare in VHDL e testare mediante simulazione un multiplexer indirizzabile 16:1, utilizzando un approccio di progettazione per composizione a partire da multiplexer 4:1.

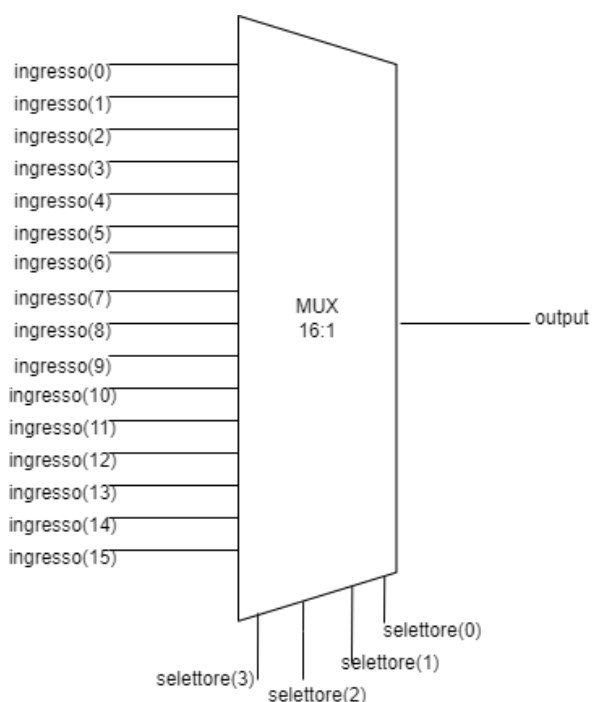


Figura 1.1: Multiplexer 16:1

1.1.2 Progetto e Architettura

L'esercizio 1.1 dell'elaborato richiede di progettare e implementare un Multiplexer 16:1. Seguendo l'approccio richiesto, realizziamo quest'ultimo per composizione, in particolar modo un multiplexer 16:1 può essere realizzato interconnettendo tra loro 5 multiplexer 4:1 su due livelli.

Come prima cosa disegniamo l'architettura di un Multiplexer 4:1

Il multiplexer 4:1 ha 4 ingressi, 2 selettori e una singola uscita. Invece il multiplexer che vogliamo realizzare ha 16 ingressi e 4 segnali di selezione, ciò significa che abbiamo bisogno di almeno 4 multiplexer 4:1 nel primo livello che prenderanno sedici ingressi (4×4). Questo primo livello ci darà 4 output distinti che verranno dati in ingresso ad un quinto multiplexer 4:1.

Questa architettura ci garantisce il funzionamento classico di un multi-

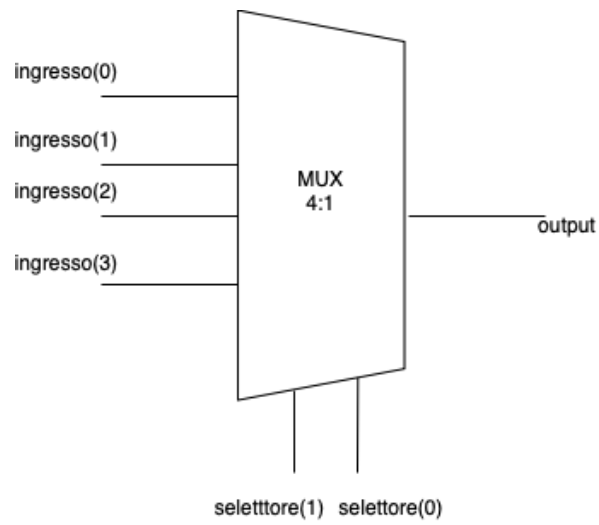


Figura 1.2: Multiplexer 4:1

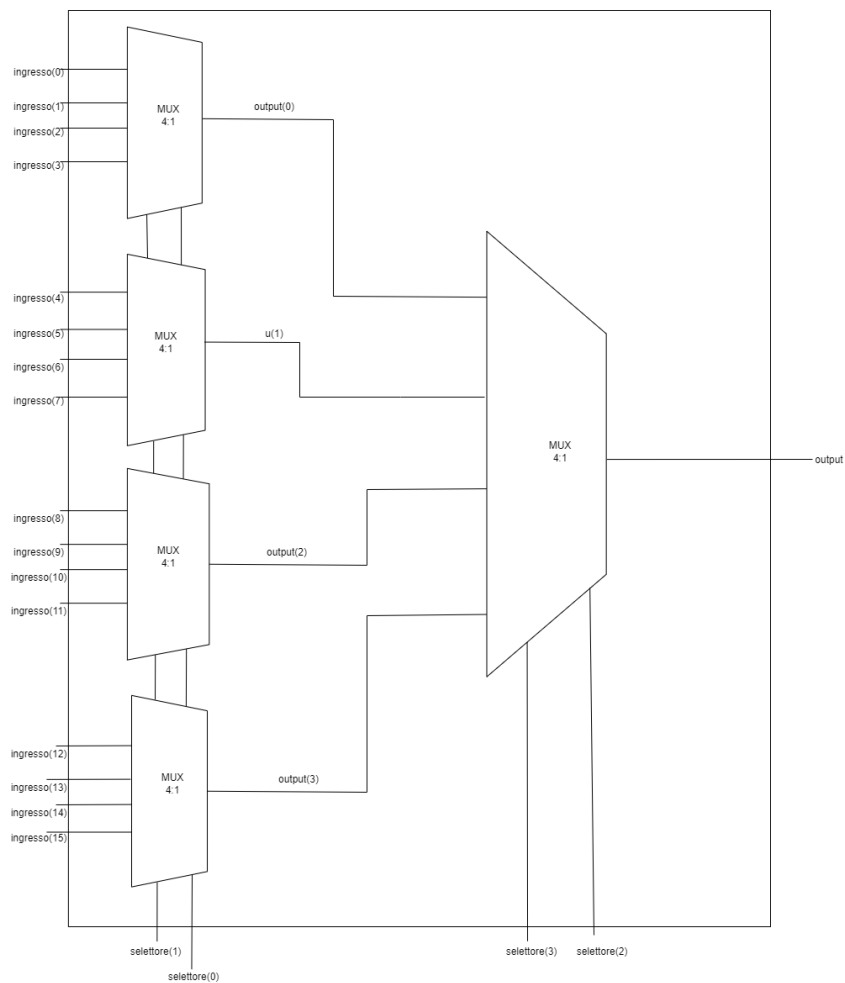


Figura 1.3: Multiplexer Strutturale

plexer 16:1. Un multiplexer non è nient'altro che una macchina combinatoria notevole composta da N ingressi e $\log_2 N$ selettori ed una singola uscita. In questa macchina il selettore può "attivare" una sola linea per volta. L'output sarà pari all'ingresso della linea selezionata. Ciò rende tale macchina utile per l'instradamento dei segnali.

1.1.3 Implementazione

Multiplexer 4:1

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 entity mux_4_1 is
4     Port ( ingresso : in std_logic_vector(3 downto 0);
5           selettore  : in std_logic_vector(1 downto 0);
6           output     : out STD_LOGIC);
7 end mux_4_1;
8 architecture Behavioral of mux_4_1 is
9 begin
10     process(ingresso, selettore)
11     begin
12         case selettore is
13             when "00" => output<=ingresso(0);
14             when "01" => output<=ingresso(1);
15             when "10" => output<=ingresso(2);
16             when "11"=>output<=ingresso(3);
17             when others => output<= '-';
18         end case;
```

```
19     end process;  
20 end Behavioral;
```

Multiplexer 16:1

```
1 entity mux_16_1 is  
2     Port ( ingresso : in std_logic_vector(15 downto 0);  
3           selettore : in STD_LOGIC_vector(3 downto 0);  
4           output    : out STD_LOGIC);  
5 end mux_16_1;  
6  
7 architecture Structural of mux_16_1 is  
8     component mux_4_1  
9         Port ( ingresso : in std_logic_vector(3 downto 0);  
10             selettore : in std_logic_vector(1 downto 0);  
11             output    : out STD_LOGIC);  
12 end component;  
13  
14 signal ing_app: std_logic_vector(3 downto 0) := (others =>  
15     '0'); --segnale d'appoggio  
16  
17 begin  
18     mux_i: for i in 0 to 3 generate  
19         m: mux_4_1 port map(ingresso(i*4 +3 downto i*4 ),  
20             selettore(1 downto 0), ing_app(i));  
21     end generate;
```

```
21     mux_out: mux_4_1 port map(ing_app, selettore(3 downto 2),  
                                output);  
22 end Structural;
```

1.1.4 Simulazione

L'implementazione mostrata in precedenza è stato oggetto di test sia per il multiplexer 4:1 che per il multiplexer 16:1.

```
1  entity mux_16_1_tb is  
2  -- Port ( );  
3  end mux_16_1_tb;  
4  
5  architecture Structural of mux_16_1_tb is  
6      component mux_16_1  
7          Port ( ingresso : in std_logic_vector(15 downto 0);  
8                selettore : in STD_LOGIC_vector(3 downto 0);  
9                output  : out STD_LOGIC);  
10     end component;  
11  
12     signal ingresso : std_logic_vector(15 downto 0) :=(others  
13         =>'0');  
14     signal selettore : std_logic_vector(3 downto 0) :=(others  
15         =>'0');  
16     signal output : std_logic :='0';
```

```
17 begin
18
19 uut: mux_16_1 port map(ingresso, selettore, output);
20 processo: process
21     begin
22         wait for 10 ns;
23         ingresso <= "0000000000000000";
24         selettore <= "0001";
25         wait for 10 ns;
26         ingresso <= "1010101010101010";
27         selettore <= "1111";
28         wait for 10 ns;
29         ingresso <= "1101111111111111";
30         selettore <= "0011";
31         wait for 10 ns;
32         ingresso <= "1101111111111111";
33         selettore <= "1000";
34         wait;
35         end process;
36
37 end Structural;
```

Questo Testbench è stato realizzato allo scopo di verificare la correttezza del top module Multiplexer 16:1. Sono stati presi in analisi quattro casi di test:

- Ingresso:0000000000000000,

selettore:0001

Output: In questo esempio il selettore è pari ad 1 e il corrispondente ingresso è 0, come ci aspettavamo l'output è zero (come si evince dalla Figura 1.4)

- Ingresso:1010101010101010,

selettore:1111,

Output: 1

- Ingresso:1101111111111111,

selettore:0011,

Output: 1

- Ingresso:1101111111111111,

selettore:1000,

Output: 1

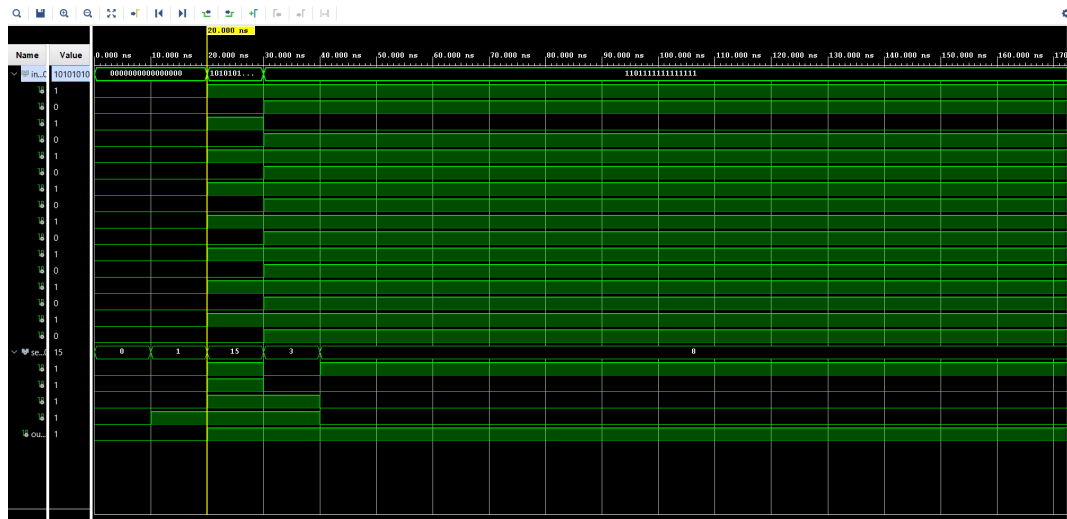


Figura 1.4: Simulazione del Testbench Mux16:1

1.2 Esercizio 1.2

Utilizzando il componente sviluppato al punto precedente, progettare, implementare in VHDL e testare mediante simulazione una rete di interconnessione a 16 sorgenti e 4 destinazioni.

1.2.1 Progetto e Architettura

A differenza dell'esercizio precedente, in questo caso, è richiesta un'architettura a 16 sorgenti e 4 destinazioni. Ciò implica che i primi 2 livelli saranno progettati seguendo la logica dell'esercizio precedente e ad essi verrà aggiunto un ulteriore livello con un demultiplexer 1:4. Questo prenderà in ingresso l'output del multiplexer 16:1 e darà in output quattro linee d'uscita gestite da 2 selettori (logica speculare rispetto al multiplexer).

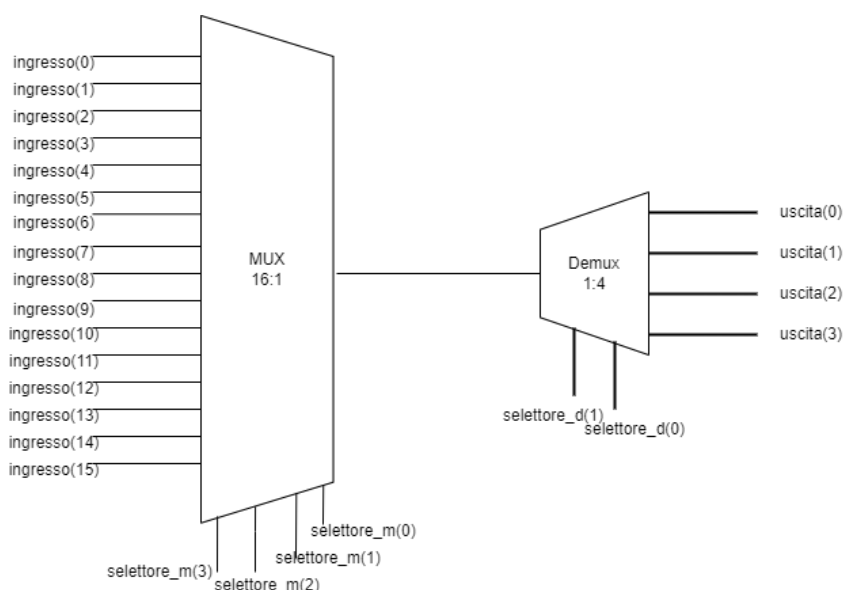


Figura 1.5: Architettura rete di interconnessione 16:4

1.2.2 Implementazione

```
1 entity mux_interco is
2     Port ( input : in STD_LOGIC_vector(15 downto 0);
3           s_m : in STD_LOGIC_vector(3 downto 0);
4           s_d : in STD_LOGIC_vector(1 downto 0);
5           output : out std_logic_vector(3 downto 0));
6 end mux_interco;
7
8 architecture Structural of mux_interco is
9     component mux_16_1
10         Port ( ingresso : in std_logic_vector(15 downto 0);
11              selettore : in STD_LOGIC_vector(3 downto 0);
12              output : out STD_LOGIC);
13     end component;
14
15     component demux_1_4
16         Port ( ingresso : in std_logic ;
17              selettore : in std_logic_vector (1 downto 0);
18              output : out std_logic_vector (3 downto 0));
19     end component;
20     signal u_mux: std_logic := '0';
21 begin
22     multiplexer: mux_16_1 port map(input, s_m, u_mux);
23     demux: demux_1_4 port map(u_mux,s_d,output);
24 end Structural;
```

1.2.3 Simulazione

```
1 entity mux_interco_tb is
2   -- Port ( );
3 end mux_interco_tb;
4 architecture Structural of mux_interco_tb is
5     component mux_interco is
6         Port ( input : in STD_LOGIC_vector(15 downto 0);
7               s_m  : in STD_LOGIC_vector(3  downto 0);
8               s_d  : in STD_LOGIC_vector(1  downto 0);
9               output : out std_logic_vector(3  downto 0));
10    end component;
11 signal ingresso: std_logic_vector(15 downto 0) := (others
12    =>'0');
13 signal selettore_m: std_logic_vector(3  downto 0) := (others
14    =>'0');
15 signal selettore_d: std_logic_vector(1  downto 0) := (others
16    =>'0');
17 signal output: std_logic_vector(3  downto 0) := (others =>'0');
18
19 begin
20     uut: mux_interco port map(ingresso, selettore_m,
21     selettore_d, output);
22     processo: process
23         begin
24             wait for 5ns;
25             ingresso <= "1111111111111111";
26             selettore_m <= "0010";
27             selettore_d <= "01";
```

```
24      wait for 5ns;
25      ingresso <= "0111111111111111";
26      selettore_m <= "1111";
27      selettore_d <= "11";
28      wait for 5ns;
29      ingresso <= "1010101010101010";
30      selettore_m <= "0000";
31      selettore_d <= "11";
32      wait for 5ns;
33      ingresso <= "1010101010101010";
34      selettore_m <= "0000";
35      selettore_d <= "00";
36      wait;
37      end process;
38 end Structural;
```

In questo Testbench abbiamo verificato la correttezza del top module per l'interconnessione 16:4. In questo caso abbiamo avuto bisogno di: uno standard logic vector di dimensione 16 per gli ingressi al multiplexer 16:1 (ingresso), due selettori anch'essi standard logic vector di dimensione rispettivamente 4 (selettore_m) e 2 (selettore_d) e un output standard logic vector di dimensione 4. In seguito abbiamo fornito 4 casi di test:

- ingresso = 1111111111111111;
selettore_m = 0010;
selettore_d = 01;
output = 0010

- `ingresso = 0111111111111111;`
`selettore_m = 1111;`
`selettore_d = 11;`
`output = 0000`
- `ingresso = 1010101010101010;`
`selettore_m = 0000;`
`selettore_d = 11;`
`output = 0000`
- `ingresso = 1010101010101010;`
`selettore_m = 0000;`
`selettore_d = 00;`
`output = 0000`

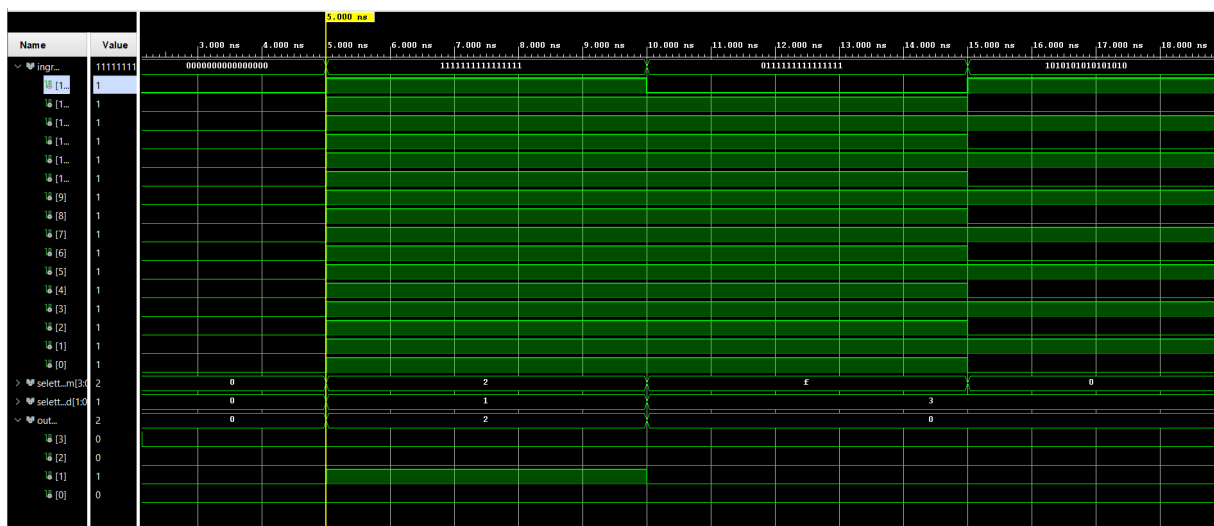


Figura 1.6: Simulazione Interconnessione 16:4

1.3 Esercizio 1.3

Sintetizzare ed implementare su board il progetto della rete di interconnessione sviluppato al punto 1.2, utilizzando gli switch per fornire gli input di selezione e i led per visualizzare i 4 bit di uscita. Per quanto riguarda i 16 bit dato in input, essi devono essere immessi mediante switch, 8 bit alla volta, sviluppando un'apposita "rete di controllo" per l'acquisizione che utilizzi due bottoni della board per caricare rispettivamente la prima e la seconda metà del dato in ingresso.

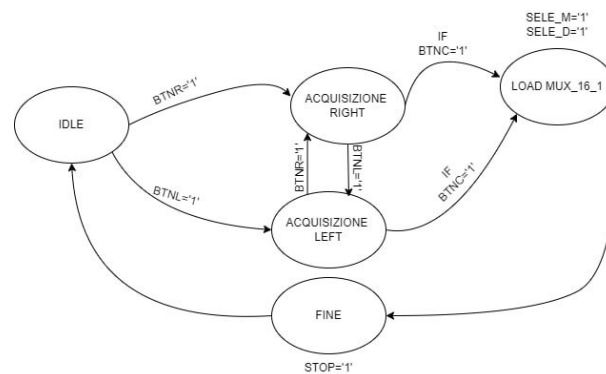
1.3.1 Progetto e Architettura

Partendo dall'esercizio 1.2 abbiamo implementato un codice che ci permette di visualizzare sulla board (NEXYS A7) il funzionamento della rete di interconnessione, sfruttando gli switch per acquisire i valori di input. Gli switch acquisiranno gli input 8 bit alla volta. Utilizziamo gli switch in questo modo per la nostra scelta architettureale, infatti, dato il limitato numero di switch (16), non possiamo utilizzare uno switch per ogni bit. Quindi, abbiamo scelto di utilizzare:

- 8 switch per gli ingressi di rete di interconnessione(nel codice denominati "switch"): R13, U18, T18, R17, R15, M13, L16, J15;
- 6 per i selettori:
 - selettore_m (4): V10, V11, V12, H6;
 - selettore_d (2): T13, R16.

Per la gestione del programma utilizziamo 4 bottoni:

- btnr: acquisizione destra;
- btnl: acquisizione sinistra;
- btnc: bottone per il caricamento;
- rst: bottone per il reset.



1.3.2 Implementazione

Il codice di partenza è quello dell'esercizio 1.2, al quale abbiamo aggiunto la Control Unit:

```

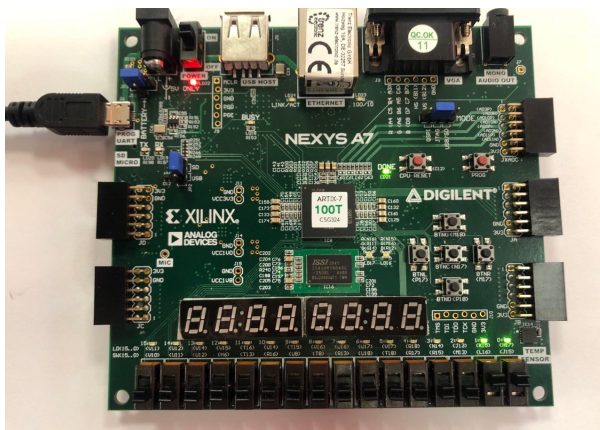
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity Control_Unit is
7      Port ( clk : in STD_LOGIC;
8            rst : in STD_LOGIC;
  
```

```

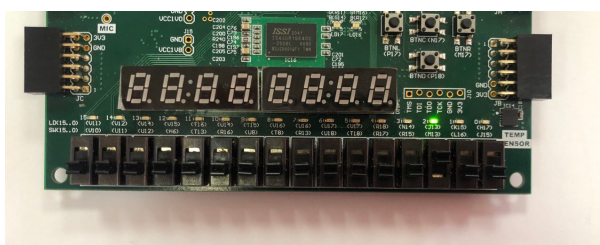
9      btn_start1 : in STD_LOGIC;
10     btn_start2 : in STD_LOGIC;
11     switches :in std_logic_vector(7 downto 0);
12     data_out  : out STD_LOGIC_VECTOR(15 downto 0)
13         );
14 end Control_Unit;
15
16 architecture Behavioral of Control_Unit is
17     type State_Type is (IDLE, ACQUIRE1, ACQUIRE2, DATA_READY);
18     signal state : State_Type := IDLE;
19     signal data_reg1 : STD_LOGIC_VECTOR(7 downto 0) := (others
20         => '0');
21     signal data_reg2 : STD_LOGIC_VECTOR(7 downto 0) := (others
22         => '0');
23
24 begin
25     process(clk, rst)
26     begin
27         if rst = '1' then
28             state <= IDLE;
29             data_reg1 <= (others => '0');
30             data_reg2 <= (others => '0');
31         elsif rising_edge(clk) then
32             case state is
33                 when IDLE =>
34                     if btn_start1 = '1' then
35                         state <= ACQUIRE1;
36                     elsif btn_start2 = '1' then

```

```
35         state <= ACQUIRE2;
36     else
37         state <= IDLE;
38     end if;
39 when ACQUIRE1 =>
40     data_reg1 <= switches; -- Mantieni lo
41                             stato dei primi 8 bit
42     if btn_start1 = '0' then
43         state <= DATA_READY;
44     end if;
45 when ACQUIRE2 =>
46     data_reg2 <= switches; -- Mantieni lo
47                             stato dei secondi 8 bit
48     if btn_start2 = '0' then
49         state <= DATA_READY;
50     end if;
51 when DATA_READY =>
52     data_out <= data_reg1 & data_reg2; --
53                             Concatena i due segnali da 8 bit
54     state <= IDLE;
55 end case;
56 end if;
57 end process;
58 end Behavioral;
```



1.3.3 Simulazione



[H]

1.4 Esercizio 2: Sistema ROM+M

1.4.1 Esercizio 2.1

Progettare, implementare in VHDL e testare mediante simulazione un sistema S composto da una ROM puramente combinatoria di 16 locazioni da 8 bit ciascuna e da una macchina combinatoria M che opera come segue: fornito al sistema un indirizzo A di 4 bit, il sistema restituisce il valore contenuto nella ROM all'indirizzo A opportunamente “trasformato” attraverso la macchina M. Il comportamento della macchina M è totalmente a scelta dello studente, l'unico vincolo è che essa prenda in ingresso 8 bit e ne fornisca in uscita 4.

1.4.2 Progetto e Architettura

La ROM è un elemento puramente combinatorio, dunque, per progettare il sistema richiesto è bastato l'utilizzo di una macchina, che dato un ingresso x associa un'uscita y :

- $x: "00000001" \Rightarrow y: "0001";$
- $x: "00000010" \Rightarrow y: "0010";$
- $x: "00000011" \Rightarrow y: "0011";$
- $x: "00000100" \Rightarrow y: "0100";$
- $x: \text{others} \Rightarrow y: "0000";$

Nello specifico la ROM ha una struttura che simula un contenitore di standard logic vector, con 16 locazioni da 8 bit ciascuna. In base all'indirizzo (address) dato in ingresso essa fornisce in uscita (dout) il contenuto della locazione stessa. L'output, mediante un segnale d'appoggio, verrà fornito in ingresso alla macchina precedentemente descritta.

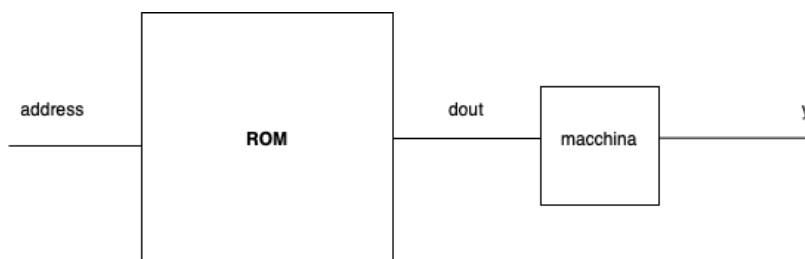


Figura 1.7: Architettura del sistema ROM+M

Infine, abbiamo realizzato un top module che unisce i componenti sopracitati al fine di implementare la logica complessiva del sistema.

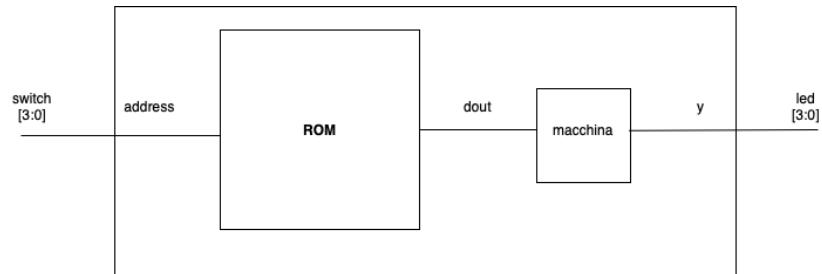


Figura 1.8: Architettura del sistema ROM+M

1.4.3 Implementazione

Di seguito mostriamo l'implementazione della ROM:

```

1 entity rom is
2     Port (address: in std_logic_vector(3 downto 0);
3           dout: out std_logic_vector(7 downto 0)); --dataout
4 end rom;
5
6 architecture Behavioral of rom is
7     type memory_16_8 is array(0 to 15) of std_logic_vector(7
8         downto 0);
9     constant rom: memory_16_8 := ("00000000", "00000001", "
10         00000010", "00000011", "00000100", "00000101", "00000110", "
11         00000111", "00001000", "00001001", "00001010", "00001011", "
12         00001100", "00001101", "00001110", "00001111");
13
14 begin
15
16 process (address)
17     begin
18         dout<=rom(TO_INTEGER(unsigned(address)));
19 end process;
20
21 end Behavioral;

```

Implementazione macchina:

```

1
2 entity macchina is
3     Port ( x : in STD_LOGIC_vector(7 downto 0);

```

```
4         y : out STD_LOGIC_vector(3 downto 0));
5     end macchina;
6
7     architecture Behavioral of macchina is
8
9     begin
10         process (x)
11         begin
12             case x is
13                 when "00000001"=>y<="0001";
14                 when "00000010"=>y<="0010";
15                 when "00000011"=>y<="0011";
16                 when "00000100"=>y<="0100";
17                 when others=>y<="0000";
18             end case;
19         end process;
20
21     end Behavioral;
```

Implementazione top module:

```
1     entity top_module is
2         Port (mem: in std_logic_vector(3 downto 0);
3              output: out std_logic_vector (3 downto 0));
4     end top_module;
5
6     architecture Structural of top_module is
7
8     signal u0: std_logic_vector (7 downto 0):=(others=>'0');
9
10    component rom
```

```
10      Port (address: in std_logic_vector(3 downto 0);
11            dout: out std_logic_vector(7 downto 0)); --dataout
12  end component;
13  component macchina
14      Port ( x : in STD_LOGIC_vector(7 downto 0);
15            y : out STD_LOGIC_vector(3 downto 0));
16  end component;
17
18  begin
19
20      rom_0: rom port map (address=>mem, dout=>u0);
21      macchina_0: macchina port map(x=>u0, y=>output);
22
23  end Structural;
```

1.4.4 Simulazione

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4
5  entity top_module_tb is
6
7  end top_module_tb;
8
9  architecture Behavioral of top_module_tb is
10
11  component top_module is
```

```
12 Port
13 (
14     mem:in std_logic_vector(3 downto 0);
15     output: out std_logic_vector(3 downto 0)
16
17 );
18 end component;
19
20 signal input :std_logic_vector(3 downto 0) :=(others=>'0');
21 signal output1 :std_logic_vector(3 downto 0) :=(others=>'0');
22
23 begin
24 uut: top_module
25     port map
26     (
27         mem=>input,
28         output=>output1
29     );
30 stim_proc: process
31 begin
32
33 wait for 100 ns;
34 input<="0001";
35
36 wait for 10 ns;
37 input<="0010";
38
39 wait for 10 ns;
```

```
40 input<="0100";
41
42 wait for 10 ns;
43 input<="1000";
44
45 wait;
46
47 end process;
48
49
50
51 end Behavioral;
```

Nel Testbench abbiamo verificato la correttezza del top module .In questo caso abbiamo avuto bisogno di: uno standard logic vector di dimensione 4 per gli ingressi (input), uno standard logic vector di dimensione rispettivamente 4 (output). In seguito abbiamo fornito 4 casi di test:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5 entity top_module_tb is
6
7 end top_module_tb;
8
9 architecture Behavioral of top_module_tb is
10
11 component top_module is
```

```
12 Port
13 (
14     mem:in std_logic_vector(3 downto 0);
15     output: out std_logic_vector(3 downto 0)
16
17 );
18 end component;
19
20 signal input :std_logic_vector(3 downto 0) :=(others=>'0');
21 signal output1 :std_logic_vector(3 downto 0) :=(others=>'0');
22
23 begin
24 uut: top_module
25     port map
26     (
27         mem=>input,
28         output=>output1
29     );
30 stim_proc: process
31 begin
32
33 wait for 100 ns;
34 input<="0001";
35
36 wait for 10 ns;
37 input<="0010";
38
39 wait for 10 ns;
```

```
40 input<="0100";  
41  
42 wait for 10 ns;  
43 input<="1000";  
44  
45 wait;  
46  
47 end process;  
48  
49 end Behavioral;
```

1.4.5 Esercizio 2.2

Sintetizzare ed implementare su board il progetto del sistema ROM+M sviluppato al punto 2.1, utilizzando gli switch per fornire l'indirizzo della ROM da cui leggere i valori da trasformare e i led per visualizzare i 4 bit di uscita.

1.4.6 Progetto e Architettura

Partendo dall'esercizio 2.1 abbiamo implementato e simulato un codice che ci permette di visualizzare sulla board (NEXYS A7) il funzionamento della ROM. Sono stati utilizzati gli switch(con precisione 4) per acquisire l'ingresso. Per visionare l'uscita invece, è stato utilizzato un led. Essendo la ROM un componente puramente combinatorio per l'implementazione necessitiamo soltanto di un top module e non di una Control Unit.

1.4.7 Implementazione

Il top module è stato implementato nel seguente modo:

```
1 entity top_module is
2     Port ( switch: in std_logic_vector(3 downto 0);
3           led: out std_logic_vector(3 downto 0));
4 end top_module;
5
6 architecture Structural of top_module is
7     component rom
8     Port (address: in std_logic_vector(3 downto 0);
9           dout: out std_logic_vector(7 downto 0)); --dataout
10 end component;
11
12     component macchina
13     Port ( x : in STD_LOGIC_vector(7 downto 0);
14           y : out STD_LOGIC_vector(3 downto 0));
15 end component;
16
17     signal output_rom: std_logic_vector(7 downto 0) :=(others
18     =>'0');
19 begin
20     ROM0: rom port map(switch,output_rom);
21     Macchina0: macchina port map(output_rom,led);
22 end Structural;
```

1.4.8 Simulazione

Di seguito mostriamo la simulazione sulla board NEXYS A7-100T:

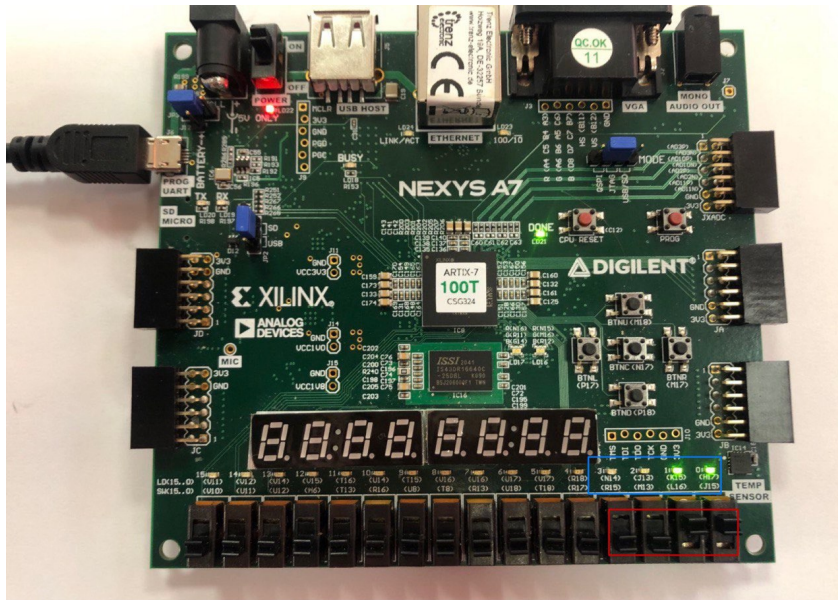


Figura 1.9: Output corrispondente all'ingresso 0011

Tramite switch, evidenziati in rosso, acquisiamo la locazione della memoria ROM, il cui contenuto viene fornito in input alla macchina combinatoria che produce un'uscita di lunghezza 4 bit (mostrati dai led evidenziati in blu). Output del secondo caso:

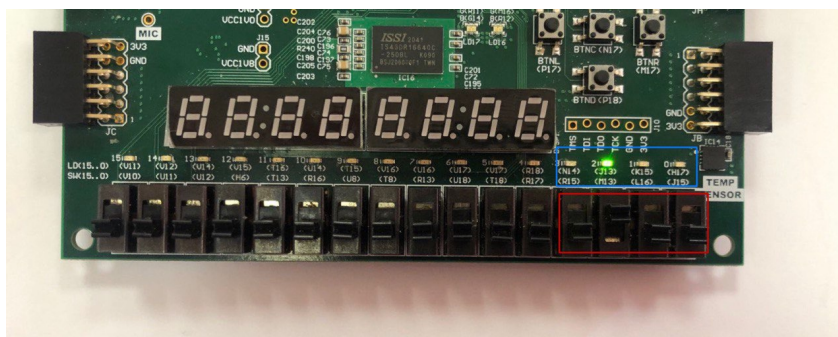


Figura 1.10: Output corrispondente all'ingresso 0100

Capitolo 2

Reti sequenziali elementari

2.1 Esercizio 3: Riconoscitore di sequenze

2.1.1 Esercizio 3.1

Progettare, implementare in VHDL e testare mediante simulazione una macchina in grado di riconoscere la sequenza 101. La macchina prende in ingresso un segnale binario i che rappresenta il dato, un segnale A di tempificazione e un segnale M di modo, che ne disciplina il funzionamento, e fornisce un'uscita Y alta quando la sequenza viene riconosciuta. In particolare,

- se $M=0$, la macchina valuta i bit seriali in ingresso a gruppi di 3 (sequenze non sovrapposte),
- se $M=1$, la macchina valuta i bit seriali in ingresso uno alla volta, tornando allo stato iniziale ogni volta che la sequenza viene correttamente riconosciuta (sequenze parzialmente sovrapposte).

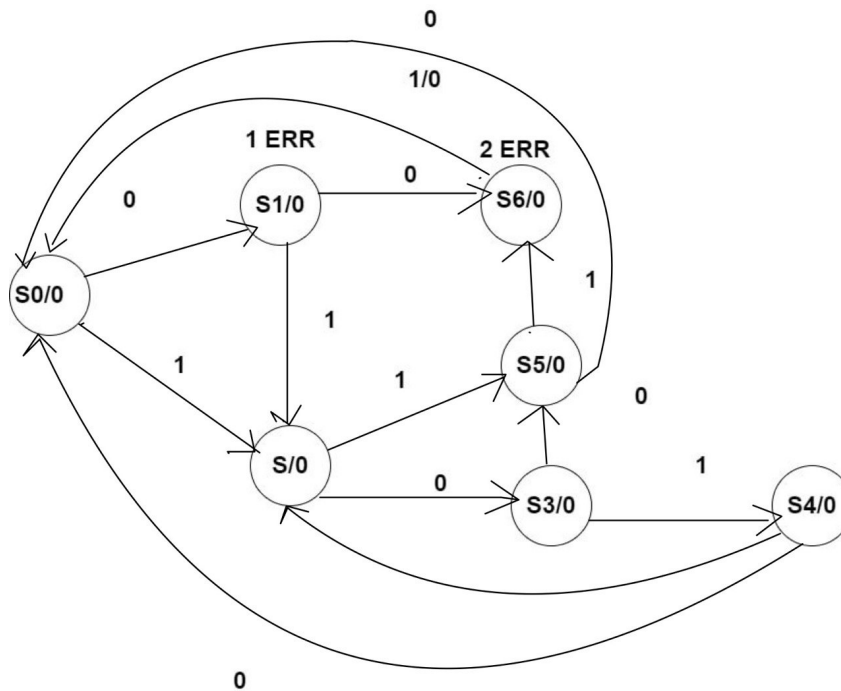


Figura 2.1:

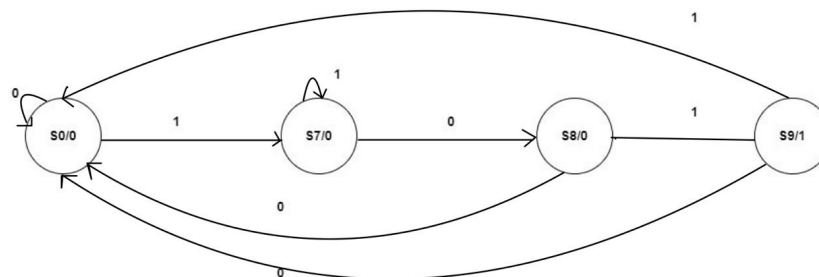


Figura 2.2:

2.1.2 Progetto e Architettura

Per poter implementare un riconoscitore di sequenze, come prima cosa, abbiamo realizzato l'automa a stati finiti di Moore. In un automa di Moore l'uscita è solo in funzione dello stato. Abbiamo realizzato due riconoscitori di sequenza '101': uno non sovrapposto e uno parzialmente sovrapposto. NON SOVRAPPOSTO: PARZIALMENTE SOVRAPPOSTO:

Una volta realizzato l'automa questo ci darà le linee guida per l'imple-

mentazione. Nel nostro source come prima cosa abbiamo indicato:

- ingresso(input) uno std_logic di input;
- uscita(output)std_logic di output;
- clock(clk)std_logic di input;
- reset(rst) std_logic di input;
- modo(mode che sarebbe non sovrapposto o parzialmente sovrapposto)
std_logic di input

Successivamente abbiamo indicato i nostri 10 stati (S0,S1,S2,S3,S4,S5,S6,S7,S8,S9). In più abbiamo indicato due variabili di tipo stato : stato corrente e stato prossimo. Una volta indicati gli elementi abbiamo realizzato un process combinatorio che in base al mode(0 o 1), lo stato corrente e l'input(0 o 1) assegna lo stato prossimo(stato_prossimo<=stato_corrente) e un process sequenziale che in corrispondenza dei fronti di salita del clock ci fa passare dallo stato corrente allo stato prossimo. Grazie ad essi esploriamo tutte le possibili combinazioni(in funzione dell'ingresso) indicandone le corrispettive uscite.

2.1.3 Implementazione

Implementazione del riconoscitore:

```
1  entity riconoscitore is
2      Port (input: in std_logic ;
```

```
3      clk: in std_logic ;
4      mode: in std_logic;
5      rst: in std_logic;
6      output: out std_logic );
7 end riconoscitore;
8
9 architecture Behavioral of riconoscitore is
10     type stato is (S0,S1,S2,S3,S4,S5,S6,S7,S8,S9);
11
12     signal stato_corrente: stato:= S0;
13     signal stato_prossimo: stato;
14
15
16 begin
17
18     stato_uscita: process (stato_corrente,input)
19     begin
20         case stato_corrente is
21             when S0 =>
22                 if mode = '0' then
23                     if input = '1' then
24                         stato_prossimo <= S2;
25                         output <= '0';
26                     else
27                         stato_prossimo <= S1;
28                         output <= '0';  --primo errore
29                     end if;
30                 elsif mode = '1' then
```

```
31      if input='1' then
32          stato_prossimo <= S7;
33          -- il riconoscitore passa allo stato S7
34          output <= '0';
35      else
36          stato_prossimo <= S0; --input 0
37          output <= '0';
38      end if;
39  end if;
40  when S1 =>
41
42      if input = '1' then
43          stato_prossimo <= S2;
44          output <= '0';
45      else
46          stato_prossimo <= S6; --errore
47          output <= '0';
48      end if;
49
50  when S2 =>
51
52      if input = '1' then
53          stato_prossimo <= S5; --errore1
54          output <= '0';
55      else
56          stato_prossimo <= S3;
57          output <= '0';
58      end if;
```

```
59
60
61     when S3 =>
62
63         if input = '1' then
64             stato_prossimo <= S4; --sequenza
65                                     riconosciuta
66             output <= '0';
67         else
68             stato_prossimo <= S5; -- 1 errore
69             output <= '0';
70         end if;
71
72     when S4 =>
73
74         if input = '1' then
75             stato_prossimo <= S2;
76             output <= '1';
77         else
78             stato_prossimo <= S0;
79             output <= '1';
80         end if;
81
82     when S5 =>
83
84         if input = '1' then
85             stato_prossimo <= S6; -- errore 2
```

```
86         output <= '0';
87     else
88         stato_prossimo <= S0;
89         output <= '0';
90     end if;
91
92     when S6 => -- ho sempre due errori
93
94         if input = '1' then
95             stato_prossimo <= S0;
96             output <= '0';
97         else
98             stato_prossimo <= S0;
99             output <= '0';
100         end if;
101     when S7 =>
102         if input = '1' then
103             stato_prossimo <= S7;
104             output <= '0';
105         else
106             stato_prossimo <= S8;
107             output <= '0';
108         end if;
109     when S8 =>
110         if input = '1' then
111             stato_prossimo <= S9;
112             output <= '0';
113         else
```

```
114         stato_prossimo <= S0;
115         output <='0';
116     end if;
117     when S9 =>
118         if input ='1' then
119             stato_prossimo <=S7;
120             output <='1';
121         else
122             stato_prossimo <= S0;
123             output <='1';
124         end if;
125
126     end case;
127 end process;
128
129 -- gestione tempificazione
130
131 mem: process (clk)
132 begin
133     if(rising_edge (clk)) then
134         if (rst='1') then
135             stato_corrente <=S0;--reset asincrono
136         else
137             stato_corrente <= stato_prossimo;
138         end if;
139     end if;
140
141 end process;
```

```
142  
143  
144  
145 end Behavioral;
```

Implementazione del top module:

```
1 entity top_module is  
2   Port (  
3     switch:  in std_logic;  
4     mode:    in std_logic;  
5     reset:   in std_logic;  
6     clock:   in std_logic;  
7     led:     out std_logic  
8   );  
9 end top_module;  
10  
11 architecture Behavioral of top_module is  
12   component clock_divider  
13     generic(  
14       clock_frequency_in : integer := 100000000;--  
15         100MHz  
16       clock_frequency_out : integer := 1-- Hz--c'era  
17         1 qui  
18     );  
19   Port ( clock_in : in  STD_LOGIC;  
20         reset  : in  STD_LOGIC;  
21         clock_out : out STD_LOGIC);  
22 end component;  
23  
24   component riconoscitore
```

```
22      Port (input: in std_logic ;
23            clk: in std_logic ;
24            mode: in std_logic;
25            rst: in std_logic;
26            output: out std_logic );
27 end component;
28 signal clock_effective: std_logic := '0';
29 begin
30
31     clock_divider0: clock_divider port map(clock, reset,
32                                           clock_effective);
33
34     riconoscitore0: riconoscitore port map(switch,
35                                           clock_effective, mode, reset, led);
36
37 end Behavioral;
```

2.1.4 Simulazione

Nel Testbench abbiamo constatato la correttezza dell'architettura progettata in precedenza. In particolare, abbiamo testato entrambi i modi di funzionamento, fornendo in input delle sequenze che contenessero la sequenza ricercata, ovvero "101". Di seguito mostriamo l'implementazione del testbench:

```
1 library IEEE; use IEEE.STD_LOGIC_1164.ALL;
```

```
2
```

```
3 entity riconoscitore_tb is-- Port ( );
4 end riconoscitore_tb;
5 architecture Behavioral of riconoscitore_tb is
6     component riconoscitore is
7         port (
8             input : in STD_LOGIC;
9             clk : in STD_LOGIC;
10            rst : in STD_LOGIC;
11            mode : in STD_LOGIC;
12            output : out STD_LOGIC);
13        end component;
14 --Input
15 signal ingresso: std_logic:='0';
16 signal Modo: std_logic:='0';
17 signal clock: std_logic:='0';
18 signal reset_clock: std_logic:='0';--Output
19 signal uscita: std_logic;
20 --costanti utilizzate
21 constant clock_period: time := 10ns;
22 begin
23 uut:riconoscitore port map(
24     input=>ingresso,
25     clk=>clock,
26     rst=>reset_clock,
27     mode=>Modo,
28     output=>uscita
29 );
30 CLK_process: process
```

```
31 begin
32     clock<='0';
33     wait for clock_period/2;
34     clock<='1';
35     wait for clock_period/2;
36     end process;
37 --SIMULAZIONE TB
38 stim_process: process
39     begin
40         wait for 100 ns;
41         Modo<='1';
42         ingresso<='1';
43         wait for 10 ns;
44         ingresso<='0';
45         wait for 10 ns;
46         ingresso<='1';
47         wait for 10 ns;
48         ingresso<='0';
49         wait for 10 ns;
50         ingresso<='0';
51
52         wait for 10 ns;
53         Modo<='0';
54         ingresso<='1';
55         wait for 10ns;
56         ingresso<='0';
57         wait for 10 ns;
58         ingresso<='1';
```

```
59     wait for 10 ns;  
60     ingresso<='1';  
61     wait for 10 ns;  
62     ingresso <= '0';  
63     wait for 10 ns;  
64  
65     ingresso <= '1';  
66     wait for 100 ns;  
67     ingresso<='1';  
68     reset_clock<='1';  
69     wait;  
70 end process;  
71 end Behavioral;
```

L'output della simulazione, con tali input, è il seguente:

:

2.2 Esercizio 3.2

Sintetizzare e implementare su board la rete sviluppata al punto precedente, utilizzando uno switch S1 per codificare l'input i e uno switch S2 per codificare il modo M, in combinazione con due bottoni B1 e B2 utilizzati rispettivamente per acquisire l'input da S1 e S2 in sincronismo con il segnale di temporizzazione A, che deve essere ottenuto a partire dal clock della board. Infine, l'uscita Y può essere codificata utilizzando un led.

2.2.1 Progetto e Architettura

Sulla base dell'esercizio 3.1 abbiamo implementato un codice che ci consente di acquisire mediante due switch l'input e il modo

2.3 Esercizio 4: Shift Register

Progettare, implementare in VHDL e testare mediante simulazione un registro a scorrimento di N bit in grado di shiftare a destra o a sinistra di un numero Y variabile di posizioni a seconda di una opportuna selezione. In particolare, i valori possibili di Y sono 1 e 2. L'utente tramite selezione deve scegliere di quante posizioni shiftare. Il componente deve essere realizzato utilizzando sia un a) approccio comportamentale sia un b) approccio strutturale.

Nota: il numero di bit del registro deve essere implementato come un generic, e dall'esterno deve poter essere scelta la modalità di funzionamento mediante opportuni segnali di selezione.

2.3.1 Approccio Comportamentale

Progetto e Architettura

Per la realizzazione dello shift register (behavioral) abbiamo utilizzato un registro dati di N bit e, tramite l'operatore di concatenazione in VHDL, abbiamo implementato la logica dello scorrimento. Come richiesto dalla traccia, il registro, grazie all'utilizzo di opportuni input di controllo, permette di shiftare il suo contenuto a destra oppure a sinistra (shift_direction) di una o due posizioni(shift_amount).

Implementazione

Implementazione shift register (behavioral):

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.STD_LOGIC_ARITH.ALL;
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6
7 -- Uncomment the following library declaration if using
8 -- arithmetic functions with Signed or Unsigned values
9 --use IEEE.NUMERIC_STD.ALL;
10
```

```
11 -- Uncomment the following library declaration if
    instantiating
12 -- any Xilinx leaf cells in this code.
13 --library UNISIM;
14 --use UNISIM.VComponents.all;
15
16 entity shift_reg is
17     generic(
18         N: integer := 8 --numero di bit a reg
19     );
20     Port (
21         clk: in std_logic ;
22         rst: in std_logic ;
23         shift_direction : in std_logic; --0 a destra 1 a
            sinistra
24         shift_amount : in std_logic; --numero di posiszioni di
            shift
25
26         ingresso: in std_logic ;
27         dout : out std_logic_vector(N-1 downto 0)
28     );
29 end shift_reg;
30
31 architecture Behavioral of shift_reg is
32     signal temp : std_logic_vector(N-1 downto 0);
33     signal temp1: std_logic_vector(N-1 downto 0);
34     --signal s: std_logic ;
35
```

```
36 begin
37     process (clk,rst)
38     begin
39         if rst = '1' then
40             temp1<=(others => '0');
41
42
43             temp<=(others => '0');
44         elsif rising_edge(clk) then
45             if shift_amount= '0' and shift_direction = '0'
46                 then --shifta a destra
47
48                 temp<=ingresso & temp(N-1 downto 1) ;
49
50             elsif shift_amount= '0' and shift_direction = '1'
51                 then
52
53                 temp<= temp(N-2 downto 0) & ingresso;
54             elsif shift_amount= '1' and shift_direction = '0'
55                 then
56
57                 --temp1<= ingresso & temp(N-1 downto 1) ;
58                 temp<=ingresso&ingresso & temp1(N-1 downto
59                     2) ;
```

```
        then
60
61            temp<=temp(N-3 downto 0) & ingresso & ingresso;
62
63        end if;
64        end if;
65
66
67
68
69
70
71
72        end process;
73        dout<= temp;
74
75
76    end Behavioral;
```

Simulazione

Nel Testbench abbiamo constatato la correttezza dell'architettura progettata in precedenza. In particolar modo abbiamo provato tutte le possibili modalità di funzionamento:

- shift di un bit a destra
- shift di un bit a sinistra

- shift di due bit a destra
- shift di due bit a sinistra
- reset

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.STD_LOGIC_ARITH.ALL;
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6 -- Uncomment the following library declaration if using
7 -- arithmetic functions with Signed or Unsigned values
8 --use IEEE.NUMERIC_STD.ALL;
9
10 -- Uncomment the following library declaration if
    instantiating
11 -- any Xilinx leaf cells in this code.
12 --library UNISIM;
13 --use UNISIM.VComponents.all;
14
15 entity shift_reg_tb is
16     -- Port ( );
17 end shift_reg_tb;
18
19 architecture Behavioral of shift_reg_tb is
20     component shift_reg is
21         generic(
22             N: integer := 8 --numero di bit a reg
23         );
```

```
24     Port (
25         clk: in std_logic ;
26         rst: in std_logic ;
27         shift_direction : in std_logic; --0 a destra 1 a
           sinistra
28         shift_amount : in std_logic; --numero di posiszioni di
           shift
29
30         ingresso: in std_logic ;
31         dout : out std_logic_vector(N-1 downto 0)
32     );
33 end component;
34
35 signal clock,reset,dir,ing,s_a:std_logic:='0';
36
37 signal data_out : std_logic_vector(3 downto 0):=(others=>'0');
38
39 constant clock_period : time:=10 ns;
40 begin
41     CLk_process: process
42     begin
43         clock<='0';
44         wait for clock_period/2;
45         clock<='1';
46         wait for clock_period/2;
47     end process;
48
49 utt: shift_reg generic map(4) port map(clock,reset,dir,s_a,ing
```

```
    , data_out);  
50  
51 stim_proc: process  
52  
53  
54  
55 begin  
56  
57 wait for 10ns;  
58 reset<='1';  
59 wait for 10ns;  
60 reset<='0';  
61 --caso1: dx di 1  
62 ing<='1';  
63 dir<='0';  
64 s_a<='0';  
65  
66 --caso2: sx di 1  
67 wait for 10ns;  
68 ing<='1';  
69 dir<='1';  
70 s_a<='0';  
71  
72 --caso3: dx di 2  
73 wait for 10ns;  
74 ing<='1';  
75 s_a<='1';  
76 dir<='0';
```

```
77
78 --caso4 sx di 2
79 wait for 10ns;
80 ing<='1';
81 s_a<='1';
82 dir<='1';
83 wait for 10ns;
84
85
86 reset<='1';
87
88
89
90 --sinistra di due
91
92
93 wait for 1000ns;
94 --wait;
95 end process;
96
97 end Behavioral;
```

l'output relativo al precedente test è:

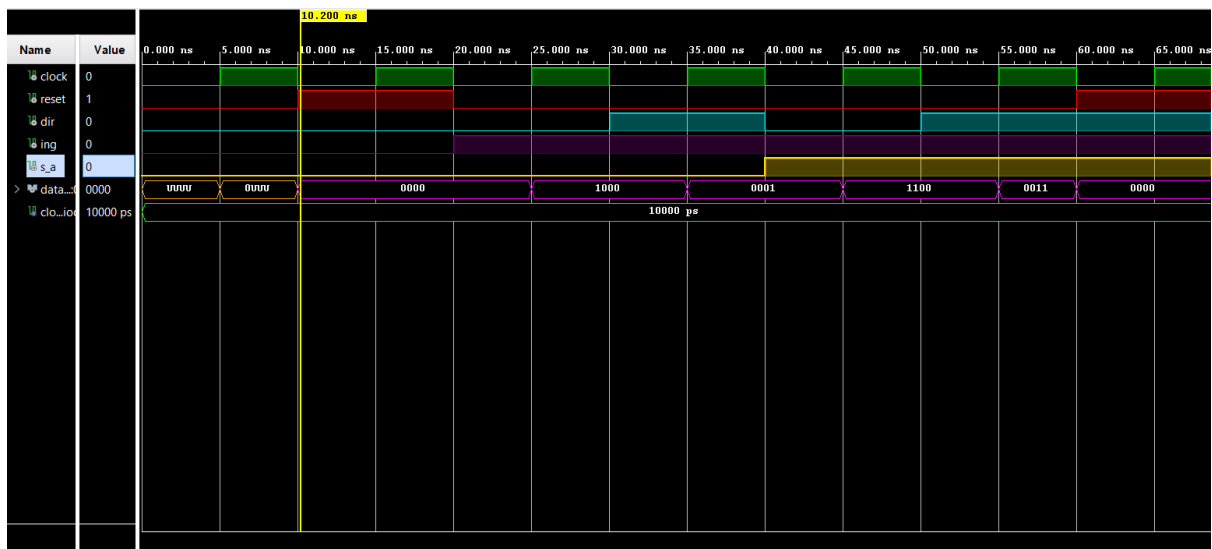


Figura 2.3: simulazione esercizio 4

2.3.2 Approccio strutturale

Progettazione e Architettura

L'approccio strutturale prevede l'implementazione dello shift register partendo da componenti elementari. Tali elementi, nel nostro caso, sono N multiplexer 4:1 ed N flip-flop di tipo D in cascata .

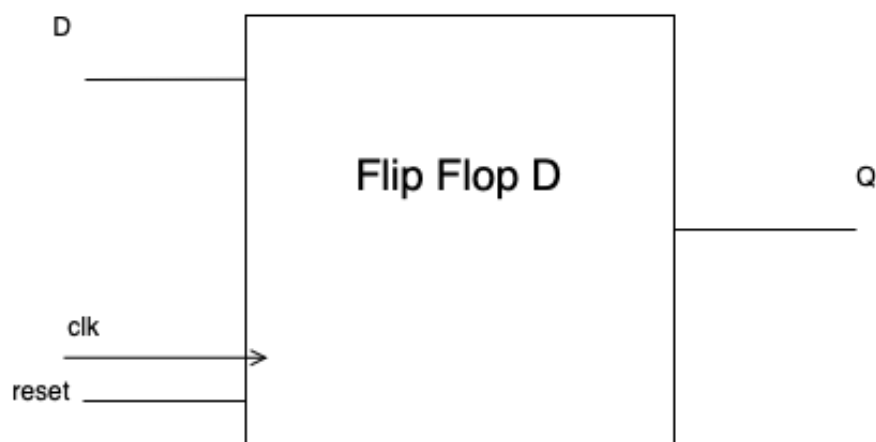


Figura 2.4: Schema a blocchi Flip Flop D

La scelta del flip flop D è legata alla sua logica di funzionamento, infatti esso prevede che dato un certo ingresso sincronizzato sul fronte di clock (nel nostro caso un fronte di salita) esso si propaghi in uscita e rimane stabile fino a quando tale ingresso non è modificato. La nostra scelta architetturale prevede in primis la creazione di un blocco composto da un multiplexer 4:1 ed un Flip flop D, questi sono collegati tra loro. In particolare l'uscita del multiplexer sarà data in ingresso al flip flop.

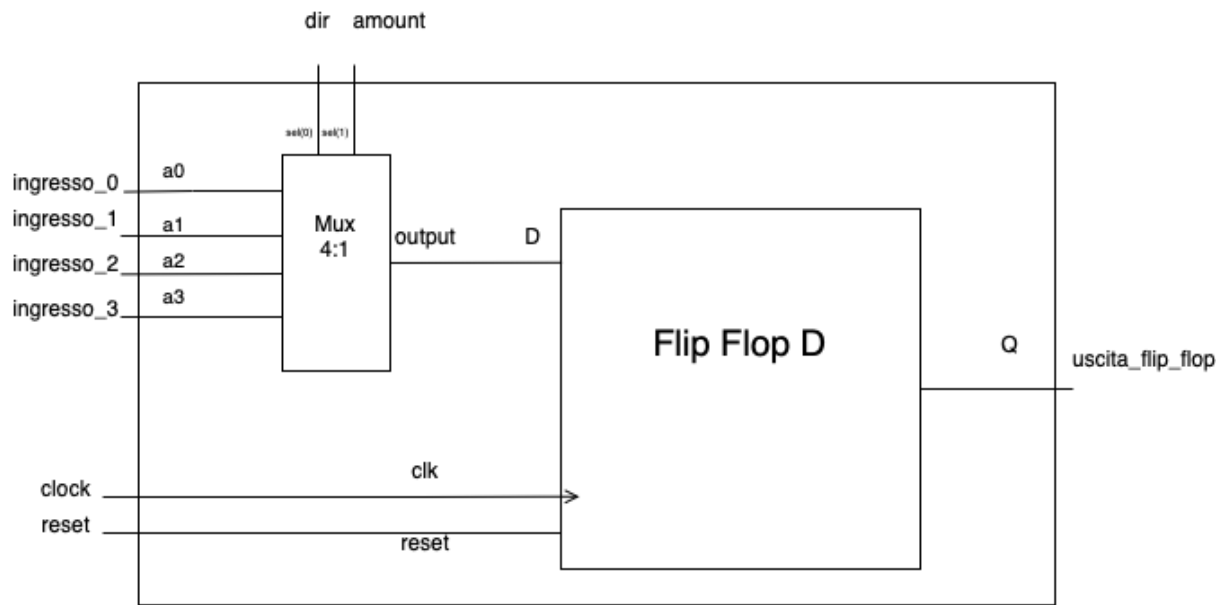


Figura 2.5: Schema a blocchi dell' architettura composta da Mux ed FF

Questi blocchi saranno poi istanziati e interconnessi in serie tra loro.

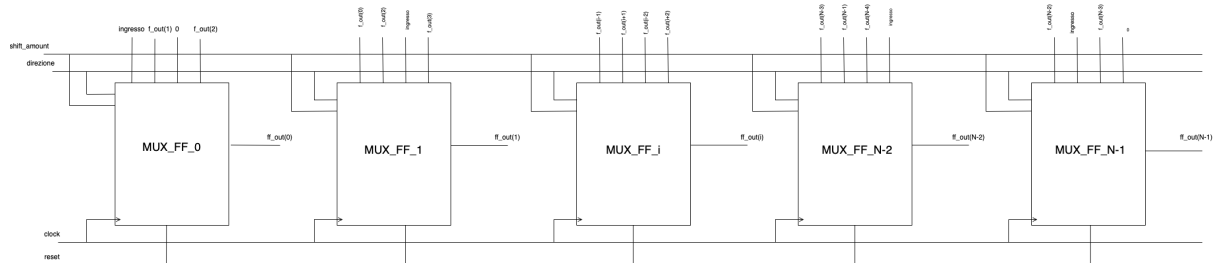


Figura 2.6: Schema a blocchi dello Shift Register

Implementazione

Implementazione flip flop D:

```

1 library IEEE; use IEEE.STD_LOGIC_1164.ALL;
2 entity D_FlipFlop is
3     port (
4         D, clk, reset : in std_logic;
5         Q : out std_logic );

```

```

5 end entity D_FlipFlop;
6 architecture Behavioral of D_FlipFlop is      signal Q_int :
    std_logic := '0';
7     begin      process (clk, reset)
8         begin      if reset = '1' then
9             Q_int <= '0';  -- Reset          elsif
                rising_edge(clk) then
10                Q_int <= D;      -- Memorizza il valore di
                    D al fronte di salita del clock
                        end if;
11        end process;
12    Q <= Q_int; end architecture Behavioral;

```

L'implementazione del multiplexer è fedele a quella del primo esercizio.

Implementazione blocco Mux_FF:

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5
6 entity Mux_FF is
7     Port ( ingresso_0: in std_logic;
8           ingresso_1: in std_logic;
9           ingresso_2: in std_logic;
10          ingresso_3: in std_logic;
11          clock : in std_logic;
12          reset: in std_logic;
13          dir : in std_logic;

```

```
14         amount :in std_logic;
15         uscita_flip_flop : out std_logic
16     );
17 end Mux_FF;
18
19 architecture Behavioral of Mux_FF is
20
21
22 component Mux_4_1 is
23     Port (
24         a0 : in STD_LOGIC;
25         a1 : in STD_LOGIC;
26         a2 : in STD_LOGIC;
27         a3 : in STD_LOGIC;
28         sel : in STD_LOGIC_VECTOR(0 to 1);
29         Output : out STD_LOGIC
30     );
31 end component;
32
33 component D_FlipFlop is
34     port (
35         D, clk, reset : in std_logic;
36         Q : out std_logic
37     );
38 end component;
39 signal selezione: std_logic_vector(0 to 1):=(others=>'0');
40 signal mux_output: std_logic:='0';
41
```

```

42 begin
43 selezione<=amount&dir;
44
45 mux: Mux_4_1 port map(ingresso_0,ingresso_1,ingresso_2,
    ingresso_3,selezione,mux_output);
46 FLIP_FLOP :D_FlipFlop port map(mux_output,clock,reset,
    uscita_flip_flop);
47
48
49
50
51 end Behavioral;

```

Implementazione Shift Register strutturale:

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity Top_module is
5 generic(
6     N:integer := 8
7 );
8 port (
9     clock,reset: in std_logic;
10    ingresso: in std_logic;
11    direzione: in std_logic;
12    shift_amount: in std_logic;-- se 0 shift singolo se 1
    shift doppio
13    registro: out std_logic_vector(0 to N-1)
14 );

```

```
15 end Top_module;
16 architecture structural of Top_module is
17
18 component Mux_FF is
19     Port ( ingresso_0: in std_logic;
20           ingresso_1: in std_logic;
21           ingresso_2: in std_logic;
22           ingresso_3: in std_logic;
23           clock : in std_logic;
24           reset: in std_logic;
25           dir : in std_logic;
26           amount : in std_logic;
27           uscita_flip_flop : out std_logic
28         );
29 end component;
30
31 signal f_out: std_logic_vector(0 to N-1) := (others => '0');
32
33 begin
34
35     MUX_FF_0 : Mux_ff port map(ingresso, f_out(1), '0', f_out(2),
36                               clock, reset, direzione, shift_amount, f_out(0));
37
38     MUX_FF_1: Mux_ff port map(f_out(0), f_out(2), ingresso,
39                               f_out(3), clock, reset, direzione, shift_amount, f_out(1));
40
41     MUX_FF_N_1 : Mux_ff port map(f_out(N-2), ingresso, f_out(N-3)
42                               , '0', clock, reset, direzione, shift_amount, f_out(N-1));
```

```
40
41 MUX_FF_N_2 : Mux_ff port map (f_out (N-3), f_out (N-1), f_out (N
    -4), ingresso, clock, reset, direzione, shift_amount, f_out (N-2)
    );
42
43
44
45 Mux_FF_interni: for i in 2 to N-3 generate
46 Mux_FF_i: Mux_ff port map (f_out (i-1), f_out (i+1), f_out (i-2)
    , f_out (i+2), clock, reset, direzione, f_out (i));
47
48 end generate ; -- identifier
49
50 registro<=f_out ;
51
52
53
54 end Structural;
```

Quest'ultimo frammento di codice è il TOP module dello shift register.

Ad multiplexer interni verranno forniti i rispettivi ingressi: uscita F_F precedente, uscita F_F successivo, uscita F_F con offset di 2 a sinistra rispetto al corrente, uscita F_F con offset di 2 a destra rispetto al corrente. La logica è diversa per i due più a sinistra e i due più a destra dei quali sono assegnato gli input del sistema. Inoltre sono forniti due segnali di selezione (amount e dir) che indicano segnali da propagare in uscita.

Simulazione

Per testare il funzionamento è stato implementato tale test bench:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity Top_module_TB is
5 -- Port ( );
6 end Top_module_TB;
7
8 architecture Behavioral of Top_module_TB is
9
10 component Top_module is
11
12 generic(
13     N:integer := 8
14 );
15 port (
16     clock,reset: in std_logic;
17     ingresso: in std_logic;
18     direzione: in std_logic;
19     shift_amount: in std_logic;-- se 0 shift singolo se 1
20     shift doppio
21     registro: out std_logic_vector(0 to N-1)
22 );
23
24 end component;
25
26 signal clk,rst,input,dir,amount: std_logic:='0';
27 signal reg:std_logic_vector(0 to 3):=(others=>'0');
```

```
26 constant clock_period :time :=10 ns;
27
28 begin
29
30
31 CLK_process: process
32 begin
33     clk<='0';
34     wait for clock_period/2;
35     clk<='1';
36     wait for clock_period/2;
37 end process;
38
39
40 UUT: Top_module generic map(4) port map(clk,rst,input,dir,
41     amount,reg);
42
43 sim_proc: process
44 begin
45
46 wait for 100 ns;
47
48 rst<='1';
49
50 wait for 10 ns;
51
52 rst<='0';
```

```
53 input<='1';
54 dir<='0';
55 amount<='1';
56
57 wait for 10 ns;
58 dir<='1';
59 input<='1';
60 amount<='0';
61
62 wait for 10 ns;
63 dir<='1';
64 amount<='1';
65 input<='1';
66
67 wait for 10 ns;
68 rst<='1';
69
70 wait for 10 ns;
71
72
73 wait ;
74 end process;
75
76
77
78 end Behavioral;
```

Gli input del sistema sono la direzione dir(0 destra 1 sinistra) e amount che

mi dice di quanti bit shiftare(0 un solo bit 1 due bit).

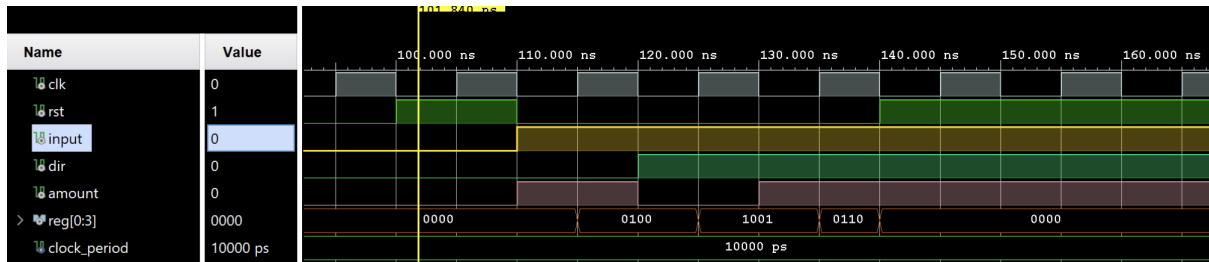


Figura 2.7: Output simulazione

2.4 Esercizio 5: Cronometro

2.4.1 Esercizio 5.1

Progettare, implementare in VHDL e testare mediante simulazione un cronometro, in grado di scandire secondi, minuti e ore a partire da una base dei tempi prefissata (es. si consideri il clock a disposizione sulla board). Il progetto deve prevedere la possibilità di inizializzare il cronometro con un valore iniziale, sempre espresso in termini di ore, minuti e secondi, mediante un opportuno ingresso di set, e deve prevedere un ingresso di reset per azzerare il tempo. Il componente deve essere realizzato utilizzando un approccio strutturale, collegando opportunamente dei contatori secondo uno schema a scelta.

Progetto e Architettura

Il nostro scopo è quello di implementare un cronometro che conti secondi, minuti ed ore. Per poter fare ciò abbiamo bisogno di 3 contatori collegati in

serie tra di loro. Quest' ultimi saranno sincronizzati sul fronte di salita di un solo clock dato in ingresso al primo contatore(contatore dei secondi).

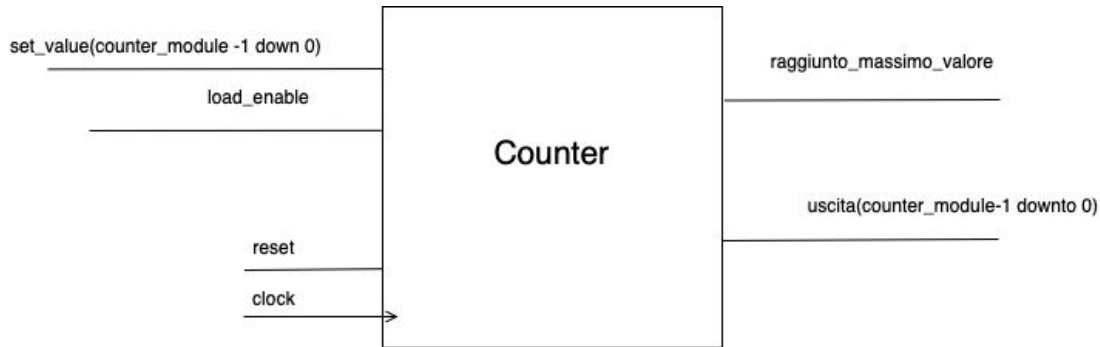


Figura 2.8: schema a blocchi di un contatore

Ogni contatore riceverà in ingresso un segnale di reset, un segnale di clock, un segnale di load per abilitare la memorizzazione ed un segnale set value che permette di inizializzare il contatore ad un valore diverso dallo zero. Inoltre sono presenti due segnali di output: raggiunto valore massimo (60 per secondi e minuti) e l' uscita che invece custodisce il valore corrente del contatore.

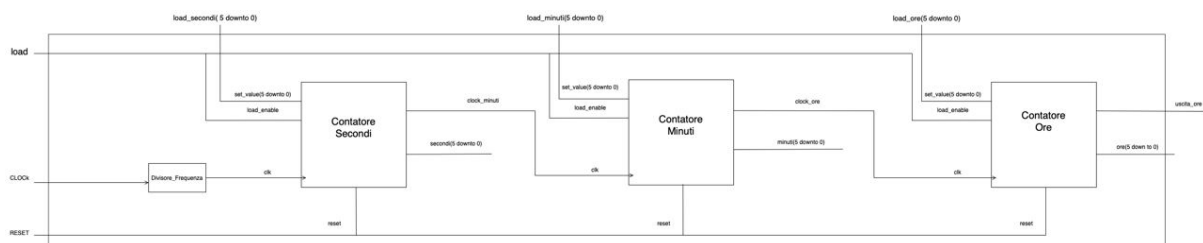


Figura 2.9: schema a blocchi del cronometro

Come già detto, soltanto il primo contatore riceve in ingresso il clock opportunamente prescalato(per scandire i secondi), i successivi riceveranno il segnale di clock alto in corrispondenza dell'uscita "raggiunto valore massimo" alta del contatore precedente nella serie.

Implementazione

Implementazione contatore:

```
1 library IEEE;use IEEE.STD_LOGIC_1164.ALL;
2 use IEEE.numeric_std.all;
3 -- Uncomment the following library declaration if using--
   arithmetic functions with Signed or Unsigned values
4 --use IEEE.NUMERIC_STD.ALL;
5 -- Uncomment the following library declaration if
   instantiating-- any Xilinx leaf cells in this code.
6 --library UNISIM;--use UNISIM.VComponents.all;
7 entity counter is
8     generic      (
9         counter_module: integer;          counter_max_value:
           integer
10    );
11    Port ( clk : in STD_LOGIC;
12          reset : in STD_LOGIC;          load_enable : in
           STD_LOGIC;
13          set_value : in std_logic_vector(counter_module-1
           downto 0);          uscita : out
           STD_LOGIC_vector(counter_module-1 downto 0);
```

```
14         raggiunto_valore_massimo : out STD_LOGIC
15         );
16 end counter;
17 architecture Behavioral of counter is
18     signal internal_value: std_logic_vector(counter_module -1
19         downto 0) := (others=>'0');
20 begin
21     aggiorna_conteggio: process(clk, reset, load_enable)
22     begin if reset = '1' then
23         internal_value<=(others=>'0');
24         raggiunto_valore_massimo<='0'; elsif load_enable='1' then
25         internal_value<=set_value; if (unsigned(set_value)>=
26             counter_max_value) then
27             raggiunto_valore_massimo<='1'; end if;
28         elsif (clk='0' and clk'event) then if (unsigned(
29             internal_value)>=counter_max_value) then
30             internal_value<=(others=>'0');
31             raggiunto_valore_massimo<='0'; elsif(
32             unsigned( internal_value)=counter_max_value-1)
33             then
34             raggiunto_valore_massimo<='1';
35             internal_value<=std_logic_vector(unsigned(
36                 internal_value)+1);--modifico qui else
37             raggiunto_valore_massimo<='0';
38             internal_value<=std_logic_vector(unsigned(
39                 internal_value)+1);
40         end if; end if;
41     end process; uscita<=internal_value;
```

```
end Behavioral;
```

Implementazione cronometro:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity Top_module is
5 Port (
6     clock:in  std_logic;
7     reset: in std_logic;      load: in std_logic;
8     load_seondi:in std_logic_vector(5 downto 0);
9     load_minuti: in std_logic_vector(5 downto 0);
10    load_ore: in std_logic_vector(5 downto 0);
11    secondi: out std_logic_vector(5 downto 0);
12    minuti:out std_logic_vector(5 downto 0);      ore:out
13    std_logic_vector(5 downto 0);
14    uscita_ore: out std_logic
15    );
16 end Top_module;
17
18 architecture Behavioral of Top_module iscomponent counter is
19     generic      (
20         counter_module: integer;
21         counter_max_value: integer
22     );
23     Port ( clk : in STD_LOGIC;      reset : in STD_LOGIC;
24         load_enable : in STD_LOGIC;      set_value :
25             in std_logic_vector(counter_module-1 downto 0);
26         uscita : out STD_LOGIC_vector(counter_module-1
```

```

        downto 0);          raggiunto_valore_massimo :
        out STD_LOGIC

23    );end component;
24 component divisore_di_frequenza is
25     generic (          CLKIN_freq : integer := 100000000; --
        Frequenza del clock di ingresso (100 MHz)
26     CLKOUT_freq : integer := 1          -- Frequenza
        desiderata del clock di uscita (1 Hz)      );
27     Port (
28         clock_in : in  STD_LOGIC;
29         reset : in STD_LOGIC;
30         clock_out : out STD_LOGIC
31     );end component;
32
33 signal clock_secondi: std_logic:='0';
34 signal clock_minuti,clock_ore: std_logic:='0'; signal MIN,SEC,
        HOUR : std_logic_vector(5 downto 0):=(others=>'0');
35 signal data_out_min,data_out_ore,data_out_sec:
        std_logic_vector(7 downto 0):=(others=>'0'); signal data_in :
        std_logic_vector(31 downto 0):=(others=>'0');
36 begin
37 secondi<= SEC;
38 minuti<= MIN;
39 ore <= HOUR;
40
41 CONTATORE_SECONDI: counter generic map(6,59) port map(CLOCK,
        RESET,load,load_secondi,SEC,clock_minuti);--
42 CONTATORE_MINUTI: counter generic map(6,59) port map(

```

```

        clock_minuti, RESET, load, load_minuti, MIN, clock_ore);
43 CONTATORE_ORE: counter generic map (6, 23)
44 port map (clock_ore, RESET, load, load_ore, HOUR, uscita_ore);
45
46 end Behavioral;

```

Simulazione

In seguito è riportato un Tb per poter testare tutte le funzionalità precedentemente implementate:

```

1 library IEEE; use IEEE.STD_LOGIC_1164.ALL;
2
3 entity TOP_MODULE_TB is -- Port ( );
4 end TOP_MODULE_TB;
5 architecture Behavioral of TOP_MODULE_TB is
6 component Top_module isPort (
7     clock: in std_logic;          reset: in std_logic;
8     load: in std_logic;
9     load_seondi: in std_logic_vector(5 downto 0);
10     load_minuti: in std_logic_vector(5 downto 0);
11     load_ore: in std_logic_vector(5 downto 0);
12     seondi: out std_logic_vector(5 downto 0);
13     minuti: out std_logic_vector(5 downto 0);      ore: out
14     std_logic_vector(5 downto 0);
15     uscita_ore: out std_logic
16 );end component;
17
18 signal clk, rst, load: std_logic := '0';

```

```
15 signal min, sec, ore, output_ore, output_sec, output_min:
    std_logic_vector (5 downto 0) := (others => '0'); signal output:
    std_logic := '0';
16 constant clock_period: time := 10 ns;
17
18 begin clock_process: process
19 begin      clk <= '0';
20          wait for clock_period/2;      clk <= '1';
21          wait for clock_period/2; end process;
22
23 UUT: Top_module port map(clk, rst, load, sec, min, ore, output_sec,
    output_min, output_ore, output);
24 stim_proc: process begin
25 rst <= '1';
26 wait for 10 ns; rst <= '0';
27 wait for 100 ns;
28 load <= '1';
29 sec <= "010111"; min <= "010110";
30 ore <= "000001";
31 wait for 10 ns; load <= '0';
32
33 wait ; end process;
34
35
36
37 end Behavioral;
```

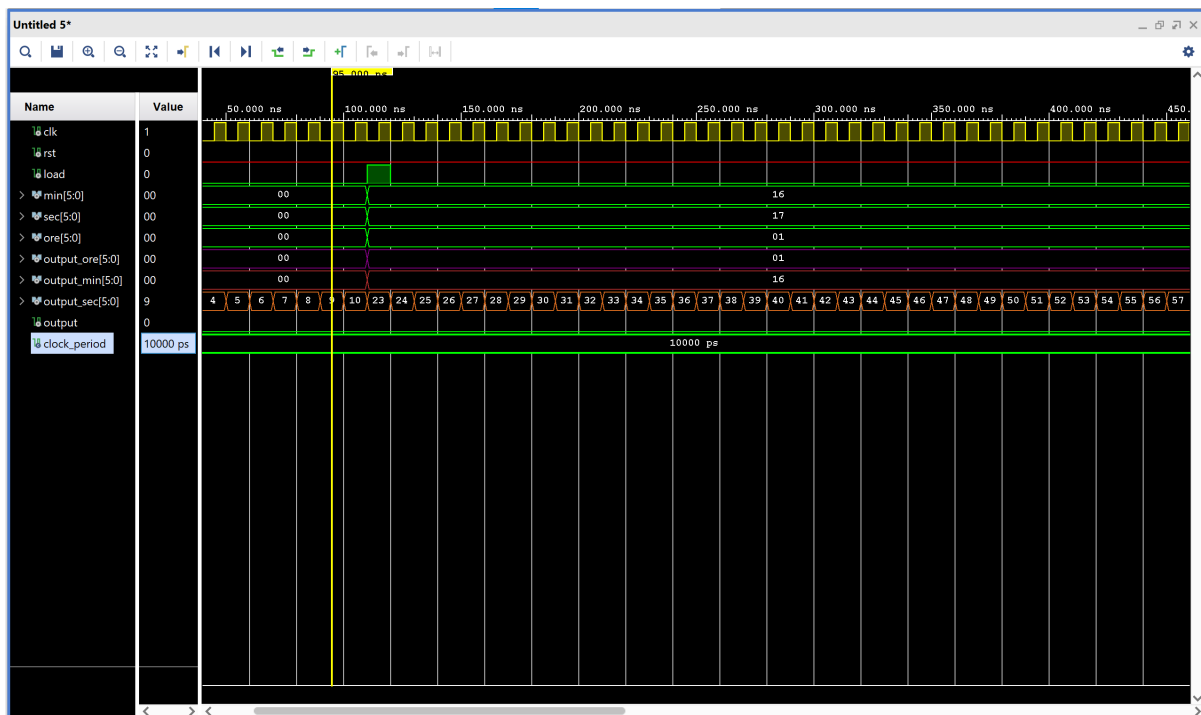


Figura 2.10: Simulazione cronometro

2.4.2 Esercizio 5.2

Sintetizzare ed implementare su board il componente sviluppato al punto precedente, utilizzando i display a 7 segmenti per la visualizzazione dell'orario (o una combinazione di display e led nel caso in cui i display a disposizione siano in numero inferiore a quello necessario), gli switch per l'immissione dell'orario iniziale e due bottoni, uno per il set dell'orario e uno per il reset. Si utilizzi una codifica a scelta dello studente per la visualizzazione dell'orario sui display (esadecimale o decimale).

Progetto e Architettura

In questo punto vogliamo sintetizzare ed implementare il cronometro su board. Per poter far ciò abbiamo utilizzato il componente precedentemente sviluppato aggiungendo una Control Unit per poter gestire in modo efficace l'acquisizione dei diversi input ricevuti dalla board. AUTOMA A STATI DELLA CONTROL UNIT

Implementazione

Implementazione Control Unit:

```
1      library IEEE;use IEEE.STD_LOGIC_1164.ALL;
2
3  entity control_unit is
4      Port (
5          clk: in std_logic;    rst: in std_logic;
6          button: in std_logic;    switch:in std_logic_vector(2
7              downto 0) ;
8          ws,wm,wo: out std_logic );
9  end control_unit;
10 architecture Behavioral of control_unit is
11     type state is (idle, acquisisci_ore,acquisisci_minuti,
12         acquisisci_secondi); signal current_state,next_state: state;
13     begin
14         --segnali azzerati
15         --ws<='0'; --wm<='0';
16         --wo<='0';
17         process_per_update_dello_stato: process(clk,rst)
```

```

16      begin      if (clk'event and clk='1') then
17                  if (rst='1') then          current_state <=idle;
18      else          current_state <=next_state;
19      end if;      end if;
20      end process;
21  comb: process (current_state,button,switch)      begin
22
23      CASE current_state is
24      WHEN idle =>          ws<='0';
25          wm<='0';
26          wo<='0';
27          if (button='1' and switch="001") then
28      next_state <= acquisisci_secondi;      elsif (button
29      ='1' and switch="010") then
30      next_state<= acquisisci_minuti;      elsif (button
31      ='1' and switch="100") then
32      next_state<= acquisisci_ore;      else
33      next_state <= idle;      end if;
34
35      WHEN acquisisci_secondi =>          wo<='0';
36          wm<='0';          ws<='1';
37          if (button='0') then      next_state <= idle;
38      end if;      WHEN acquisisci_minuti=>
39          wo<='0';          ws<='0';
40          wm<='1';          if (button='0') then
41      next_state <= idle;      end if;
42
43      WHEN acquisisci_ore =>          ws<='0';

```

```
42          wm<='0';          wo<='1';  
43      if (button='0') then      next_state <= idle;  
44          end if;  
45      end CASE; end process;  
46 end Behavioral;
```

Simulazione

In seguito è riportato il link per mostrare il corretto funzionamento del cronometro su board: [Video Cronometro](#)

2.4.3 Esercizio 5.3

Estendere il componente sviluppato ai punti precedenti in modo che sia in grado di acquisire e memorizzare internamente fino ad un numero N di intertempi in corrispondenza di un ingresso di stop. Opzionalmente, il componente può prevedere una modalità di visualizzazione in cui, alla pressione di un bottone, vengano visualizzati sui display gli intertempi memorizzati (uno per ogni pressione).

Progetto e Architettura

Per poter memorizzare n intertempi e poi visualizzarli c'è bisogno di una memoria sulla quale è possibile effettuare operazioni sia di lettura che di scrittura. Per quanto riguarda l'implementazione, abbiamo utilizzato un multiplexer che ci permette, tramite un selettore, di selezionare una delle

sue due linee di ingresso (32 bit) che sono acquisizione e visualizzazione. Abbiamo dunque sfruttato uno switch per distinguere queste due modalità:

- visualizzazione (bit pari a 1);
- altrimenti 0.

In più abbiamo sfruttato due bottoni, M18 e P17, rispettivamente per acquisizione e visualizzazione.

Implementazione

Implementazione memoria:

```
1      library IEEE; use IEEE.STD_LOGIC_1164.ALL;
2  use IEEE.NUMERIC_STD.ALL;
3  entity Memoria is Generic (
4      NUM_LOCATIONS : positive := 8    -- Numero di locazioni
        di memoria    );
5  Port (      clk      : in  STD_LOGIC;          -- Clock
        input
6      rst : in  STD_LOGIC;          -- Reset input
        acquire : in  STD_LOGIC;
7      read: in std_logic;          mode: in
        std_logic;
8      data_in : in  STD_LOGIC_VECTOR(31 downto 0); -- Data
        input (32-bit)      data_out : out
        STD_LOGIC_VECTOR(31 downto 0) -- Data output (32-bit
        )
9  ); end Memoria;
```

```

10 architecture Behavioral of Memoria is
11     type mem_array is array (0 to NUM_LOCATIONS-1) of
        STD_LOGIC_VECTOR(31 downto 0); -- Memory array with
        NUM_LOCATIONS locations of 32 bits each    signal memory
        : mem_array := (others => (others => '0')); --
        Initialize memory with all zeros
12     signal addr : integer range 0 to NUM_LOCATIONS-1 := 0; --
        Address counter to cycle through memory locations    --
        signal timer : integer range 0 to 100000000 := 0; --
        Timer to generate 1-second intervals
13 begin
14     process (clk, rst)    begin
15         if rst = '1' then            addr <= 0;
16             memory<=(others=>(others=>'0'));
17         elsif rising_edge(clk) then            if read= '1'
            then    -- 1-second interval
18                 addr <= (addr + 1) mod
                    NUM_LOCATIONS; --
                    Increment address
                    counter
19                 end if;
20                 if acquire = '1' then
                    memory(addr) <=
                        data_in;
21                 end if;            data_out <= memory(addr);
22                 end if;
23     end process;
24 end Behavioral;

```

Simulazione

Di seguito è riportata la simulazione su Board: Video Cronometro 5.3

2.5 Esercizio 6: Sistema di lettura e elaborazione-scrittura PO_PC

2.5.1 Esercizio 6.1

Progettare, implementare in VHDL e verificare mediante simulazione un sistema dotato di una memoria ROM di N locazioni da 8 bit ciascuna, una macchina combinatoria M in grado di trasformare (secondo una funzione a scelta dello studente) la stringa di 8 bit letta dalla ROM in una stringa di 4 bit, e una memoria MEM di N locazioni che memorizza la stringa in output da M. Il sistema si avvia in corrispondenza di un segnale di START che viene fornito esternamente. Una volta avviato, tramite un'apposita unità di controllo che gestisce la tempificazione del sistema, viene scandita una locazione alla volta della ROM e viene scritta la corrispondente locazione di MEM. Gli indirizzi di memoria sono forniti da un contatore. Le memorie ROM e MEM hanno rispettivamente un read e un write sincrono.

Progetto e Architettura

Il nostro sistema deve leggere N locazioni dalla ROM, ciascuna di esse verrà data in ingresso alla nostra macchina combinatoria M. Nel nostro caso la macchina è implementata in modo da effettuare il complemento a due di ciascuna stringa prendendo i 4 bit più significativi. Infine, la stringa convertita verrà memorizzata nella memoria MEM.

Il sistema è composto da:

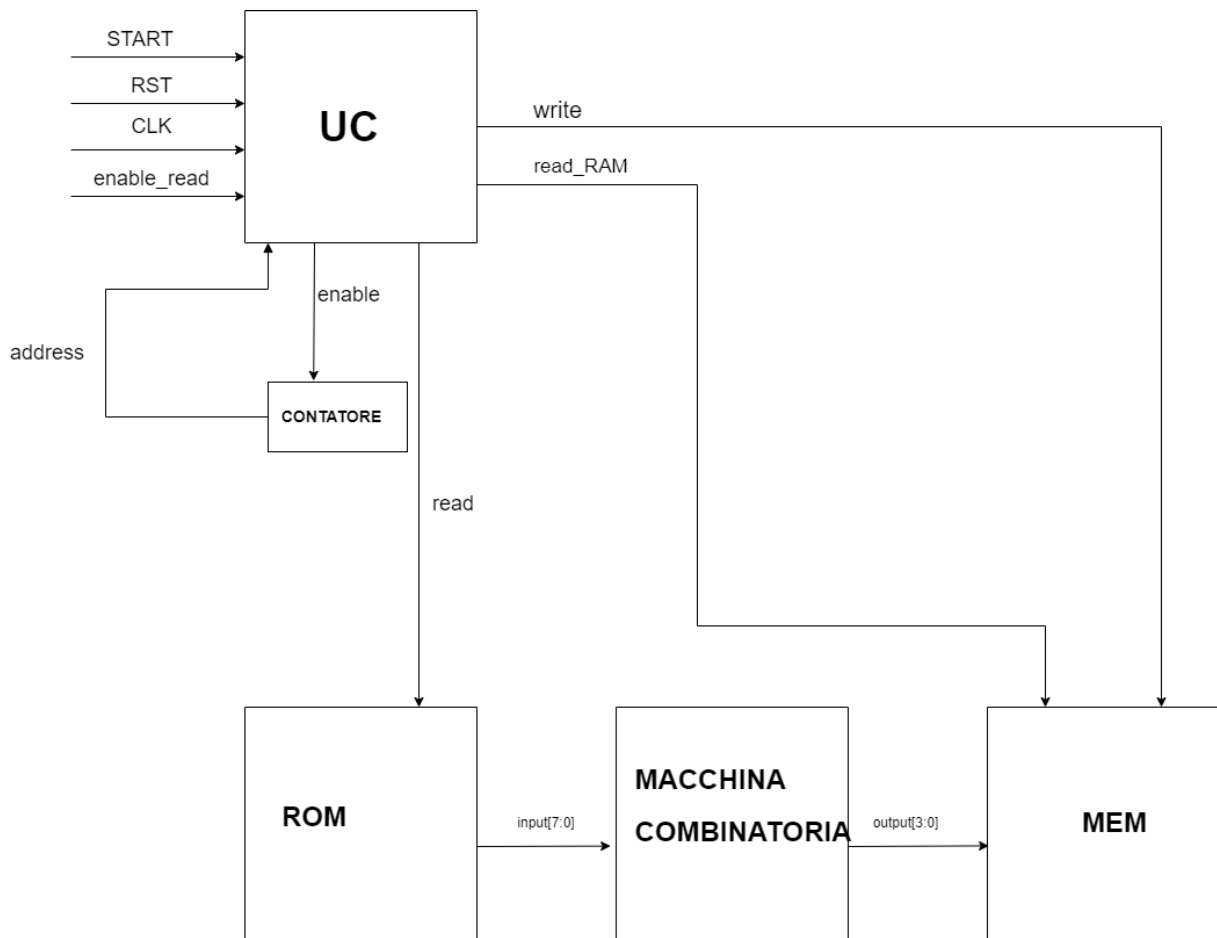


Figura 2.11: Architettura 6.1

- un contatore: fornisce in uscita un segnale d'appoggio "address" per scandire le diverse locazioni della ROM;
- la ROM: contiene i valori da convertire e successivamente memorizzare nella MEM;
- macchina combinatoria: esegue la funzione di conversione secondo la logica precedentemente spiegata;
- la memoria (MEM): memorizza i risultati della macchina combinatoria;

- la Control Unit: gestisce la tempificazione dei segnali di enable (segnale di abilitazione del contatore), read per la ROM, read per la MEM e infine write.

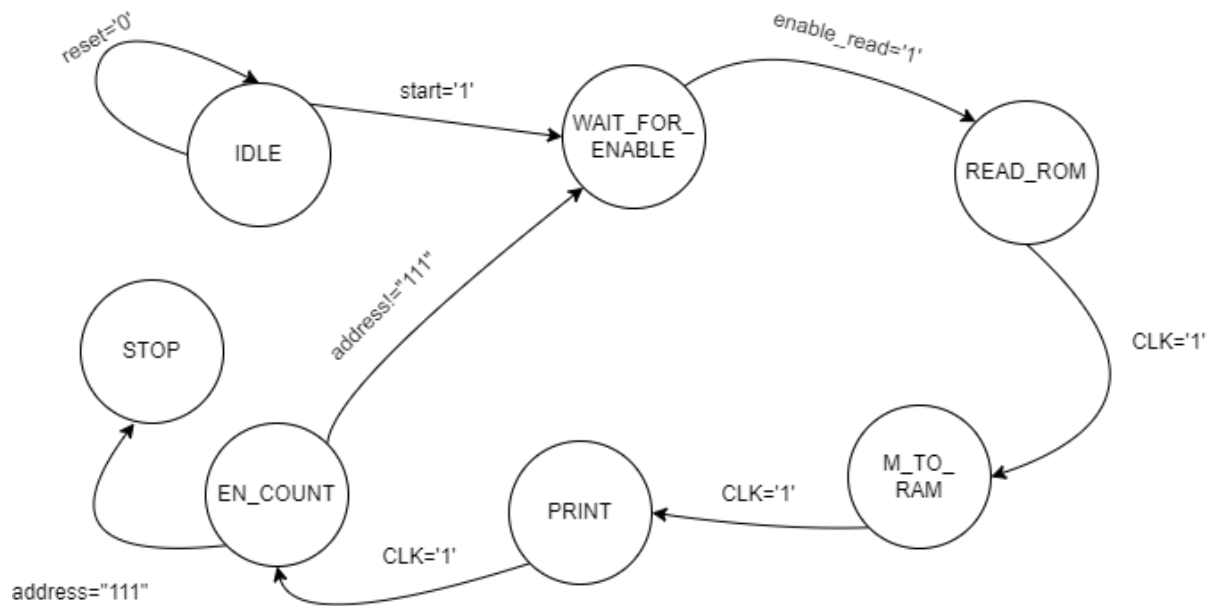


Figura 2.12: Automa a stati finiti dell'esercizio 6.1

Implementazione

Implementazione Control Unit:

```

1
2 library IEEE;
3 use IEEE.STD_LOGIC_1164.ALL;
4
5 entity control_unit is
6     port (
7         start: in STD_LOGIC;
  
```

```
8      enableRead: in STD_LOGIC;
9      clk: in STD_LOGIC;
10     reset: in STD_LOGIC;
11     read: out STD_LOGIC;
12     read_ram: out STD_LOGIC;
13     write: out STD_LOGIC;
14     address: in STD_LOGIC_VECTOR(2 downto 0);
15     stop: out STD_LOGIC;
16     en: out STD_LOGIC
17 );
18 end control_unit;
19
20 architecture Behavioral of control_unit is
21 type stato is (IDLE, STOP_STATE, WAITING_FOR_ENABLE, EN_COUNT,
22               READ_ROM, ROM_TO_M, M_TO_RAM, PRINT);
23
24 signal stato_corrente : stato := IDLE;
25 signal stato_prossimo: stato;
26
27 begin
28
29 EXIT_STATE: process(stato_corrente, start, reset, address)
30 begin
31
32     en <= '0';
33     read <= '0';
34     read_ram <= '0';
```

```
35     write <= '0';
36     stop <= '0';
37
38     case stato_corrente is
39
40         when IDLE =>
41             if(start = '1') then
42                 stato_prossimo <= WAITING_FOR_ENABLE;
43             else
44                 stato_prossimo <= IDLE;
45             end if;
46
47         when STOP_STATE =>
48             stop <= '1';
49
50         when WAITING_FOR_ENABLE =>
51             if(enableRead = '1') then
52                 stato_prossimo <= READ_ROM;
53             else
54                 stato_prossimo <= WAITING_FOR_ENABLE;
55             end if;
56
57         when READ_ROM =>
58             read <= '1';
59             stato_prossimo <= M_TO_RAM;
60
61         when EN_COUNT =>
62             if address="111" then
```

```
63         stato_prossimo <= STOP_STATE;
64     else
65         en <= '1';
66         stato_prossimo <= WAITING_FOR_ENABLE;
67     end if;
68
69     when M_TO_RAM =>
70         write <= '1';
71         stato_prossimo <= PRINT;
72
73     when PRINT =>
74         read_ram <= '1';
75         stato_prossimo <= EN_COUNT;
76
77     when others =>
78         stato_prossimo <= IDLE;
79
80     end case;
81 end process;
82
83
84
85 MEM: process (clk)
86     begin
87         if (clk'event AND clk='1') then
88             if reset = '1' then
89                 stato_corrente <= IDLE;
90             else
```

```
91         stato_corrente <= stato_prossimo;
92     end if;
93 end if;
94 end process;
95
96 end Behavioral;
```

Implementazione macchina combinatoria:

```
1     library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity Circuito_combinatorio is
5
6 port (
7     din: in std_logic_vector(7 downto 0);
8     dout: out std_logic_vector(3 downto 0)
9 );
10 end Circuito_combinatorio;
11
12 architecture behavioral of Circuito_combinatorio is
13 signal complemento: std_logic_vector (7 downto 0);
14 begin
15
16 process(din)
17 begin
18     complemento<=not din;
19 end process;
20
21 dout<=complemento(3)&complemento(2)&complemento(1)&complemento(0);
```

```
(0);
```

```
end behavioral;
```

Implementazione MEM:

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
use ieee.std_logic_arith.all;
```

```
entity mem is
```

```
generic(
```

```
    lunghezza_indirizzo: natural := 3;
```

```
    lunghezza_dati: natural := 4
```

```
);
```

```
port(
```

```
    clock: in std_logic;
```

```
    read: in std_logic;
```

```
    write: in std_logic;
```

```
    address: in std_logic_vector((lunghezza_indirizzo - 1)
```

```
        downto 0);
```

```
    data_input: in std_logic_vector ((lunghezza_dati - 1)
```

```
        downto 0);
```

```
    data_output: out std_logic_vector ((lunghezza_dati - 1)
```

```
        downto 0) := "0000"
```

```
);
```

```
end mem;
```

```
architecture arch of mem is
```

```

22     type ram_type is array (0 to (2**(lunghezza_indirizzo) -1)
      ) of std_logic_vector((lunghezza_dati - 1) downto 0);
23     signal ram: ram_type;
24 begin
25
26 process(clock) is
27 begin
28     if rising_edge(clock) then
29         if(read = '1') then
30             data_output <= ram(conv_integer(unsigned(address))
31                               );
32             elsif (write = '1') then
33                 ram(conv_integer(unsigned(address))) <= data_input
34                 ;
35             end if;
36         end if;
37     end process;
38 end arch;

```

Implementazione Top Module:

```

1
2     library IEEE;
3 use IEEE.STD_LOGIC_1164.ALL;
4
5 entity Top_module is
6     Port (
7         START: in std_logic;

```

```
8      enable_read: in std_logic;
9      CLK: in std_logic;
10     RST: in std_logic;
11     STOP: out std_logic;
12     USCITA: out std_logic_vector( 3 downto 0)
13
14 );
15 end Top_module;
16
17 architecture Behavioral of Top_module is
18 --Definizioni componenti principali
19
20 component control_unit is
21     port (
22         start: in STD_LOGIC;
23         enableRead: in STD_LOGIC;
24         clk: in STD_LOGIC;
25         reset: in STD_LOGIC;
26         read: out STD_LOGIC;
27         read_ram: out STD_LOGIC;
28         write: out STD_LOGIC;
29         address: in STD_LOGIC_VECTOR(2 downto 0);
30         stop: out STD_LOGIC;
31         en: out STD_LOGIC
32     );
33 end component;
34 --CLOCK_DIVIDER
35
```

```
36 component clock_divider is
37     generic(
38         clock_frequency_in : integer := 100000000;--
39         100MHz
40         clock_frequency_out : integer := 2-- Hz
41     );
42     Port ( clock_in : in  STD_LOGIC;
43           reset  : in  STD_LOGIC;
44           clock_out : out STD_LOGIC);
45 end component;
46
47 --ROM
48 component ROM is
49     generic (
50         Number_of_locations: integer :=8
51     );
52     Port (
53         r: in std_logic;
54         address :in std_logic_vector(2 downto 0);
55         dout: out std_logic_vector (7 downto 0)
56     );
57 end component;
58
59 --CIRCUITO COMBINATORIO
60 component combinatoria is
61
62 port (
```

```
63     din: in std_logic_vector(7 downto 0);
64     dout: out std_logic_vector(3 downto 0)
65 );
66 end component;
67
68 --Memoria RAM
69 component mem is
70 generic(
71     lunghezza_indirizzo: natural := 3;
72     lunghezza_dati: natural := 4
73 );
74 port(
75     clock: in std_logic;
76     read: in std_logic;
77     write: in std_logic;
78     address: in std_logic_vector((lunghezza_indirizzo - 1)
79         downto 0);
80     data_input: in std_logic_vector ((lunghezza_dati - 1)
81         downto 0);
82     data_output: out std_logic_vector ((lunghezza_dati - 1)
83         downto 0) := "0000"
84 );
85 end component;
86
87 --CONTATORE MODULO 8 PER SCANDIRE LE 8 LOCAZIONI DELLA ROM
88
89 component counter_mod8 is
90     Port ( clock : in STD_LOGIC;
91           reset : in STD_LOGIC;
```

```

88         enable : in STD_LOGIC;--viene dato dalla control
            unit
89         counter : out  STD_LOGIC_VECTOR (2 downto 0)); --
            COUNTER corrisponde ad ADDRESS
90 end component;
91
92 --BUTTON DEBOUNCER
93 component ButtonDebouncer is
94     generic (
95         CLK_period:    integer := 10;  -- periodo del clock
            della board 10 nanosecondi
96         btn_noise_time: integer := 6500000 --intervallo di
            tempo in cui si ha l'oscillazione del bottone
97                                     --assumo che duri 6.5
                                     ms=6500microsec
                                     =6500000ns
98     );
99     Port ( RST : in STD_LOGIC;
100           CLK : in STD_LOGIC;
101           BTN : in STD_LOGIC;
102           CLEARED_BTN : out STD_LOGIC);
103 end component;
104 --segnali
105
106 signal Control_unit_reset,Control_unit_EN: std_logic:='0';
107 signal Read_ROM,Read_RAM,write : std_logic:='0';
108 signal address: std_logic_vector(2 downto 0):=(others=>'0');
109 signal clear_START,clear_EN_read,clear_RST: std_logic:='0';

```

```
110 signal uscita_ROM : std_logic_vector(7 downto 0) := (others
    => '0');
111 signal uscita_combinatorio: std_logic_vector(3 downto 0) := (
    others=>'0');
112 signal uscita_MEM: std_logic_vector(3 downto 0) := (others
    => '0');
113 signal CLOCK_Divided: std_logic:='0';
114 begin
115
116 --AGGIUNGERE I TRE BUTTON DEBOUNCER
117 --CLOCK DIVIDER
118
119 --Debouncer_START:ButtonDebouncer port map(RST,CLK,START,
    clear_START);
120 --Debouncer_READ:ButtonDebouncer port map(RST,CLK,enable_read
    ,clear_EN_read);
121 --Debouncer_RESET:ButtonDebouncer port map(RST,CLK,RST,
    clear_RST);
122
123 --divisore_di_clock:Clock_Divider generic map(10,1) port map(
    CLK,RST,CLOCK_divided);
124
125 CU: control_unit port map(START,enable_read,CLK,RST,Read_ROM,
    Read_RAM,write,address,STOP,Control_unit_EN);
126
127 CONTATORE: counter_mod8 port map(CLK,RST,Control_unit_EN,
    address);
128
```

```
129 Memoria_ROM: ROM generic map (8) port map (Read_ROM, address,
      uscita_ROM);
130
131
132 Macchina_Combinatoria: combinatoria port map (uscita_ROM,
      uscita_combinatorio);
133
134 Memoria_RAM: mem generic map (3, 4) port map (CLK, Read_RAM, write
      , address, uscita_combinatorio, uscita_MEM);
135
136
137
138 --assegno il segnale d uscita della memoria a i 4 led
139 USCITA<=uscita_MEM;
140
141 end Behavioral;
```

Simulazione

Di seguito è riportato il TestBench utilizzato per testare le funzionalità precedentemente descritte:

```
1
2 library ieee;
3 use ieee.std_logic_1164.all;
4
5 entity tb_system is
6 end tb_system;
7
```

```
8 architecture tb of tb_system is
9
10     component Top_module
11         port (START : in std_logic;
12              enable_read: in std_logic;
13              CLK      : in std_logic;
14              RST       : in std_logic;
15              STOP      : out std_logic;
16              USCITA : out std_logic_vector (3 downto 0));
17     end component;
18
19     signal START : std_logic;
20     signal RST   : std_logic;
21     signal STOP  : std_logic;
22     signal OUTPUT : std_logic_vector (3 downto 0) := "0000";
23
24     signal enableRead: std_logic;
25
26     signal clk_tb : std_logic;
27     constant clk_period : time := 20 ns;
28
29 begin
30
31     -- Clock generation
32     clk_process : process
33     begin
34         clk_tb <= '0';
35         wait for clk_period/2;
```

```
36     clk_tb <= '1';
37     wait for clk_period/2;
38 end process;
39
40
41 dut : Top_module
42 port map (START => START,
43           enable_read => enableRead,
44           CLK      => clk_tb,
45           RST      => RST,
46           STOP     => STOP,
47           USCITA => OUTPUT);
48
49 stimuli : process
50 begin
51
52     START <= '0';
53     RST <= '0';
54     wait for 30 ns;
55
56
57     START <= '1';
58     wait for 21ns;
59     start <= '0';
60     enableRead <= '1';
61     wait for 10 ns;
62     --enableRead<='0';
63     wait for 700 ns;
```

```

64
65     RST <= '1';
66
67     --start <= '1';
68
69     wait for 50 ns;
70
71     RST <= '0';
72
73     wait for 50ns;
74
75     start <= '0';
76
77     wait;
78
79     end process;
80
81 end tb;

```

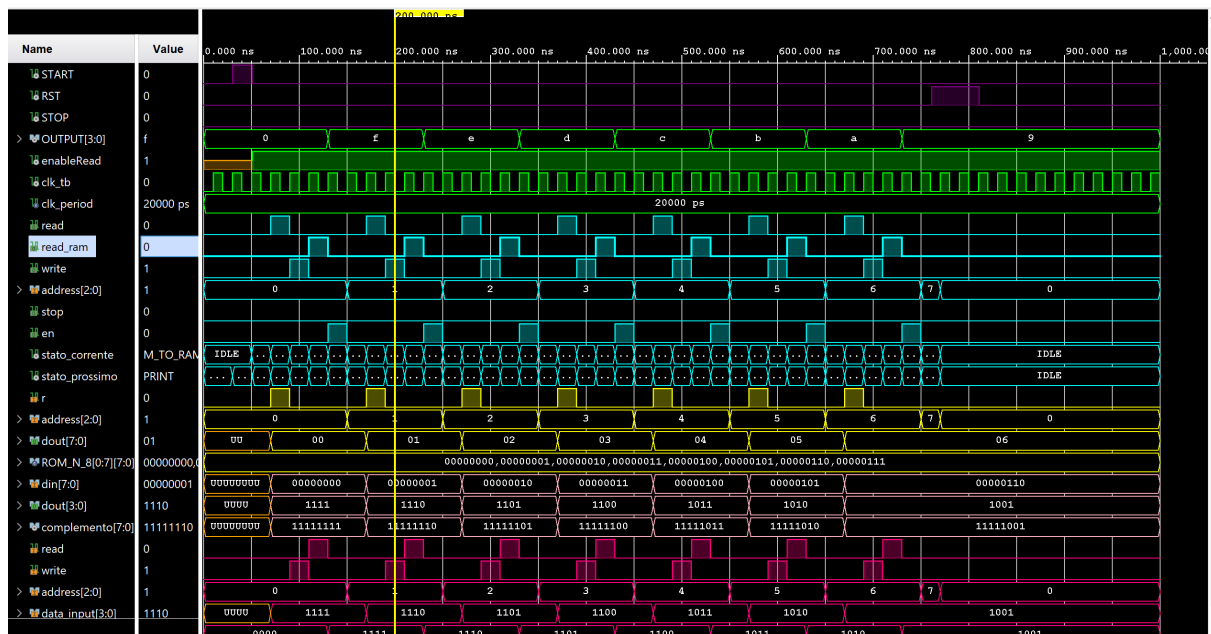


Figura 2.13: Simulazione dell'esercizio 6.1

2.5.2 Esercizio 6.2

Sintetizzare ed implementare su board il componente sviluppato al punto precedente, utilizzando due bottoni per i segnali di read e reset rispettivamente e i led per la visualizzazione delle uscite della macchina istante per istante.

Implementazione

L'unica modifica necessaria apportata al 6.1 per la sintetizzazione su board è stata l'aggiunta di un constraint file.

Di seguito è riportata la simulazione su board: esecuzione esercizio 6.2.

Capitolo 3

Macchine Aritmetiche

3.1 Esercizio 7: Moltiplicatore di Booth

3.1.1 Esercizio 7.1

Progettare, implementare in VHDL e simulare una macchina moltiplicatore di Booth in grado di effettuare il prodotto di 2 stringhe A e B da 8 bit ciascuna

Progetto e Architettura

Lo scopo di questo esercizio è l'implementazione di una macchina moltiplicatore di Booth. L'algoritmo di Booth permette, a differenza di quello di Robertson, di ridurre il numero di moltiplicazioni e somme necessarie per il prodotto di numeri relativi. L'algoritmo esamina ad ogni passo i bit $q[0]$ e $q[-1]$. $Q[-1]$ è un bit "aggiuntivo", che viene inizialmente posto a 0 e successivamente conterrà il bit che dovrebbe essere scartato in seguito allo shift.

In seguito a tale controllo verrà effettuata una codifica secondo il seguente criterio:

- 00/11: viene eseguito solo lo shift;
- 01: viene eseguita l'addizione del registro accumulatore e del registro M, contenente l'operando Y, e in seguito viene effettuato lo shift;
- 10: viene eseguita la sottrazione del registro accumulatore e del registro M, contenente l'operando Y, e in seguito viene effettuato lo shift;

L'architettura di Booth comprende i seguenti elementi:

- registro M: registro che contiene l'operando y (moltiplicando);
- shift register(A.Q): contiene la concatenazione del registro A (accumulatore), inizialmente riempito con 8 bit pari a 0, e Q inizializzato con il valore dell'operando X. Si occupa di shiftare verso destra di un bit ogni qualvolta l'algoritmo lo prevede;
- add/sub: si occupa di effettuare le operazioni di addizione (subtract='0') o sottrazione (subtract='1');
- contatore: conta le n iterazioni;
- control unit: gestisce i segnali esterni(start, reset e clock), fornisce l'inizializzazione, il reset del contatore, il segnale di subtract e riceve in ingresso $q[0]$, $q[-1]$ e il conteggio, per eseguire i rispettivi controlli;

- multiplexer 2:1: ci permette di distinguere la prima operazione che inizializza lo shift register, come precedentemente descritto, dalle successive.

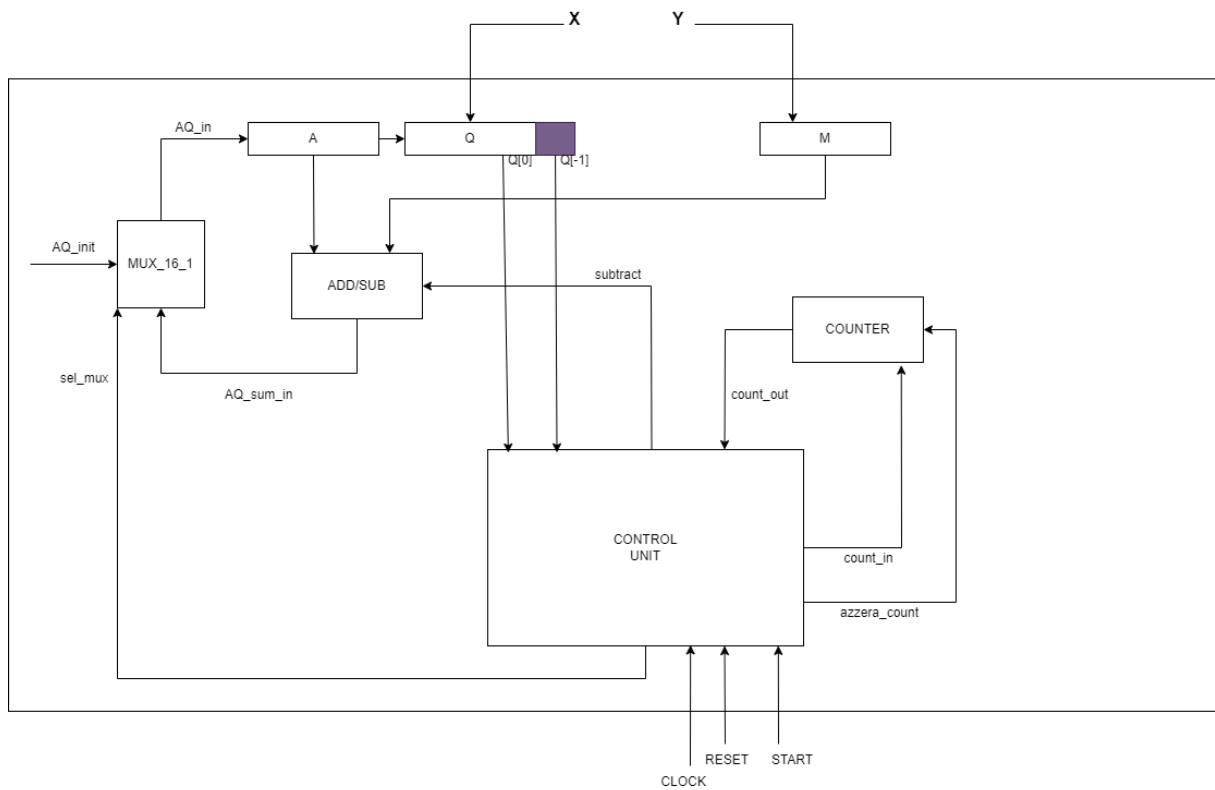


Figura 3.1: Architettura 7

In seguito si può visualizzare l'automa a stati finiti che evidenzia i vari passaggi tra gli stati e cosa comportano:

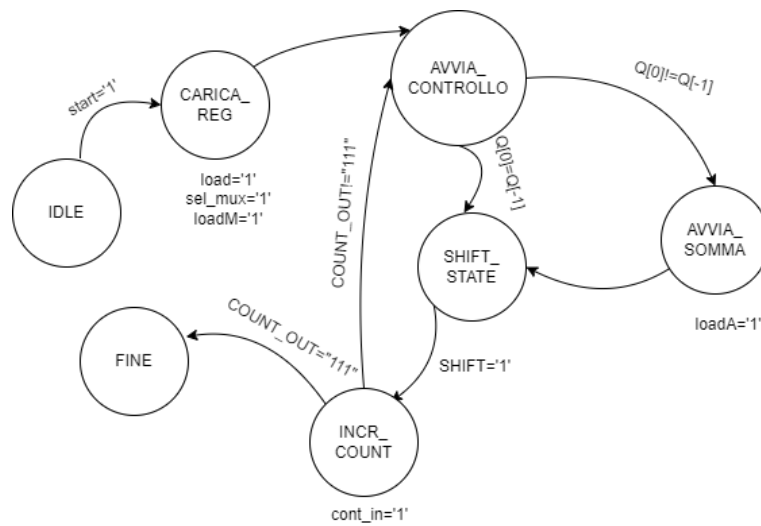


Figura 3.2: Automa a stati finiti 7.1

Implementazione

Implementazione Control Unit:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5
6 entity Control_Unit is
7     Port (
8         start: in std_logic ; --segnale esterno di start
9         clk,rst: in std_logic;--clock e reset
10        count_out: in std_logic_vector(2 downto 0);--uscita
11                contatore
12        Q_0,Q_1: in std_logic;--shift register
13        sel_mux: out std_logic;
14        subtract: out std_logic;--segnale da porre in ingresso
15                al parallel adder
16        load,loadM,loadQ,shift: out std_logic;--shift register
17        count_in: out std_logic;--valore del conteggio in
18                uscita dal contatore
19        sel_end: out std_logic
20    );
21 end Control_Unit;
22
23 architecture Behavioral of Control_Unit is
24
25     type state is (IDLE,CARICA_REG,AVVIA_CONTROLLO,AVVIA_SOMMA,
26                     SHIFT_STATE,INCR_COUNT,FINE);
```

```
23 signal current_state,next_state: state;
24 signal condition: std_logic_vector(1 downto 0) :=(others=>'0');
25
26 begin
27 condition<= Q_0&Q_1;
28
29
30 reg_stato: process(clk)
31     begin
32         if(clk'event and clk='1') then
33             if(rst='1') then
34                 current_state <=idle;
35             else
36                 current_state <=next_state;
37             end if;
38         end if;
39     end process;
40
41 comb: process(current_state, start,count_out,condition)
42 begin
43
44     subtract<='0';
45     load<='0';
46     loadM<='0';
47     shift<='0';
48     count_in<='0';
49     sel_end<='0';
50     sel_mux<='0';
```

```
51
52 CASE current_state is
53
54     WHEN IDLE =>
55
56
57         if(start='1') then
58             next_state <= CARICA_REG;
59         else
60             next_state <= IDLE;
61         end if;
62
63         --fornisce i segnali di caricamento operandi
64     WHEN CARICA_REG=>
65
66         loadQ<='1'; --abilita il caricamento del
67                     moltiplicatore nel registro Q
68         loadM <='1';
69         sel_mux<='1';
70         load<='1'; --abilita il caricamento del
71                     moltiplicando nel registro M
72         next_state <= AVVIA_CONTROLLO;
73
74         --acquisisce gli operandi, su cui il sommatore inizia
75         a lavorare immediatamente
76     WHEN AVVIA_CONTROLLO =>
77
78         --sel_mux<='1';
79         if condition ="01" then
```

```
76         next_state<= AVVIA_SOMMA;
77
78         elsif condition="10" then
79             next_state<=AVVIA_SOMMA;
80         elsif condition="11" then
81
82             next_state <=SHIFT_STATE;
83
84         elsif condition="00" then
85
86             next_state <=SHIFT_STATE;
87         end if;
88
89 --      WHEN LOAD_STATE =>
90 --          --sel_mux<='1';
91 --          --load<='1';
92 --          next_state<=SHIFT_STATE;
93
94     WHEN AVVIA_SOMMA =>
95 -- sel_mux<='1';
96         if condition="10" then
97             subtract<='1';
98             load<='1';--
99             next_state<=SHIFT_STATE;
100        elsif condition="01" then
101            subtract<='0';
102            load<='1';--
103            next_state <= SHIFT_STATE ;
```

```
104         end if ;
105
106
107
108     WHEN SHIFT_STATE =>
109         --sel_mux<='1';
110         shift <='1';
111
112         next_state <= INCR_COUNT;
113
114         --esegue lo shift, abilita incremento conteggio e
115         --predispone per nuova iterazione
116
117     WHEN INCR_COUNT =>
118
119         count_in <= '1';
120         if count_out="111" then
121             next_state<=FINE;
122         else
123
124             next_state <= AVVIA_CONTROLLO;
125
126         end if;
127
128
129
130     WHEN fine =>
```

```
131         sel_end<='1';
132         --next_state<= IDLE;
133
134
135
136     end CASE;
137 end process;
138
139
140 end Behavioral;
```

Implementazione Top Module:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5 entity top_module is
6     Port (
7         clk,rst: std_logic;
8         start: in std_logic;
9         Moltiplicando: in std_logic_vector(7 downto 0);
10        Moltiplicatore: in std_logic_vector(7 downto 0);
11        risultato: out std_logic_vector(15 downto 0)
12
13    );
14 end top_module;
15
16 architecture Structural of top_module is
17 component Control_Unit is
```

```
18  Port (
19      start: in std_logic ; --segnale esterno di start
20      clk,rst: in std_logic;--clock e reset
21      count_out: in std_logic_vector(2 downto 0);--uscita
                contatore
22      Q_0,Q_1: in std_logic;--shift register
23      sel_mux: out std_logic;
24      subtract: out std_logic;--segnale da porre in ingresso
                al parallel adder
25      load,loadM,loadQ,shift: out std_logic;--shift register
26      count_in: out std_logic;--valore del conteggio in
                uscita dal contatore
27      sel_end: out std_logic
28  );
29  end component;
30
31  --contatore
32  component cont_mod8 is
33      port( clock, reset: in std_logic;
34            count_in: in std_logic;
35            count: out std_logic_vector(2 downto 0));
36  end component;
37
38  --Registro a 8 bit
39  component registro_8 is
40      port( A: in std_logic_vector(7 downto 0);
41            clk, rst, load: in std_logic;
42            B: out std_logic_vector(7 downto 0));
```

```
43 end component;
44 --Shift Register
45 component shift_register is
46     port( parallel_in: in std_logic_vector(16 downto 0);
47           serial_in: in std_logic;
48           clock, reset, load, shift: in std_logic;
49           parallel_out: out std_logic_vector(16 downto 0)
50           --Qdi_meno_1: out std_logic
51           );
52
53 end component;
54
55 --addizionatore/sottrattore
56 component adder_sub is
57     port( X, Y: in std_logic_vector(7 downto 0);
58           cin: in std_logic;
59           Z: out std_logic_vector(7 downto 0);
60           cout: out std_logic);
61 end component;
62
63 --Mux2_1
64 component mux_21 is
65     generic (width : integer range 0 to 20 := 8);
66     port( x0, x1: in std_logic_vector(width-1 downto 0);
67           s: in std_logic;
68           y: out std_logic_vector(width-1 downto 0));
69 end component;
```

```
71 signal valore_contatore: std_logic_vector(2 downto 0) := (others
    =>'0');
72 signal Q_0,Q_1,count_in,sel,subtract,load_SHIFT,load_REGM,
    load_REGQ,shift,carry_out:std_logic:='0';
73 signal uscita_registro_M: std_logic_vector(7 downto 0) := (
    others=>'0');
74 signal Q,uscita_addizionatore: std_logic_vector(7 downto 0);
75
76 signal AQ_init: std_logic_vector(16 downto 0) := (others=>'0');
77 signal sum,ingresso_adder: std_logic_vector (7 downto 0) := (
    others=>'0');
78 signal AQ_in,AQ_sum_in,AQ_out: std_logic_vector(16 downto 0)
    :=(others=>'0');
79 signal selAq: std_logic:='0';
80 begin
81
82 AQ_init<="00000000"& Moltiplicatore &'0';
83 AQ_sum_in <= sum &AQ_out(8 downto 0);
84
85 --Modifica il valore del mux sel
86
87 CU: Control_Unit port map(start,clk,rst,valore_contatore,
    AQ_out(1),AQ_out(0),selAq,subtract,load_SHIFT,load_REGM,
    load_REGQ,shift,count_in,sel);
88
89 CONTATORE: cont_mod8 port map(clk,rst,count_in,
    valore_contatore);
90
```

```
91 Registro_m : registro_8 port map (Moltiplicando, clk, rst,  
    load_REGM, ingresso_adder);  
92  
93 SUB_ADD: adder_sub port map (AQ_out (16 downto 9), ingresso_adder  
    , subtract, sum, carry_out);  
94  
95 MUX: mux_21 generic map (17) port map (AQ_sum_in, AQ_init, selAq,  
    AQ_in);  
96  
97 SR: shift_register port map (AQ_in, AQ_out (16), clk, rst,  
    load_SHIFT, shift, AQ_out);  
98  
99  
100  
101  
102 risultato <= AQ_out (16 downto 1);  
103  
104 end Structural;
```

Implementazione sub adder:

```
1 library IEEE;  
2 use IEEE.STD_LOGIC_1164.ALL;  
3  
4 entity adder_sub is  
5     port ( X, Y: in std_logic_vector (7 downto 0);  
6           cin: in std_logic;  
7           Z: out std_logic_vector (7 downto 0);  
8           cout: out std_logic);  
9 end adder_sub;
```

```
10
11 architecture structural of adder_sub is
12     component ripple_carry is
13     port( X, Y: in std_logic_vector(7 downto 0);
14          c_in: in std_logic;
15          c_out: out std_logic;
16          Z: out std_logic_vector(7 downto 0));
17     end component;
18
19     signal complementoy: std_logic_vector(7 downto 0);
20
21     begin
22
23     complemento_y: FOR i IN 0 TO 7 GENERATE
24         complementoy(i) <= Y(i) xor cin;
25     END GENERATE;
26
27     RA: ripple_carry port map(X, complementoy, cin, cout, Z);
28
29     end structural;
```

l'implementazione dello shift register, mux e contatore è già presente negli esercizi precedenti.

Simulazione

Di seguito è riportato il TestBench utilizzato per testare le funzionalità precedentemente descritte:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity TB_TOP is
5 -- Port ( );
6 end TB_TOP;
7
8 architecture Behavioral of TB_TOP is
9 component top_module is
10     Port (
11         clk,rst: std_logic;
12         start: in std_logic;
13         Moltiplicando: in std_logic_vector(7 downto 0);
14         Moltiplicatore: in std_logic_vector(7 downto 0);
15         risultato: out std_logic_vector(15 downto 0)
16
17     );
18 end component;
19 constant clock_period: time:=10 ns;
20 signal clock,rst,start: std_logic:='0';
21 signal ingresso_1,ingresso_2: std_logic_vector(7 downto 0):=(
22     others=>'0');
23
24
25 signal risultato : std_logic_vector(15 downto 0):=(others
26     =>'0');
```

```
27 CLK_process: process
28 begin
29     clock<='0';
30     wait for clock_period/2;
31     clock<='1';
32     wait for clock_period/2;
33 end process;
34
35 UUT:top_module port map(clock,rst,start,ingresso_1,ingresso_2,
    risultato);
36
37 stim_proc:
38 process
39 begin
40 start<='1';
41 ingresso_1<="00000010";
42 ingresso_2<="11111100";
43 wait for 10 ns;
44 start<='0';
45
46 wait ;
47 end process;
48
49
50
51 end Behavioral;
```

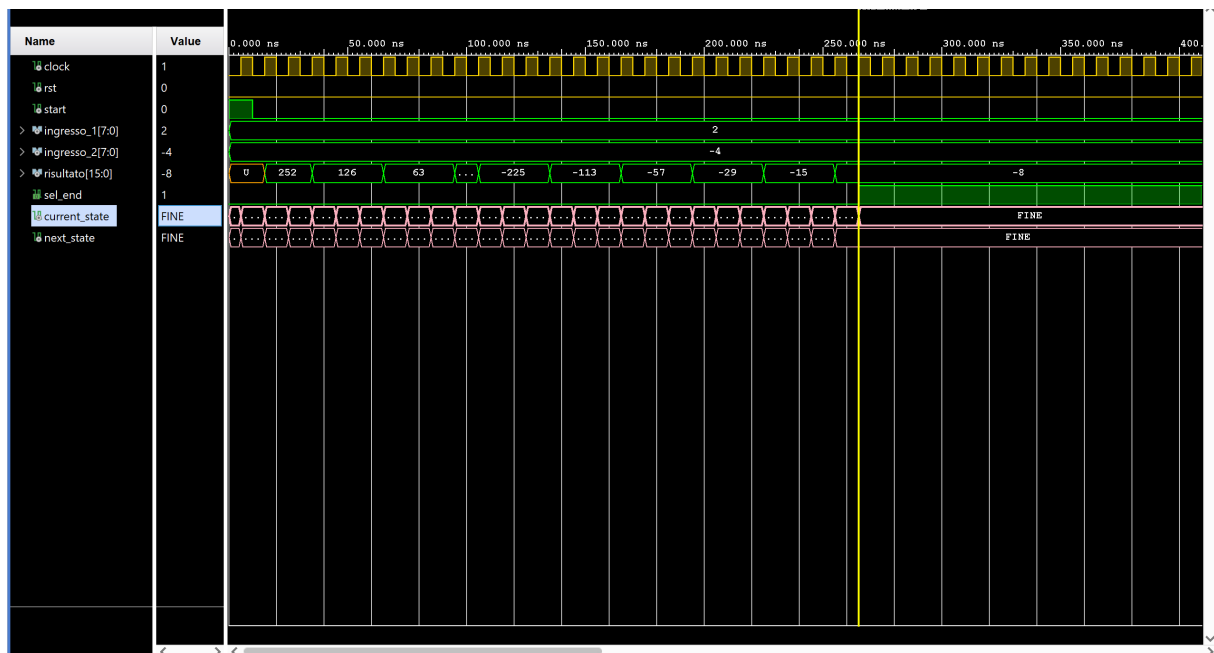


Figura 3.3: Simulazione 7.1

3.1.2 Esercizio 7.2

Sintetizzare il moltiplicatore implementato al punto 7.1 su FPGA e testarlo mediante l'utilizzo dei dispositivi di input/output (switch, bottoni, led, display) presenti sulla board di sviluppo in dotazione. La modalità di utilizzo degli stessi è a completa discrezione degli studenti.

Progetto e Architettura

Per la sintetizzazione dell'esercizio precedente su board acquisiamo i due operandi tramite gli switch, otto per il primo e otto per il secondo. L'automa non necessita di modifiche. Le uniche modifiche necessarie per il corretto funzionamento sono: l'aggiunta del file constraint, l'unità per la conversione del risultato in decimale(sia per i numeri positivi che per i negativi), blocco per il display ad 7 segmenti, e il top module.

Implementazione

Implementazione Top Module:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity Top_module is
5     Port (
6         clock: in std_logic;
7         reset: in std_logic;
8         START_Button: in std_logic;
9         anodes_out : out  STD_LOGIC_VECTOR (7 downto 0);
```

```
10 cathodes_out : out STD_LOGIC_VECTOR (7 downto 0);
11 WRITE_button : in std_logic;
12 switch:in std_logic_vector(15 downto 0)
13 );
14 end Top_module;
15
16 architecture Behavioral of Top_module is
17 component display_seven_segments is
18     Generic (
19         CLKIN_freq : integer := 100000000;
20         CLKOUT_freq : integer := 500
21     );
22     Port ( CLK : in STD_LOGIC;
23           RST : in STD_LOGIC;
24           VALUE : in STD_LOGIC_VECTOR (31 downto 0);
25           ENABLE : in STD_LOGIC_VECTOR (7 downto 0); --
26               decide quali cifre abilitare
27           DOTS : in STD_LOGIC_VECTOR (7 downto 0); -- decide
28               quali punti visualizzare
29           ANODES : out STD_LOGIC_VECTOR (7 downto 0);
30           CATHODES : out STD_LOGIC_VECTOR (7 downto 0));
31 end component;
32
33 --Control UNit
34
35 component Control_Unit is
36     Port (
37         start: in std_logic ; --segnale esterno di start
```

```
36     clk,rst: in std_logic;--clock e reset
37     count_out: in std_logic_vector(2 downto 0);--uscita
           contatore
38     button: in std_logic;
39     Q_0,Q_1: in std_logic;--shift register
40     sel_mux: out std_logic;
41     subtract: out std_logic;--segnale da porre in ingresso
           al parallel adder
42     load,loadM,loadQ,shift: out std_logic;--shift register
43     count_in: out std_logic;--valore del conteggio in
           uscita dal contatore
44     sel_end: out std_logic;
45     azzera: out std_logic
46 );
47
48 end component;
49 --
50 component registro_8 is
51     port( A: in std_logic_vector(7 downto 0);
52           clk, rst, load: in std_logic;
53           B: out std_logic_vector(7 downto 0));
54 end component;
55
56 component cont_mod8 is
57     port( clock, reset: in std_logic;
58           count_in: in std_logic;
59           count: out std_logic_vector(2 downto 0));
60
```

```
61 end component;
62 --Adder_SUB
63 component adder_sub is
64     port( X, Y: in std_logic_vector(7 downto 0);
65           cin: in std_logic;
66           Z: out std_logic_vector(7 downto 0);
67           cout: out std_logic);
68 end component;
69 --MUX
70 component mux_21 is
71     generic (width : integer range 0 to 20 := 8);
72     port( x0, x1: in std_logic_vector(width-1 downto 0);
73           s: in std_logic;
74           y: out std_logic_vector(width-1 downto 0));
75 end component;
76 --sr
77 component shift_register is
78     port( parallel_in: in std_logic_vector(16 downto 0);
79           serial_in: in std_logic;
80           clock, reset, load, shift: in std_logic;
81           parallel_out: out std_logic_vector(16 downto 0)
82
83           );
84
85 end component;
86
87 --Button debouncer
88 component ButtonDebouncer is
```

```
89     generic (  
90         CLK_period:    integer := 10;  -- periodo del clock  
91         della board 10 nanosecondi  
92  
93         btn_noise_time: integer := 6500000;  
94  
95     );  
96  
97     Port ( RST : in STD_LOGIC;  
98           CLK  : in STD_LOGIC;  
99           BTN  : in STD_LOGIC;  
100          CLEARED_BTN : out STD_LOGIC);  
101  
102     end component;  
103  
104     --blocco per la realizzazione della conversione per il display  
105     component bin_to_number is  
106     Port (  
107         bin: in std_logic_vector(15 downto 0);  
108         modulo : out integer ;  
109         segno : out std_logic  
110     );  
111  
112     end component;  
113  
114     component number_to_digit is  
115     Port (  
116         segno : in std_logic;  
117         modulo : in integer;  
118         digit_for_display: out std_logic_vector(31 downto 0)  
119     );  
120  
121     end component;
```

```
116
117
118 signal reset_cont, write_M, write_Q, sel_mux, subtract, load, shift,
    read_M, read_Q, count_in, sel_end, carry_out: std_logic := '0';
119 signal count_out: std_logic_vector(2 downto 0) := (others => '0')
    ;
120 signal Moltiplicatore, Moltiplicando: std_logic_vector(7 downto
    0);
121 signal AQ_sum_in, AQ_out, AQ_init, AQ_in: std_logic_vector(16
    downto 0) := (others => '0');
122 signal sum, ingresso_adder: std_logic_vector(7 downto 0) := (
    others => '0');
123 signal cleared_START, cleared_WRITE: std_logic := '0';
124 signal data_in: std_logic_vector(15 downto 0);
125 signal modulo: integer := 0;
126 signal segno : std_logic := '0';
127 signal data_for_display: std_logic_vector(31 downto 0) := (
    others => '0');
128
129 begin
130
131
132 CU: Control_Unit port map (START_Button, clock, reset, count_out,
    WRITE_button, AQ_out(1), AQ_out(0), sel_mux, subtract, load,
    read_M, read_Q, shift, count_in, sel_end, reset_cont);
133
134 Moltiplicando <= switch(15 DOWNT0 8);
135 Moltiplicatore <= switch(7 downto 0);
```

```
136 Registro_M : registro_8 port map(Moltiplicando,clock,reset,
    read_M,ingresso_adder);
137
138 AQ_init <="00000000" & Moltiplicatore &'0';
139 AQ_sum_in <= sum & AQ_out(8 downto 0);
140 CONTATORE: cont_mod8 port map(clock,reset_cont,count_in,
    count_out);
141 ADDER_SUBtractor: adder_sub port map(AQ_out(16 downto 9),
    ingresso_adder,subtract,sum,carry_out);
142 MUX:mux_21 generic map(17) port map(AQ_sum_in,AQ_init,sel_mux,
    AQ_in);
143 SR: shift_register port map(AQ_in,AQ_out(16),clock,reset,load,
    shift,AQ_out);
144
145 data_in<=AQ_out(16 downto 1);
146
147
148 BLOCCO_CONVERSIONE: bin_to_number port map(data_in,module,
    segno);
149 BLOCCO_CONVERSIONE_2:number_to_digit port map(segno,module,
    data_for_display);
150
151
152 Display_a_sette_segmenti: display_seven_segments generic map
    (100000000,500)
153 port map(CLK=>clock,RST=>reset,VALUE=>data_for_display,ENABLE
    =>"11111111",DOTS=>"00000000",ANODES=>anodes_out,CATHODES=>
    cathodes_out);
```

154

155

156

`end Behavioral;`

Simulazione

Link con video di del funzionamento del moltiplicatore:

[Video Moltiplicatore](#)

3.2 Esercizio 7_bis: Divisore Non–Restoring

3.2.1 Esercizio 7BIS.1

Progettare, implementare in VHDL e simulare una macchina divisore (modalità non-restoring) in grado di effettuare la divisione intera fra due stringhe A e B di 4 bit ciascuna;

Progetto e Architettura

Il divisore Non-Restoring esegue divisioni tra due numeri, ad ogni passo effettua la somma o la sottrazione tra l'accumulatore e il dividendo (registro M), a seconda del segno ($A[7]$).

L'architettura comprende i seguenti elementi:

- registro M: registro che contiene l'operando y (dividendo);

- shift register(A.Q): contiene la concatenazione del registro A (accumulatore), inizialmente riempito con 8 bit pari a 0, e Q inizializzato con il valore del divisore X. Si occupa di shiftare verso sinistra di un bit ogni qualvolta l'algoritmo lo prevede;
- add/sub: si occupa di effettuare le operazioni di addizione (subtract='0') o sottrazione (subtract='1');
- contatore: conta le n iterazioni;
- control unit: gestisce i segnali esterni(start, reset e clock), fornisce l'inizializzazione, il reset del contatore, il segnale di subtract e riceve in ingresso q[0] e il conteggio, per eseguire i rispettivi controlli;
- multiplexer 3:1: ci permette di distinguere la prima operazione che inizializza lo shift register, come precedentemente descritto, dalle successive, e come terzo caso ci permette di gestire il passo correttivo, che consiste nell'addizionare il dividendo all'accumulatore, nel caso in cui il segno sia 1 e il contatore abbia accumulato le n iterazioni.

In seguito si può visualizzare l'automa a stati finiti che evidenzia i vari passaggi tra gli stati e cosa comportano:

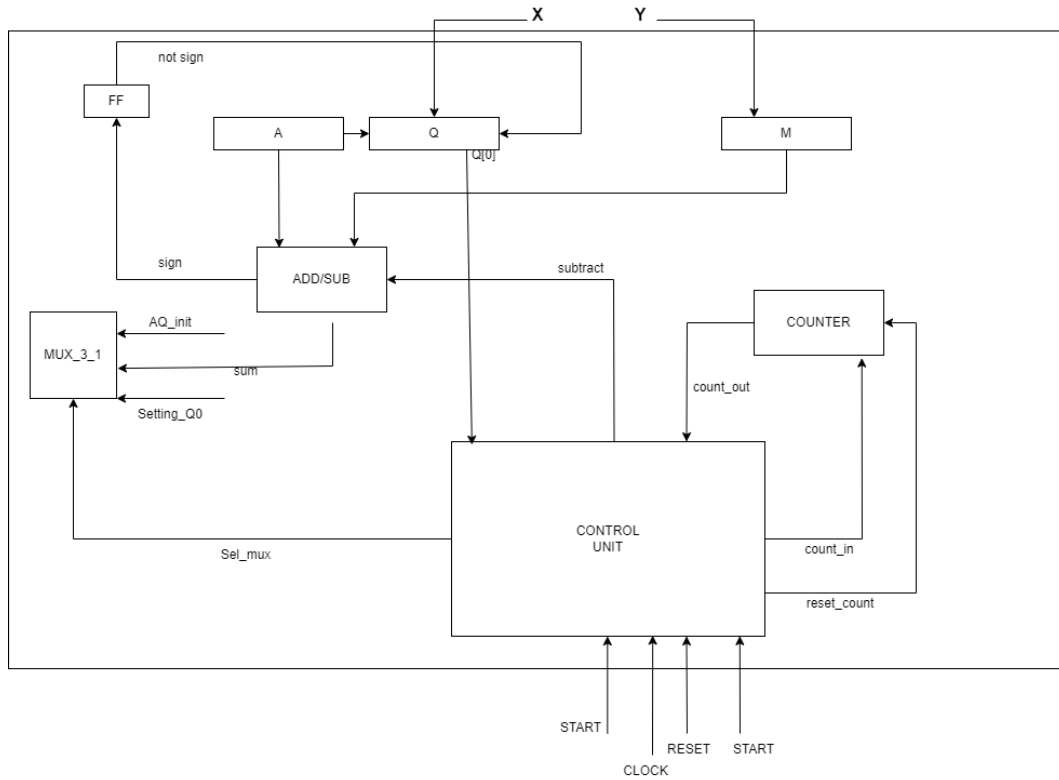


Figura 3.4: Architettura 7

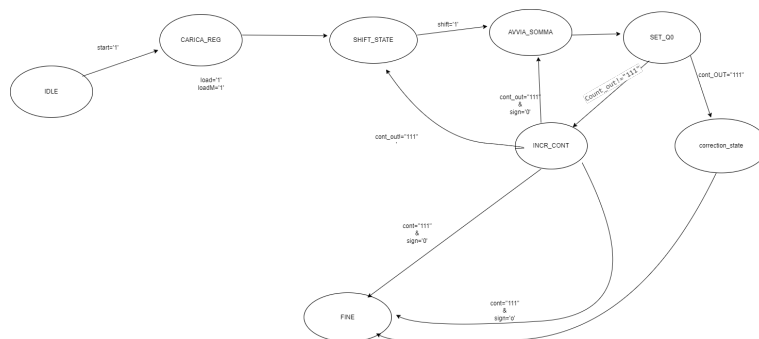


Figura 3.5: Automa a stati finiti 7.1

Implementazione

Implementazione Top Module:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5
6 entity Top_module is
7     Port (
8         clk: in std_logic;
9         rst: in std_logic;
10        start: in std_logic;
11        write: in std_logic;
12        dividendo: in std_logic_vector(7 downto 0);
13        divisore: in std_logic_vector(7 downto 0);
14        quoziente: out std_logic_vector(7 downto 0);
15        resto: out std_logic_vector(7 downto 0)
16
17    );
18 end Top_module;
19
20 architecture Behavioral of Top_module is
21
22 component Control_Unit is
23     Port (
24         start: in std_logic ; --segnale esterno di start
25         write: in std_logic;
26         clk,rst: in std_logic;--clock e reset
```

```
27     count_out: in std_logic_vector(2 downto 0);--uscita
           contatore
28     sel_mux: out std_logic_vector(1 downto 0);
29     sign: in std_logic;--segnale in ingresso dalla shift
           register
30     subtract: out std_logic;--segnale da porre in ingresso
           al parallel adder
31     load,loadM,shift: out std_logic;--shift register
32     count_in: out std_logic;--valore del conteggio in
           uscita dal contatore
33     azzera: out std_logic
34 );
35 end component;
36
37 component adder_sub is
38     port( X, Y: in std_logic_vector(7 downto 0);
39           cin: in std_logic;
40           Z: out std_logic_vector(7 downto 0);
41           cout: out std_logic);
42 end component;
43
44 component mux_3to1 is
45     Port (
46         input_0, input_1, input_2 : in STD_LOGIC_VECTOR(15
           downto 0);
47         selettore : in STD_LOGIC_VECTOR(1 downto 0);
48         output : out STD_LOGIC_VECTOR(15 downto 0)
49     );
```

```
50
51
52 end component;
53
54 component shift_register is
55     port( parallel_in: in std_logic_vector(15 downto 0);
56           serial_in: in std_logic;
57           clock, reset, load, shift: in std_logic;
58           parallel_out: out std_logic_vector(15 downto 0)
59
60           );
61 end component;
62
63 component cont_mod8 is
64     port( clock, reset: in std_logic;
65           count_in: in std_logic;
66           count: out std_logic_vector(2 downto 0));
67 end component;
68
69 component registro_8 is
70     port( A: in std_logic_vector(7 downto 0);
71           clk, rst, load: in std_logic;
72           B: out std_logic_vector(7 downto 0));
73 end component;
74
75 component FFD is
76     port( clock, reset, d: in std_logic;
77           y: out std_logic := '0');
```

```
78 end component;
79
80 signal uscita_M : std_logic_vector(7 downto 0) := (others=>'0');
81 signal count_out: std_logic_vector (2 downto 0) := (others=>'0')
    ;
82 signal serial_in,Q_0,sign,subtract,load,load_M,shift,count_in
    ,reset_count,riporto: std_logic:='0';
83 signal sum : std_logic_vector(7 downto 0) := (others=>'0');
84 signal setting_Q0,AQ_init,AQ_sum_in,AQ_in,AQ_out:
    std_logic_vector(15 downto 0) := (others=>'0');
85 signal sel_mux:std_logic_vector(1 downto 0) := (others=>'0');
86 signal temp_F: std_logic:='0';
87
88 begin
89
90 CU:Control_Unit port map(start,write,clk,rst,count_out,sel_mux
    ,sign,subtract,load,load_M,shift,count_in,reset_count);
91 CONTATORE: cont_mod8 port map(clk,reset_count,count_in,
    count_out);
92
93 REGISTRO_M: registro_8 port map(divisore,clk,rst,load_M,
    uscita_M);
94 sign<=AQ_out(15);
95 AQ_init <="00000000" & dividendo;
96 AQ_sum_in <= sum & AQ_out(7 downto 0);
97 setting_Q0<=AQ_out(15 downto 1) & (not sign);
98
99 PARA:adder_sub port map (AQ_out(15 downto 8),uscita_M,subtract
```

```
    ,sum,riporto);  
100  
101  
102 --MUX  
103 MUX:mux_3to1 port map (AQ_init,AQ_sum_in,setting_Q0,sel_mux,  
    AQ_in);  
104  
105 SR: shift_register port map(AQ_in,serial_in,clk,rst,load,shift  
    ,AQ_out);  
106  
107 Flip_flop_D: FFD port map(clk,rst,sign,serial_in);  
108  
109  
110 quoziente<=AQ_out(7 downto 0);  
111  
112 resto<=AQ_out(15 downto 8);  
113  
114  
115  
116  
117  
118  
119  
120 end Behavioral;
```

Implementazione Control Unit:

```
1 library IEEE;  
2 use IEEE.STD_LOGIC_1164.ALL;  
3
```

```
4
5
6 entity Control_Unit is
7     Port (
8         start: in std_logic ; --segnale esterno di start
9         write : in std_logic;
10        clk,rst: in std_logic;--clock e reset
11        count_out: in std_logic_vector(2 downto 0);--uscita
            contatore
12        sel_mux: out std_logic_vector(1 downto 0);--selettore
            multiplexer
13        --Q_0: out std_logic;
14        sign: in std_logic;--segnale in ingresso dalla shift
            register
15        --S_In:  in std_logic;
16        subtract: out std_logic;--segnale da porre in ingresso
            al parallel adder
17        load,loadM,shift: out std_logic;--shift register
18        count_in: out std_logic;--valore del conteggio in
            uscita dal contatore
19        azzera: out std_logic
20    );
21 end Control_Unit;
22
23 architecture Behavioral of Control_Unit is
24
25 type state is (IDLE,CARICA_REG,ACQUISISCI,AVVIA_SOMMA,
            SHIFT_STATE,SET_Q0,INCR_COUNT,correction_state,FINE);
```

```
26 signal current_state,next_state: state;
27
28
29 begin
30
31
32
33 reg_stato: process(clk)
34     begin
35         if(clk'event and clk='1') then
36             if(rst='1') then
37                 azzera<='1';
38                 current_state <=idle;
39             else
40                 current_state <=next_state;
41             end if;
42         end if;
43     end process;
44
45 comb: process(current_state, start,count_out,sign,write)
46 begin
47
48     subtract<='0';
49     load<='0';
50     loadM<='0';
51     shift<='0';
52     count_in<='0';
53     sel_mux<="00";
```

```
54      azzera<='0';
55
56
57
58
59  CASE current_state is
60
61      WHEN IDLE =>
62
63
64          if(start='1') then
65              next_state <= CARICA_REG;
66          elsif write='1' then
67              next_state<=ACQUISISCI;
68
69          else
70              next_state <= IDLE;
71          end if;
72
73      --fornisce i segnali di caricamento operandi
74      WHEN CARICA_REG=>
75
76          loadM <='1';
77
78          load<='1'; --abilita il caricamento SAQ
79                      nello shift register
80          next_state <= SHIFT_STATE;
```

```
81
82     WHEN AVVIA_SOMMA =>
83 --         if count_out="111" then
84 --             subtract<='0';
85 --             load<='1';
86 --             next_state<= CARICA_RESTO;
87 --             sel_mux<="10";
88
89 --         els
90         if sign='0' then
91             subtract<='1';
92             load<='1';
93             sel_mux<="10";
94
95
96             next_state<=SET_Q0;
97         elsif sign='1' then
98             subtract<='0';
99             load<='1';
100             sel_mux<="10";
101             next_state <= SET_Q0 ;
102
103
104         end if ;
105
106
107
108     WHEN SHIFT_STATE =>
```

```
109
110     shift <='1';
111     --Q_0<=S_In; --dubbio
112     next_state <= AVVIA_SOMMA;
113
114
115     WHEN INCR_COUNT =>
116
117         count_in <= '1';
118         if count_out="111" and sign='1' then
119             next_state<=AVVIA_SOMMA;
120         elsif count_out="111" and sign='0' then
121             next_state<=FINE;
122         else
123
124             next_state <= SHIFT_STATE;
125
126         end if;
127
128     WHEN SET_Q0 =>
129         sel_mux<="11";
130         load<='1';
131         if count_out="111" then
132             next_state<= correction_state;
133         else
134
135             next_state<=INCR_COUNT;
136         end if;
```

```
137     WHEN correction_state =>
138         if sign='1' then
139             load<='1';
140             sel_mux<="10";
141         end if;
142         next_state<=FINE;
143
144     WHEN ACQUISISCI=>
145         if write ='0' then
146             next_state<=IDLE;
147         end if;
148
149     WHEN FINE =>
150
151         azzera<='1';
152
153
154     end CASE;
155 end process;
156
157
158 end Behavioral;
```

Simulazione

Di seguito è riportato il TestBench utilizzato per testare le funzionalità precedentemente descritte:

```
1 library IEEE;
```

```
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5
6 entity tb is
7   -- Port ( );
8 end tb;
9
10 architecture Behavioral of tb is
11 component Top_module is
12   Port (
13     clk: in std_logic;
14     rst: in std_logic;
15     start: in std_logic;
16     dividendo: in std_logic_vector(7 downto 0);
17     divisore: in std_logic_vector(7 downto 0);
18     quoziente: out std_logic_vector(7 downto 0);
19     resto: out std_logic_vector(7 downto 0)
20
21   );
22
23 end component;
24
25 --component Control_Unit is
26 -- Port (
27 --   start: in std_logic ; --segnale esterno di start
28 --   clk,rst: in std_logic;--clock e reset
29 --   count_out: in std_logic_vector(2 downto 0);--uscita
```

```
    contatore
30  --      sel_mux: out std_logic;
31  --      sign: in std_logic;--segnale in ingresso dalla shift
    register
32  --      subtract: out std_logic;--segnale da porre in
    ingresso al parallel adder
33  --      load,loadM,shift: out std_logic;--shift register
34  --      count_in: out std_logic;--valore del conteggio in
    uscita dal contatore
35  --      azzera: out std_logic
36  --      );
37
38
39  --end component;
40
41  --signal start,clock,rst,sel_mux,Q_0,S,sub,load,load_M,shift,
    count_in,azzera: std_logic:='0';
42  signal ingresso_1,ingresso_2: std_logic_vector(7 downto 0):=(
    others=>'0');
43  signal quoz,resto : std_logic_vector(7 downto 0):=(others
    =>'0');
44  signal clock,rst,start: std_logic:='0';
45  constant clock_period: time:=10 ns;
46  --signal count_out : std_logic_vector(2 downto 0):=(others
    =>'0');
47
48  begin
49  CLK_process: process
```

```
50 begin
51     clock<='0';
52     wait for clock_period/2;
53     clock<='1';
54     wait for clock_period/2;
55 end process;
56 UUT: Top_module port map(clock,rst,start,ingresso_1,ingresso_2
    , quoz, resto);
57 --UUT: Control_Unit port map(start,clock,rst,count_out,sel_mux
    , S, sub, load, load_M, shift, count_in, azzer);
58 STim_pROC: process
59 begin
60     start<='1';
61     ingresso_1<="00001101";
62     ingresso_2<="00000010";
63     wait for 10 ns;
64     --start<='1';
65     --count_out<="111";
66     --s<='1';
67     wait for 1000ns ;
68     end process;
69
70 end Behavioral;
```

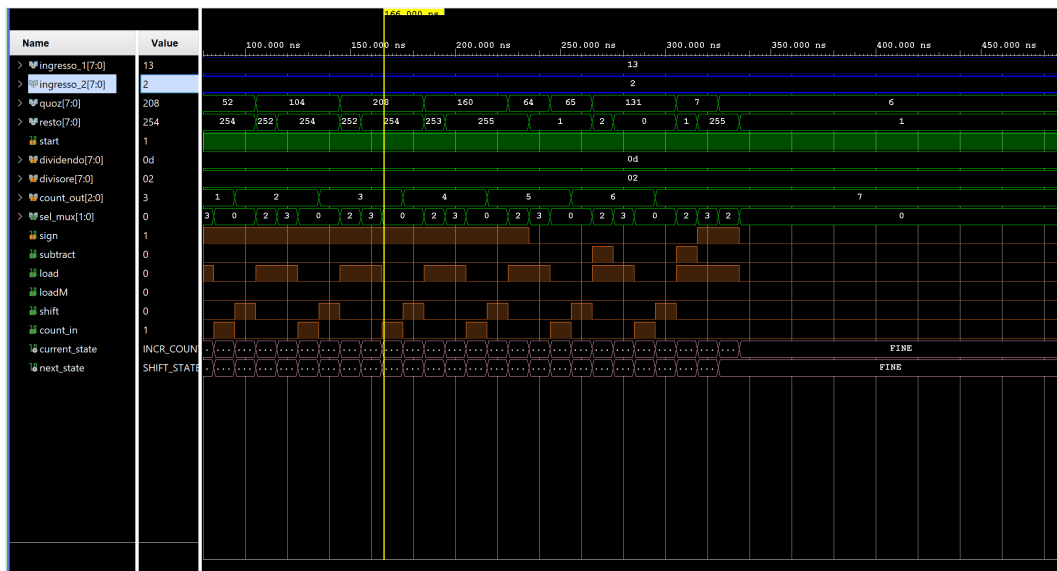


Figura 3.6: Simulazione 7 bis

3.2.2 Esercizio 7BIS.2

Sintetizzare il divisore implementato al punto 7BIS.1 su FPGA e testarlo mediante l'utilizzo dei dispositivi di input/output (switch, bottoni, led, display) presenti sulla board di sviluppo in dotazione. La modalità di utilizzo degli stessi è a completa discrezione degli studenti.

Progetto e Architettura

Implementazione

Implementazione Control Unit:

```
1      library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4
5
6  entity Control_Unit is
7      Port (
8          start: in std_logic ; --segnale esterno di start
9          write : in std_logic;
10         clk,rst: in std_logic;--clock e reset
11         count_out: in std_logic_vector(2 downto 0);--uscita
12             contatore
13         sel_mux: out std_logic_vector(1 downto 0);--selettore
14             multiplexer
15         --Q_0: out std_logic;
16         sign: in std_logic;--segnale in ingresso dalla shift
17             register
```

```
15      --S_In:  in std_logic;
16      subtract: out std_logic;--segnale da porre in ingresso
           al parallel adder
17      load,loadM,shift: out std_logic;--shift register
18      count_in: out std_logic;--valore del conteggio in
           uscita dal contatore
19      azzera: out std_logic;
20      sel_end : out std_logic
21      );
22 end Control_Unit;
23
24 architecture Behavioral of Control_Unit is
25
26 type state is (IDLE,CARICA_REG,ACQUISISCI,AVVIA_SOMMA,
           SHIFT_STATE,SET_Q0,INCR_COUNT,correction_state,FINE);
27 signal current_state,next_state: state;
28
29
30 begin
31
32
33
34 reg_stato: process(clk,rst)
35     begin
36         if(clk'event and clk='1') then
37             if(rst='1') then
38                 azzera<='1';
39                 current_state <=idle;
```

```
40
41         else
42             current_state <=next_state;
43         end if;
44     end if;
45 end process;
46
47 comb: process(current_state, start,count_out,sign,write)
48 begin
49
50     subtract<='0';
51     load<='0';
52     loadM<='0';
53     shift<='0';
54     count_in<='0';
55     sel_mux<="00";
56     azzera<='0';
57     sel_end<='0';
58
59
60
61
62 CASE current_state is
63
64     WHEN IDLE =>
65
66
67         if(start='1') then
```

```
68         next_state <= CARICA_REG;
69     elsif write='1' then
70         next_state<=ACQUISISCI;
71
72     else
73         next_state <= IDLE;
74     end if;
75
76     --fornisce i segnali di caricamento operandi
77     WHEN CARICA_REG=>
78
79         loadM <='1';
80
81         load<='1'; --abilita il caricamento SAQ
82         nello shift register
83         next_state <= SHIFT_STATE;
84
85     WHEN AVVIA_SOMMA =>
86     --         if count_out="111" then
87     --             subtract<='0';
88     --             load<='1';
89     --             next_state<= CARICA_RESTO;
90     --             sel_mux<="10";
91
92     --         els
93     if sign='0' then
94         subtract<='1';
```

```
95         load<='1';
96         sel_mux<="10";
97
98
99         next_state<=SET_Q0;
100     elsif sign='1' then
101         subtract<='0';
102         load<='1';
103         sel_mux<="10";
104         next_state <= SET_Q0 ;
105
106
107     end if ;
108
109
110
111     WHEN SHIFT_STATE =>
112
113         shift <='1';
114         --Q_0<=S_In; --dubbio
115         next_state <= AVVIA_SOMMA;
116
117
118     WHEN INCR_COUNT =>
119
120         count_in <= '1';
121         if count_out="111" and sign='1' then
122             next_state<=AVVIA_SOMMA;
```

```
123         elsif count_out="111" and sign='0' then
124             next_state<=FINE;
125         else
126
127             next_state <= SHIFT_STATE;
128
129         end if;
130
131     WHEN SET_Q0 =>
132         sel_mux<="11";
133         load<='1';
134         if count_out="111" then
135             next_state<= correction_state;
136         else
137
138             next_state<=INCR_COUNT;
139         end if;
140     WHEN correction_state =>
141         if sign='1' then
142             load<='1';
143             sel_mux<="10";
144         end if;
145         next_state<=FINE;
146
147     WHEN ACQUISISCI=>
148         if write ='0' then
149             next_state<=IDLE;
150         end if;
```

```
151
152     WHEN FINE =>
153         sel_end<='1';
154         --azzera<='1';
155         next_state<=FINE;
156
157     end CASE;
158 end process;
159
160
161 end Behavioral;
```

Implementazione Top Module:

```
1      --
2
3      -----
4
5      -- Company:
6      -- Engineer:
7      --
8      -- Create Date: 09.04.2024 14:15:25
9      -- Design Name:
10     -- Module Name: arch - Behavioral
11     -- Project Name:
12     -- Target Devices:
13     -- Tool Versions:
14     -- Description:
15     --
16     -- Dependencies:
17     --
```

```
15 -- Revision:
16 -- Revision 0.01 - File Created
17 -- Additional Comments:
18 --
19 --
20 -----
21
22 library IEEE;
23 use IEEE.STD_LOGIC_1164.ALL;
24 use IEEE.numeric_STD.all;
25
26
27 -- Uncomment the following library declaration if using
28 -- arithmetic functions with Signed or Unsigned values
29 --use IEEE.NUMERIC_STD.ALL;
30
31 -- Uncomment the following library declaration if
32 -- instantiating
33 -- any Xilinx leaf cells in this code.
34 --library UNISIM;
35 --use UNISIM.VComponents.all;
36
37 entity divisore is
38 Port (
39     CLOCK: in std_logic;
40     RESET: in std_logic;
```

```
40     START: in std_logic;
41     WRITE: in std_logic;
42     switch: in std_logic_vector(15 downto 0);
43     cathodes_out: out std_logic_vector(7 downto 0);
44     anodes_out: out std_logic_vector(7 downto 0)
45
46
47 );
48 end divisore;
49
50
51
52 architecture Behavioral of divisore is
53
54     component ButtonDebouncer is
55         generic (
56             CLK_period: integer := 10; -- periodo del clock
57                                     della board 10 nanosecondi
58             btn_noise_time: integer := 6500000 --intervallo di
59                                     tempo in cui si ha l'oscillazione del bottone
60                                     --assumo che duri 6.5
61                                     ms=6500microsec
62                                     =6500000ns
63         );
64     port ( RST : in STD_LOGIC;
65           CLK : in STD_LOGIC;
66           BTN : in STD_LOGIC;
67           CLEARED_BTN : out STD_LOGIC);
```

```
64
65 end component;
66
67
68 component Top_module is
69   Port (
70     clk: in std_logic;
71     rst: in std_logic;
72     start: in std_logic;
73     write: in std_logic;
74     dividendo: in std_logic_vector(7 downto 0);
75     divisore: in std_logic_vector(7 downto 0);
76     quoziente: out std_logic_vector(7 downto 0);
77     resto: out std_logic_vector(7 downto 0);
78     sel_end: out std_logic
79
80   );
81 end component;
82
83 component blocco_conversione is
84   Port (
85     number: in integer;
86     uscita: out std_logic_vector(31 downto 0)
87
88   );
89 end component;
90
91 component display_seven_segments is
```

```

92  Generic (
93      CLKIN_freq : integer := 100000000;
94      CLKOUT_freq : integer := 500
95  );
96  Port ( CLK : in  STD_LOGIC;
97      RST : in  STD_LOGIC;
98      VALUE : in  STD_LOGIC_VECTOR (31 downto 0);
99      ENABLE : in  STD_LOGIC_VECTOR (7 downto 0); --
      decide quali cifre abilitare
100     DOTS : in  STD_LOGIC_VECTOR (7 downto 0); -- decide
      quali punti visualizzare
101     ANODES : out STD_LOGIC_VECTOR (7 downto 0);
102     CATHODES : out STD_LOGIC_VECTOR (7 downto 0));
103 end component;
104
105 component mux_21 is
106 Port (
107     A,B: in std_logic_vector(31 downto 0);
108     sel: in std_logic;
109     C: out std_logic_vector(31 downto 0)
110 );
111
112 end component;
113
114 signal quoziente,resto,dividendo,divisore : std_logic_vector(7
      downto 0):=(others=>'0');
115 signal risultato: integer;
116 signal data_for_display,digit,digit0:std_logic_vector(31

```

```
    downto 0) := (others => '0');
117 signal cleared_start, cleared_write, sel_end: std_logic := '0';
118 begin
119
120 digit0 <= "00000000000000000000000000000000";
121 --update_risultato: process (quoziante)
122 --begin
123 --risultato <= to_integer(unsigned(quoziante));
124
125
126 dividendo <= switch(15 downto 8);
127 divisore <= switch(7 downto 0);
128
129 --Button_deb: ButtonDebouncer port map(RESET, CLOCK, START,
    cleared_start);
130 --Button_debl: ButtonDebouncer port map(RESET, CLOCK, WRITE,
    cleared_write);
131 Divisore_non_RESTORING : Top_module port map(CLOCK, RESET, START
    , WRITE, dividendo, divisore, quoziante, resto, sel_end);
132
133 risultato <= to_integer(unsigned(quoziante));
134 --risultato <= to_integer(unsigned(quoziante));
135
136 CONVERSION_BLOCK: blocco_conversione port map(risultato,
    data_for_display);
137
138 Mux_2_1: mux_21 port map(digit0, data_for_display, sel_end, digit
    );
```

```
139
140 Display_a_sette_segmenti: display_seven_segments generic map
    (100000000,500)
141 port map(CLK=>CLOCK,RST=>RESET,VALUE=>digit,ENABLE=>"1111111"
    ,DOTS=>"0000000",ANODES=>anodes_out,CATHODES=>cathodes_out)
    ;
142
143
144
145
146 end Behavioral;
```

Simulazione

Di seguito è riportata una sequenza di immagini che rappresentano una divisione tra 18 e 2.

PRIMO PASSO: nello stato di idle alziamo gli interruttori che rappresentano in binario i numeri 18 ed 2.

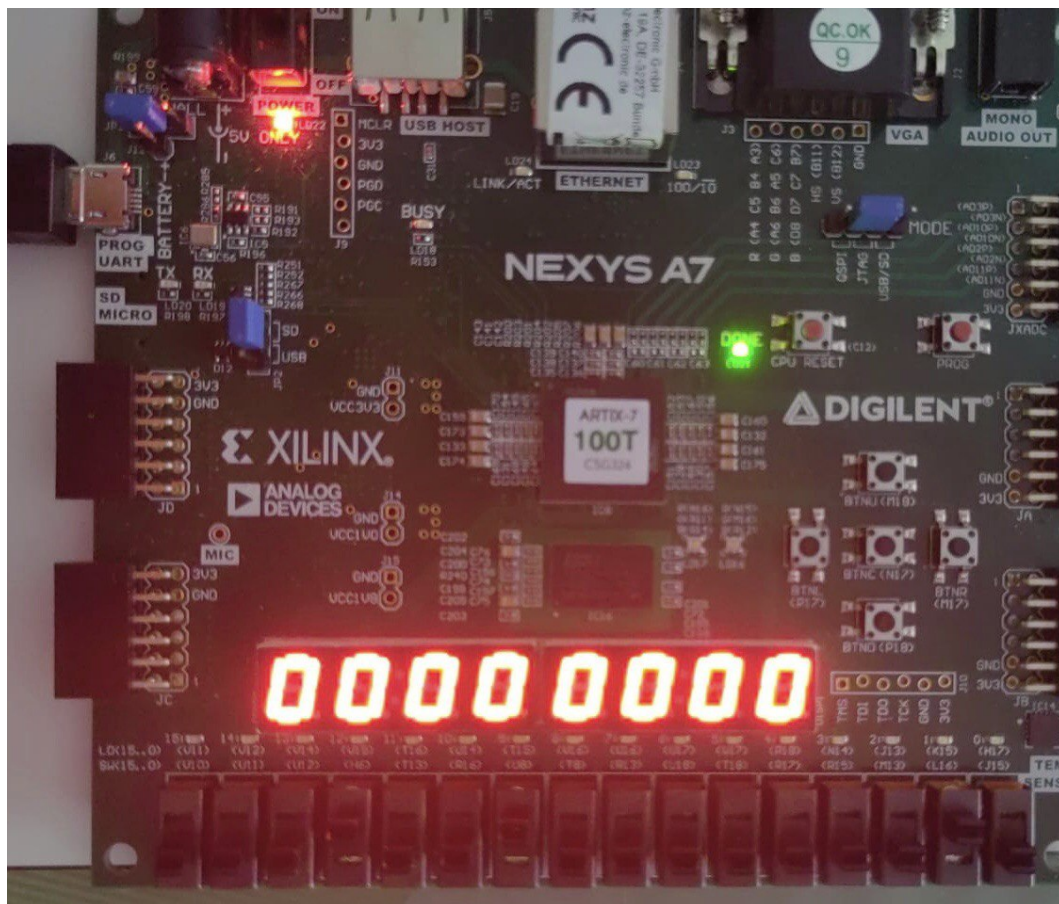


Figura 3.7: idle

SECONDO PASSO: premendo il bottone centrale per acquisire gli operandi.

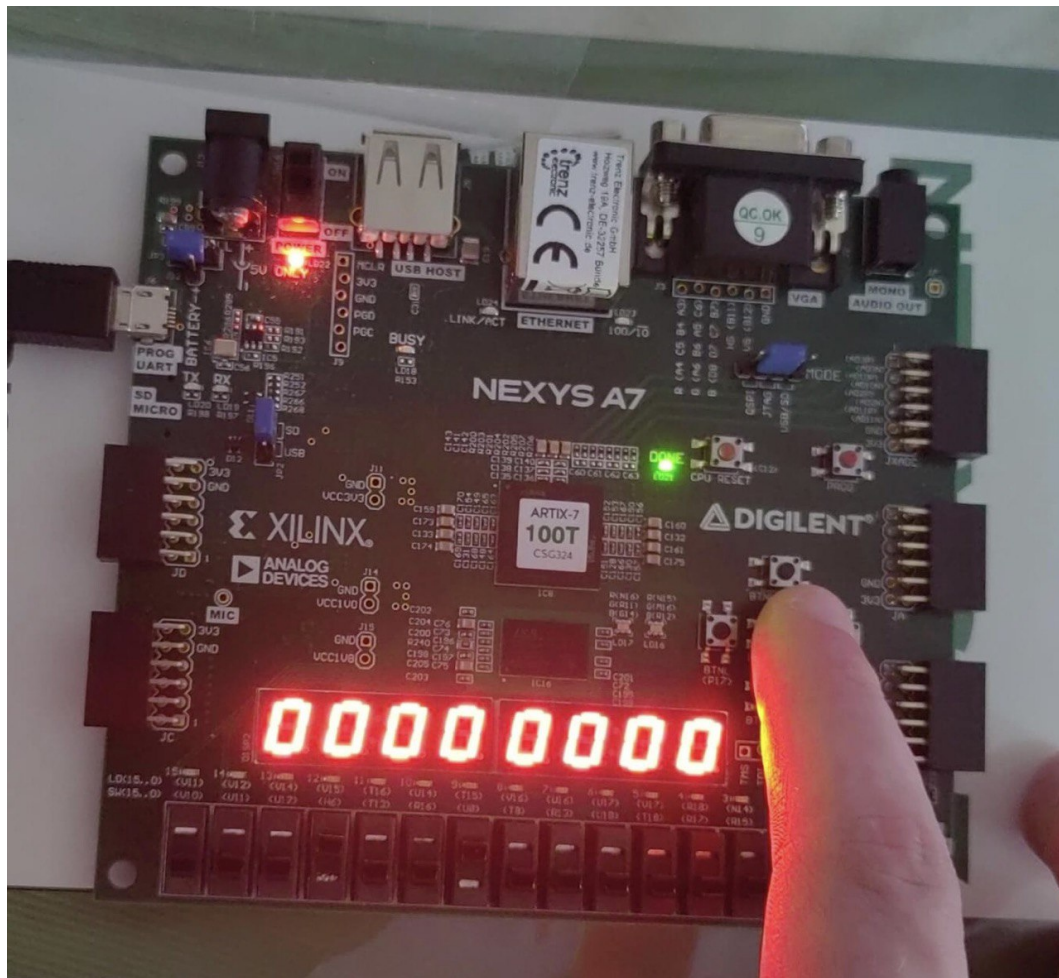


Figura 3.8: Acquisizione

TERZO PASSO: risultato. I bit cerchiato in blu è il resto e i restanti il
queziente.

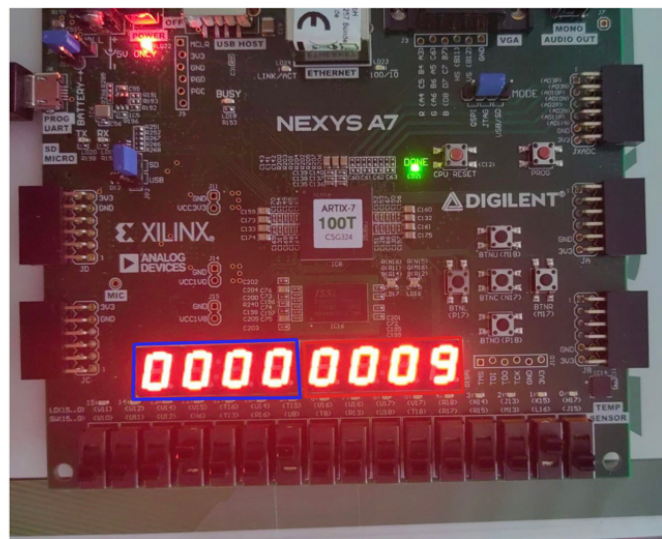


Figura 3.9: risultato

Capitolo 4

Comunicazione con Handshaking

4.1 Esercizio 8: comunicazione con handshaking

4.1.1 Esercizio 8.1

Progettare, implementare in VHDL e testare mediante simulazione un sistema composto da 2 nodi, A e B, che comunicano mediante un protocollo di handshaking. Il nodo A e il nodo B possiedono entrambi una memoria interna in cui sono memorizzate N stringhe di M bit, denominate $X(i)$ e $Y(i)$ rispettivamente ($i=0, \dots, N-1$). Il nodo A trasmette a B ciascuna stringa $X(i)$ utilizzando un protocollo di handshaking; B, ricevuta la stringa $X(i)$, calcola $S(i)=X(i)+Y(i)$ e immagazzina la somma in opportune locazioni della propria memoria interna.

Per il progetto è possibile considerare una implementazione di tipo comportamentale per effettuare la somma, mentre è necessario prevedere esplicitamente un componente contatore sia nel sistema A sia nel sistema B per scandire la trasmissione/ricezione delle stringhe e per terminare la comunicazione

Progetto e Architettura

Come richiesto sono stati creati due nodi A e B che comunicano mediante il protocollo di handshaking. Il protocollo di handshaking prevede uno scambio diretto di messaggi tra due nodi comunicanti. Nel nostro caso utilizziamo un handshaking completo, dove la comunicazione è gestita da due segnali:

- req: è il segnale utilizzato dal nodo trasmettitore (nodo A) per segnalare la volontà di trasmettere un dato;
- ack: è il segnale utilizzato dal nodo ricevitore (nodo B) per segnalare quando è alto (ack='1') che è impegnato in un'operazione, quando è basso che l'operazione è completa e può ricevere un altro carattere.

L'architettura comprende due nodi:

- A, contenente:
 - Control Unit: gestisce i segnali esterni quali clock, start, reset, count_out, count_in, address, read, req e ack;
 - contatore: conta gli n elementi da inviare;

- ROM: array contenente gli n elementi;
- B, contenente:
 - Control Unit: gestisce i segnali esterni quali clock, reset, count_out, count_in, address, read, write, read_reg, req e ack;
 - contatore: indica la locazione di memoria dalla quale leggere/-scrivere l'elemento;
 - Memoria: array contenente gli n elementi e le n somme;
 - Registro: registro dove verrà memorizzato temporaneamente il dato ricevuto;
 - adder: circuito per addizionare gli elementi.

In seguito è fornita l'architettura dell'intero sistema:

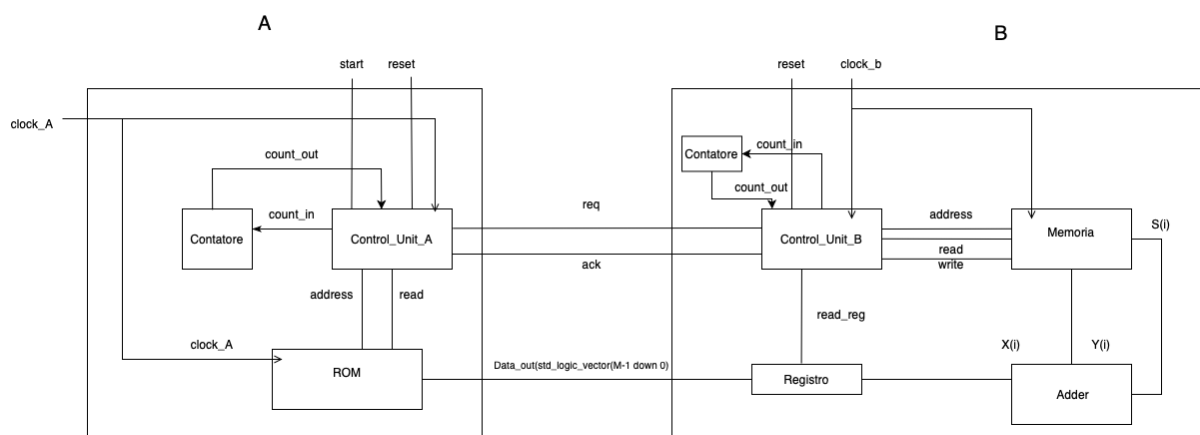


Figura 4.1: Architettura 8

Successivamente sono illustrati gli automi del nodo A e del nodo B:

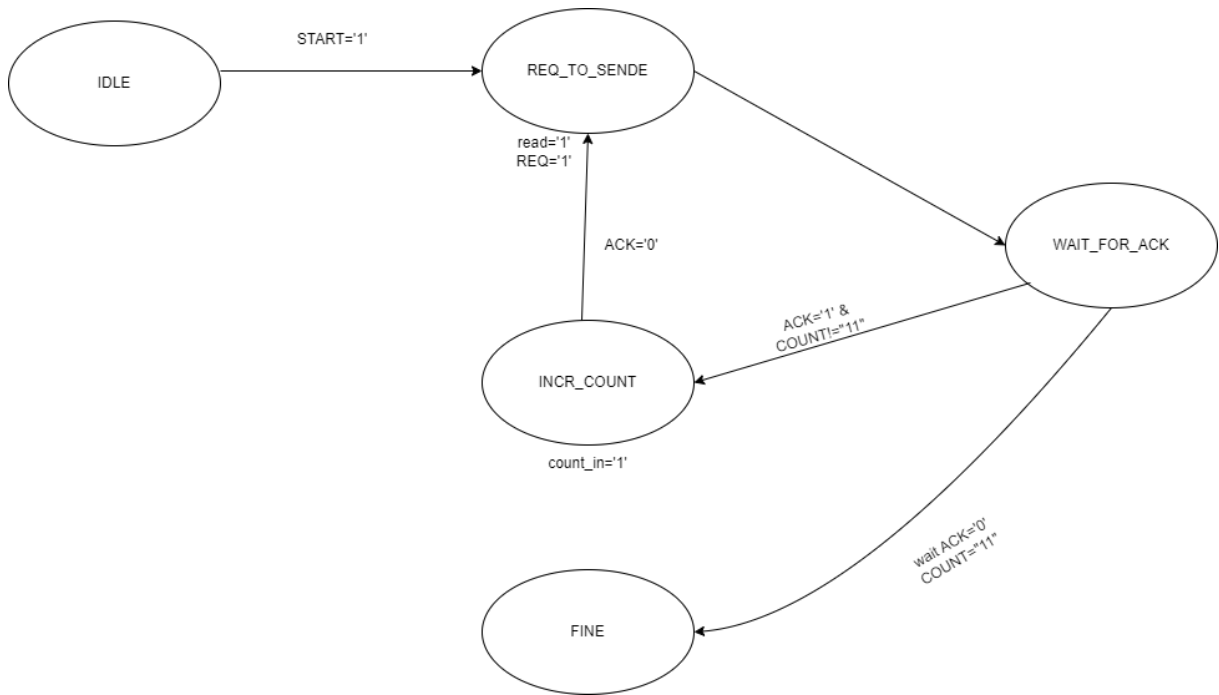


Figura 4.2: Automa nodo A

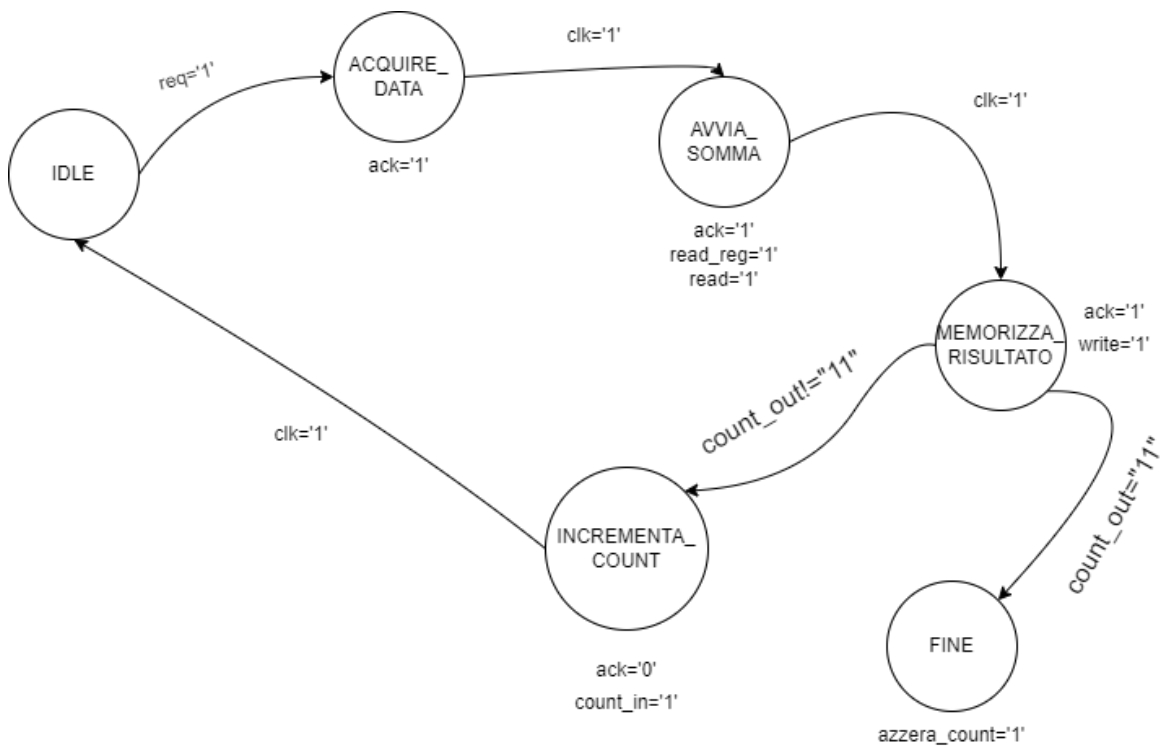


Figura 4.3: Automa nodo B

Implementazione

Implementazione Control Unit nodo A:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5 entity Control_Unit_A is
6   Generic(
7       Log: integer:=2
8   );
9
10  Port (
11      clk_a: in std_logic;
12      reset: in std_logic;
13      start: in std_logic;
14      ack: in std_logic;
15      count_out: in std_logic_vector(Log-1 downto 0);
16      address : out std_logic_vector(Log-1 downto 0);
17      count_in : out std_logic;
18      reset_count: out std_logic;
19      read: out std_logic;
20      req:out std_logic
21  );
22  end Control_Unit_A;
23
24  architecture Behavioral of Control_Unit_A is
25
26
```

```
27
28 type state is (IDLE, REQ_TO_SEND, WAIT_FOR_ACK, INCR_COUNT, FINE);
29 signal current_state, next_state: state;
30
31
32 begin
33
34
35
36 reg_stato: process(clk_A)
37     begin
38         if(clk_a'event and clk_a='1') then
39             if(reset='1') then
40                 reset_count<='1';
41                 current_state <=idle;
42             else
43                 current_state <=next_state;
44             end if;
45         end if;
46     end process;
47
48 comb: process(current_state, start, count_out, ack)
49 begin
50
51     read<='0';
52     count_in<='0';
53     address<=count_out;
54     req<='0';
```

```
55     reset_count<='0';
56
57 CASE current_state is
58
59     WHEN IDLE =>
60
61
62         if(start='1') then
63             next_state <= REQ_TO_SEND;
64
65         else
66             next_state <= IDLE;
67         end if;
68
69     --fornisce i segnali di caricamento operandi
70     WHEN REQ_TO_SEND=>
71
72         read<='1';
73         req<='1';
74         --address<=count_out;
75         next_state <= WAIT_FOR_ACK;
76
77     --acquisisce gli operandi, su cui il sommatore inizia
78     a lavorare immediatamente
79     WHEN WAIT_FOR_ACK=>
80         --
81         req<='1';
82         if ack='1' then
```

```
82         if count_out="11" then
83
84             next_state<= FINE;
85         else
86
87             next_state<=INCR_COUNT;
88             end if;
89         end if;
90
91
92     WHEN INCR_COUNT =>
93         count_in<='1';
94
95         if ack<='0' then
96             next_state<=REQ_TO_SEND;
97             end if;
98
99
100    WHEN FINE =>
101        reset_count<='1';
102        next_state<= FINE;
103
104
105
106
107
108    end CASE;
109 end process;
```

```
110  
111  
112 end Behavioral;
```

Architettura nodo A:

```
1      library IEEE;  
2  use IEEE.STD_LOGIC_1164.ALL;  
3  
4  
5  
6  entity A is  
7  generic (  
8      number_of_bit:integer:=4;  
9      Log: integer:=2  
10 );  
11  
12 Port (  
13     clk_a: in std_logic;  
14     reset: in std_logic;  
15     START: in std_logic;  
16     ack: in std_logic;  
17     req:out std_logic;  
18     data_out:out std_logic_vector(number_of_bit -1 downto  
19         0)  
20 );  
21 end A;  
22  
23 architecture Behavioral of A is
```

```
24
25
26 component Control_Unit_A is
27 Generic(
28     Log: integer:=2
29 );
30
31 Port (
32     clk_a: in std_logic;
33     reset: in std_logic;
34     start: in std_logic;
35     ack: in std_logic;
36     count_out: in std_logic_vector(Log-1 downto 0);
37     address : out std_logic_vector(Log-1 downto 0);
38     count_in : out std_logic;
39     reset_count: out std_logic;
40     read: out std_logic;
41     req:out std_logic
42 );
43 end component;
44
45 component cont_mod8 is
46     generic(
47         Log: integer := 2
48     );
49
50     port( clock, reset: in std_logic;
51         count_in: in std_logic;
```

```
52         count: out std_logic_vector(Log-1 downto 0));
53 end component;
54
55
56 component ROM is
57     generic (
58         Number_of_bit: integer:=4;
59         Number_of_locations: integer :=4;
60         Log: integer :=2 --logaritmo in base due di
61             number_of_location
62     );
63
64     Port (
65         read: in std_logic;
66         address :in std_logic_vector(Log-1 downto 0);
67         dout: out std_logic_vector (Number_of_locations-1
68             downto 0)
69     );
70
71 end component;
72
73
74
75 signal count_out,address: std_logic_vector(1 downto 0):=(
76     others=>'0');
77
78 signal count_in,reset_count,read : std_logic:='0';
79
80 begin
81
82     --Control _unit
83     CU: Control_Unit_A generic map(2) port map(clk_a,reset,START,
```

```

    ack,count_out,address,count_in,reset_count,read,req);
77 --Contatore
78 Contatore: cont_mod8 generic map(2) port map(clk_a,reset_count
    ,count_in,count_out);
79 -- ROM
80 Memoria_A:ROM generic map(4,4,2) port map(read,address,
    data_out);
81
82
83 end Behavioral;
```

Control Unit nodo B:

```

1    library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5 entity Control_Unit_B is
6     Port (
7         clock: in std_logic;
8         reset: in std_logic;
9         count_out : in std_logic_vector(1 downto 0);
10        req: in std_logic;
11        ack: out std_logic;
12        count_in :out std_logic;
13        read,write,read_reg: out std_logic;
14        address: out std_logic_vector(2 downto 0);
15        reset_count: out std_logic
16    );
17 end Control_Unit_B;
```

```
18
19 architecture Behavioral of Control_Unit_B is
20
21 type state is (IDLE, ACQUIRE_DATA, AVVIA_SOMMA, MEMORIZZA,
    INCR_COUNT, FINE);
22 signal current_state, next_state: state;
23
24
25 begin
26
27 reg_stato: process(clock, reset)
28     begin
29         if(clock'event and clock='1') then
30             if(reset='1') then
31                 reset_count<='1';
32                 current_state <=idle;
33             else
34                 current_state <=next_state;
35             end if;
36         end if;
37     end process;
38
39 comb: process(current_state, req, count_out)
40 begin
41
42     read<='0';
43     read_reg<='0';
44     ack<='0';
```

```
45     write<='0';
46     count_in<='0';
47     address<='0' & count_out;
48     reset_count<='0';
49
50 CASE current_state is
51
52     WHEN IDLE =>
53
54
55         if (req='1') then
56             next_state <= ACQUIRE_DATA;
57
58         else
59             next_state <= IDLE;
60         end if;
61
62     --fornisce i segnali di caricamento operandi
63     WHEN ACQUIRE_DATA=>
64
65
66         ack<='1';
67         next_state <= AVVIA_SOMMA;
68
69     --acquisisce gli operandi, su cui il sommatore inizia
70     a lavorare immediatamente
71     WHEN AVVIA_SOMMA=>
```

```
72         read_reg<='1';
73         read<='1';
74         ack<='1';
75
76         next_state<= MEMORIZZA;
77
78     WHEN MEMORIZZA =>
79
80         write<='1';
81         ack<='1';
82         if count_out="11" then
83
84             next_state<=FINE;
85
86         else
87             next_state<= INCR_COUNT;
88         END IF;
89
90     WHEN INCR_COUNT =>
91         count_in<='1';
92         ack<='0';
93         next_state<= IDLE;
94
95
96
97
98     WHEN FINE =>
99         reset_count<='1';
```

```
100         ack<='1';
101
102         next_state<= FINE;
103
104
105     end CASE;
106 end process;
107
108
109 end Behavioral;
```

Architettura nodo B:

```
1     library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5
6 entity B is
7     generic (
8         lunghezza_stringa: integer:=4
9     );
10
11 Port (
12     clock : in std_logic;
13     reset: in std_logic;
14     req: in std_logic;
15     x_out: in std_logic_vector(lunghezza_stringa-1 downto 0);
16     ack: out std_logic;
17     somma_out: out std_logic_vector(lunghezza_stringa-1 downto
```

```
        0)

18    );
19    end B;
20
21    architecture Behavioral of B is
22
23
24    component Control_Unit_B is
25        Port (
26            clock: in std_logic;
27            reset: in std_logic;
28            count_out : in std_logic_vector(1 downto 0);
29            req: in std_logic;
30            ack: out std_logic;
31            count_in :out std_logic;
32            read,write,read_reg: out std_logic;
33            address: out std_logic_vector(2 downto 0);
34            reset_count: out std_logic
35        );
36    end component;
37
38    component Registro_4 is
39        generic(
40
41            lunghezza_dato: integer:=4
42        );
43
44        Port (
```

```
45     clk: in std_logic;
46     reset: in std_logic;
47     data_in: std_logic_vector(lunghezza_dato-1 downto 0);
48     read: in std_logic;
49     x_i:out std_logic_vector(lunghezza_dato-1 downto 0)
50
51 );
52 end component;
53
54 component cont_mod8 is
55     generic(
56         Log: integer := 2
57     );
58
59     port( clock, reset: in std_logic;
60           count_in: in std_logic;
61           count: out std_logic_vector(Log-1 downto 0));
62 end component;
63
64 --MEMORIA
65 component mem is
66     generic(
67         lunghezza_indirizzo: integer := 3;
68         lunghezza_dati: integer := 4;
69         number_of_location: integer:= 8
70
71 );
72 port (
```

```

73  clock: in std_logic;
74  reset: in std_logic;
75  read: in std_logic;
76  write: in std_logic;
77  address: in std_logic_vector((lunghezza_indirizzo - 1)
      downto 0);
78  somma_in: in std_logic_vector((lunghezza_dati- 1) downto
      0);
79  somma: out std_logic_vector((lunghezza_dati-1) downto 0);
80  data_output: out std_logic_vector ((lunghezza_dati - 1)
      downto 0) := "0000"
81 );
82 end component;
83 --adder
84 component adder is
85     generic (
86         lunghezza_dati: integer:=4
87     );
88
89     port( X, Y: in std_logic_vector(lunghezza_dati-1 downto 0)
      ;
90         Z: out std_logic_vector(lunghezza_dati-1 downto 0)
91     );
92
93 end component;
94
95 signal count_out: std_logic_vector(1 downto 0) := (others=>'0');
96 signal read, write, read_reg, reset_count, count_in: std_logic

```

```

:= '0';
97 signal address: std_logic_vector(2 downto 0) := (others => '0');
98 signal stringa_x, stringa_y, somma_in, risultato:
    std_logic_vector(3 downto 0) := (others => '0');
99
100 begin
101
102 CU_B: Control_Unit_B port map(clock, reset, count_out, req, ack,
    count_in, read, write, read_reg, address, reset_count);
103 Registro_provvisorio: Registro_4 generic map(4) port map(clock,
    reset, x_out, read_reg, stringa_x);
104 Contatore: Cont_mod8 generic map(2) port map(clock, reset_count
    , count_in, count_out);
105
106 ---Memoria
107 RAM: mem generic map (3, 4, 8) port map(clock, reset, read, write,
    address, somma_in, risultato, stringa_y);
108
109 --ADDER
110 ADDIZIONATORE: adder generic map(4) port map(stringa_x,
    stringa_y, somma_in);
111 somma_out <= risultato;
112
113
114 end Behavioral;

```

Implementazione Top Module:

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;

```

```
3
4
5
6 entity Top_module is
7 generic (
8     lunghezza_stringa: integer := 4
9
10 );
11
12
13 Port (
14     clk_A: in std_logic;
15     clk_B: in std_logic;
16     reset: in std_logic;
17     start: in std_logic;
18     somma_out: out std_logic_vector(lunghezza_stringa-1 downto
19         0)
20 );
21 end Top_module;
22
23
24
25 architecture Behavioral of Top_module is
26
27 component A is
28 generic (
29     number_of_bit: integer:=4;
30     Log: integer:=2
```

```
30
31 Port (
32     clk_a: in std_logic;
33     reset: in std_logic;
34     START: in std_logic;
35     ack: in std_logic;
36     req: out std_logic;
37     data_out: out std_logic_vector(number_of_bit -1 downto
        0)
38
39 );
40 end component;
41
42
43
44 component B is
45     generic (
46         lunghezza_stringa: integer:=4
47     );
48
49     Port (
50         clock : in std_logic;
51         reset: in std_logic;
52         req: in std_logic;
53         x_out: in std_logic_vector(lunghezza_stringa-1 downto 0);
54         ack: out std_logic;
55         somma_out: out std_logic_vector(lunghezza_stringa-1 downto
            0)
```

```
56 );
57 end component;
58
59
60 signal ack, req: std_logic := '0';
61 signal X_i: std_logic_vector(lunghezza_stringa-1 downto 0) := (
    others => '0');
62 begin
63
64 NODO_A: A generic map(4,2) port map(clk_A, reset, start, ack, req,
    X_i);
65 NODO_B: B generic map(4) port map(clk_B, reset, req, X_i, ack,
    somma_out);
66
67
68
69
70 end Behavioral;
```

Simulazione

Testbench Top Module:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5 entity TB is
6 -- Port ( );
```

```
7 end TB;
8
9 architecture Behavioral of TB is
10 component Top_module is
11 generic (
12     lunghezza_stringa: integer := 4
13 );
14 );
15
16
17 Port (
18     clk_A: in std_logic;
19     clk_B: in std_logic;
20     reset: in std_logic;
21     start: in std_logic;
22     somma_out: out std_logic_vector(lunghezza_stringa-1 downto
23         0)
24 );
25 end component;
26
27 signal clk_A, clk_B, rst, START: std_logic := '0';
28 signal risultato: std_logic_vector(3 downto 0) := (others => '0');
29
30 constant clock_period_A: time := 10 ns;
31 constant clock_period_B: time := 28 ns;
32
33
```

```
34
35
36 begin
37 CLK_process_A: process
38 begin
39     clk_A<='0';
40     wait for clock_period_A/2;
41     clk_A<='1';
42     wait for clock_period_A/2;
43 end process;
44
45 CLK_process_B: process
46 begin
47     clk_B<='0';
48     wait for clock_period_B/2;
49     clk_B<='1';
50     wait for clock_period_B/2;
51 end process;
52
53 UUT:Top_module generic map(4) port map(clk_A,clk_B,rst,START,
    risultato);
54 STIM_PROC: process
55 BEGIN
56 rst<='1';
57 wait for 10 ns;
58 rst<='0';
59 START<='1';
60 wait for 12 ns;
```

```

61 START<='0';
62
63
64
65
66
67 wait;
68 end process;
69
70 end Behavioral;

```

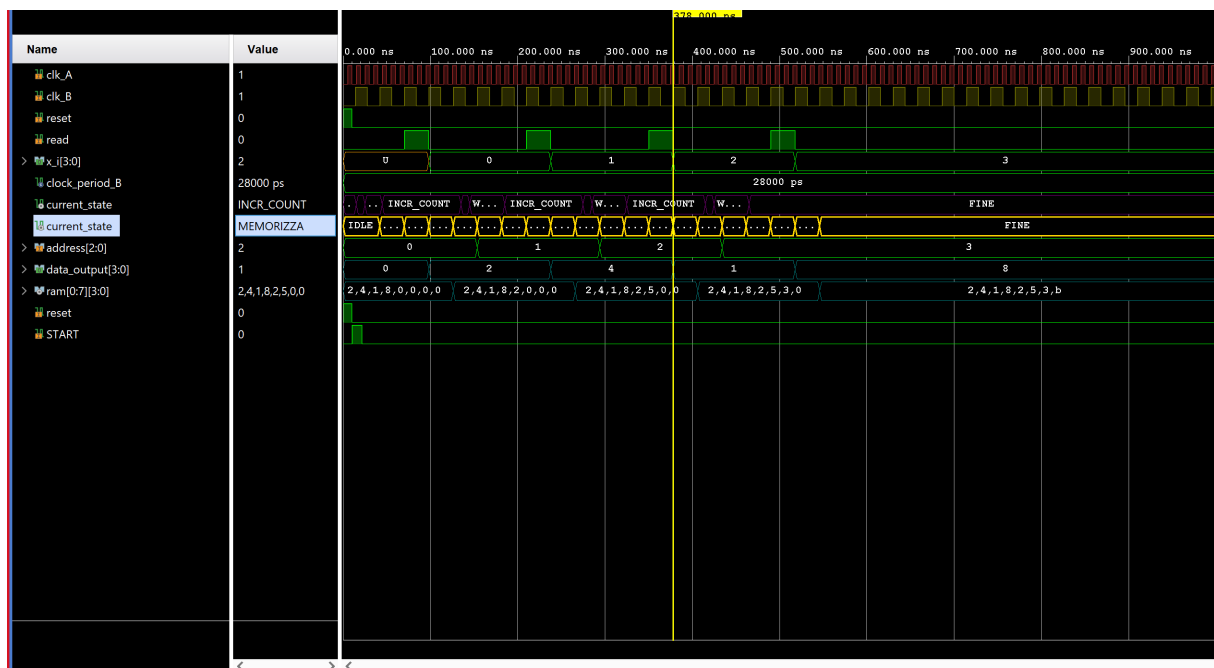


Figura 4.4: Simulazione 8

Capitolo 5

Esercizio 9

5.0.1 Esercizio 9 a

Isub

Analisi microistruzione **isub**:

```
17 isub = 0x5C:  
18     MAR = SP = SP - 1; rd  
19     H = TOS  
20     MDR = TOS = MDR - H; wr; goto main
```

Figura 5.1: isub

L'istruzione **isub** permette di effettuare un'operazione di tipo aritmetico, una sottrazione tra due operandi. Il microcodice presente nella figura 5.1 presenta a riga 17 una label che identifica l'istruzione e la locazione della control store a partire dalla quale verrà posizionata la microprocedura (*0x5C*). In seguito, sono presenti 3 microistruzioni:

- **MAR = SP = SP - 1; rd** : assegna al registro *MAR* il valore puntato dallo *Stack Pointer* decrementato di 1;

- **H=TOS** : assegna all'*Holding Register* il valore presente sul *Top Of Stack*;
- **MDR= TOS= MDR - H; wr; goto main** : in questa linea di codice abbiamo la differenza tra il valore contenuto nel *MDR* e quello contenuto nel registro *H*. Tale risultato viene memorizzato in cima allo stack, dunque viene sovrascritto il *TOS*. Il contenuto del *TOS* viene copiato nel *MDR* per renderlo disponibile all'operazione di scrittura specificata tramite *wr*. Infine, grazie al *goto main* (istruzione di salto incondizionato nella quale viene specificato l'offset al partire dal quale ci sarà l'istruzione successiva, in questo caso il main), termina l'operazione e si passa allo stato di **Idle**.

Questa microistruzione è fondamentale in quanto incrementa il *Program Counter* ed esegue il fetch del successivo byte di istruzioni, infatti il registro *MBR* contiene l'*opcode* successivo.

```

6 main:
7     PC = PC + 1; fetch; goto (MBR)
8

```

Figura 5.2: Main

Il formato delle microistruzioni che adotteremo è il seguente:

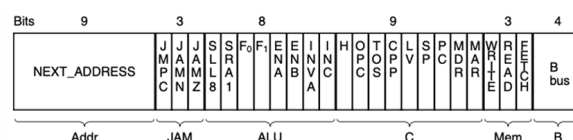


Figura 5.3: Formato microistruzione Mic-1

Le microistruzioni sono rappresentate su 36 bit dove ogni microistruzione specifica i campi:

- **ADDR**: indirizzo prossima microistruzione;
- **JAM**: determina com'è selezionata la prossima microistruzione;
- **ALU**: gestisce ALU e shift register; le possibili funzioni sono riportate nella seguente tabella:

F_0	F_1	ENA	ENB	INVA	INC	Function
0	1	1	0	0	0	A
0	1	0	1	0	0	B
0	1	1	0	1	0	$\bar{A}$
1	0	1	1	0	0	$\bar{B}$
1	1	1	1	0	0	$A + B$
1	1	1	1	0	1	$A + B + 1$
1	1	1	0	0	1	$A + 1$
1	1	0	1	0	1	$B + 1$
1	1	1	1	1	1	$B - A$
1	1	0	1	1	1	$B - 1$
1	1	1	0	1	1	$-A$
0	0	1	1	0	0	$A \text{ AND } B$
0	1	1	1	0	0	$A \text{ OR } B$
0	1	0	0	0	0	0
0	1	0	0	0	1	1
0	1	0	0	1	0	-1

Figura 5.4: Tabella funzioni ALU

- **C**: controlla quali registri vengono scritti sul bus C;
- **MEM**: regola le operazioni di memoria;
- **B**: seleziona il registro connesso al bus B codificato su 4 bit (

La **control store** è una *ROM* che contiene il microprogramma, ovvero 512×32 bit. A ogni istruzione corrisponde una microprocedura.

Figura 5.5: Control Store

```

9 92 => "001011101000001101100000010010100100",
9 93 => "001011110000000101001000000000000111",
1 94 => "000000110000001111110010000101000000",
2 95 => "001100000000001101100000000010100100",
3 96 => "001100001000000101000000000010000100",
4 97 => "001100010000000101001000000001000000",
5 98 => "001100011000000101000000000100000111",
6 99 => "001100100000001101100000000011000100",
7 100 => "000000110000000110000010000000001001",
8 101 => "001100110000001101100000010010100100",
9 102 => "001100111000000101001000000000000111",
0 103 => "000000110000001111100001000010100000",
1 104 => "000000000000000000000000000000000000",
2 105 => "000000000000000000000000000000000000",
3 106 => "000000000000000000000000000000000000",
4 107 => "000000000000000000000000000000000000",
5 108 => "000000000000000000000000000000000000",
6 109 => "000000000000000000000000000000000000",
7 110 => "000000000000000000000000000000000000",
8 111 => "000000000000000000000000000000000000",
9 112 => "000000000000000000000000000000000000",
0 113 => "000000000000000000000000000000000000",
1 114 => "000000000000000000000000000000000000",

```

Figura 5.6: Microprocedura isub

Come prima cosa, per effettuare la sottrazione, abbiamo modificato opportunamente il programma ajvm.mal presente nella cartella con il seguente percorso src/main/ajvm. Il codice è mostrato nella seguente immagine.

```

1  .main
2  .var
3  a
4  .endvar
5  BIPUSH 0xE
6  BIPUSH 0xA
7  ISUB
8  ISTORE a
9  HALT

```

Figura 5.7: programma in ajvm

Con BIPUSH abbiamo inserito nello stack i due operandi in esadecimale (14 e 10). E memorizzato il risultato in a. Una volta modificato il codice va ricreata la RAM e il control store con i seguenti comandi:

```
mlns@software-security:~/esercitazioni_mtc/mtc-$ cmake --build build --target create_control_store
Built target assemble_microprogram
Built target create_control_store
mlns@software-security:~/esercitazioni_mtc/mtc-$ cmake --build build --target create_ran
Built target assemble_program
Built target create_ran
mlns@software-security:~/esercitazioni_mtc/mtc-$ cmake --build build --target check
Built target index
analyze /home/unluna/esercitazioni_mtc/mtc-0/src/main/vhdl/common_defs.vhd
analyze /home/unluna/esercitazioni_mtc/mtc-0/src/main/vhdl/control_store.vhd
analyze /home/unluna/esercitazioni_mtc/mtc-0/src/main/vhdl/control_unit.vhd
analyze /home/unluna/esercitazioni_mtc/mtc-0/src/main/vhdl/alu.vhd
analyze /home/unluna/esercitazioni_mtc/mtc-0/src/main/vhdl/datapath.vhd
analyze /home/unluna/esercitazioni_mtc/mtc-0/src/main/vhdl/processor.vhd
analyze /home/unluna/esercitazioni_mtc/mtc-0/src/main/vhdl/db_or_ran.vhd
analyze /home/unluna/esercitazioni_mtc/mtc-0/src/test/vhdl/processor_tb.vhd
elaborate processor_tb
built target src.test.vhdl_processor_tb
analyze /home/unluna/esercitazioni_mtc/mtc-0/src/test/vhdl/alu_tb.vhd
elaborate alu_tb
Built target src.test.vhdl_alu_tb
analyze /home/unluna/esercitazioni_mtc/mtc-0/src/test/vhdl/control_unit_tb.vhd
elaborate control_unit_tb
Built target src.test.vhdl_control_unit_tb
analyze /home/unluna/esercitazioni_mtc/mtc-0/src/test/vhdl/datapath_tb.vhd
elaborate datapath_tb
Built target src.test.vhdl_datapath_tb
Test project /home/unluna/esercitazioni_mtc/mtc-0/build
Start 1: src.test.vhdl_alu_tb ..... Passed 0.03 sec
Start 2: src.test.vhdl_control_unit_tb .... Passed 0.02 sec
Start 3: src.test.vhdl_datapath_tb ..... Passed 0.02 sec
Start 4: src.test.vhdl_processor_tb ..... Passed 0.09 sec
100% tests passed, 0 tests failed out of 4

Total test time (real) = 0.18 sec
Built target check
```

Per assicurarci che la rigenerazione di control store e RAM siano andate a buon fine abbiamo modificato il testbench con il risultato atteso:

```

    reset <= '0';
    wait until mem_instr_addr = x"0000000A" and mem_data_we = '1';
    assert mem_data_out = x"00000004" report "Bad calculated value" severity failure;

```

Figura 5.8: processor_tb

A questo punto, come riportato già nella figura 5.0.1, abbiamo potuto effettuare il check. A questo punto possiamo controllare l'output nella cartella `build\test\vhdl` dove troveremo `processor_tb.vhd`

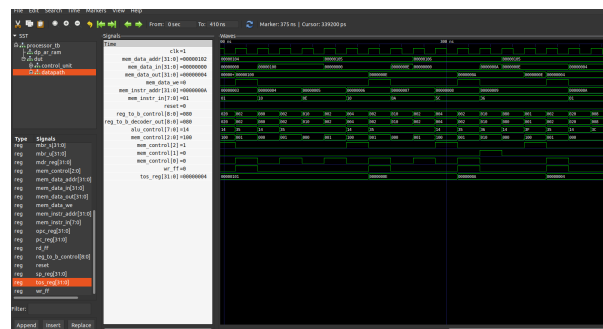


Figura 5.9: Simulazione Isub

Ifeq

Analisi microistruzione **ifeq**:

```

115 ifeq = 0x99:
116     MAR = SP = SP - 1; rd
117     OPC = TOS
118     TOS = MDR
119     Z = OPC; if (Z) goto T; else goto F

```

- **MAR= SP =SP -1; rd** : assegna al registro *MAR* il valore puntato dallo *Stack Pointer* decrementato di 1;
- **OPC=TOS** : assegna all'*OPC* il valore presente sul *Top Of Stack*;
- **TOS= MDR** : in questa linea di codice inseriamo il valore contenuto nel *MDR* al Top Of Stack (*TOS*).
- **Z= OPC; if(Z) goto T; else goto F**

: assegno il contenuto di OPC a Z e con un if e un else si salta alla label T se Z=0 e a F se Z=1;

```

9 T:
9     OPC = PC - 1; fetch; goto goto_cont
1
2 F:
3     PC = PC + 1
4     PC = PC + 1; fetch
5     goto main

```

In F viene incrementato il PC, e poi si riincrementa e si effettua una operazione di fetch per caricare la prossima istruzione nel registro MBR. In T carico in OPC il valore del program counter decrementato, effettuo una fetch e un salto incondizionato a goto cont.

```
goto_cont:
    PC = PC + 1; fetch
    H = MBR << 8
    H = MBRU OR H
    PC = OPC + H; fetch
    goto main
```

Figura 5.10: gotocont

Incremento il PC, effettuo la fetch. In seguito carico nel registro H i bit del MBR shiftati di 8, così si carica il byte più significativo dell'offset. La terza microistruzione effettua un'operazione di **OR** tra due valori: il registro **MBRU** (MBR unsigned) e **H** (il valore precedentemente calcolato dopo lo shift). Successivamente aggiorno il PC con la somma tra OPC e H e preleviamo una nuova istruzione con la fetch.

Per testare l'istruzione utilizziamo il seguente programma:

```
1  .main
2  .var
3  a
4  b
5  .endvar
6  BIPUSH 0x0
7  IFEQ END
8  BIPUSH 0xD
9  ISTORE b
10 END:
11      HALT
12 HALT
13 .endmethod
```

Come si può dedurre da questo codice, facendo un push del valore 0 nello stack e subito dopo chiamando `ifeq END` ci aspettiamo che il secondo push non avvenga mai ma che il programma salti direttamente alla label `END` e termini, come possiamo vedere dall'output della simulazione:

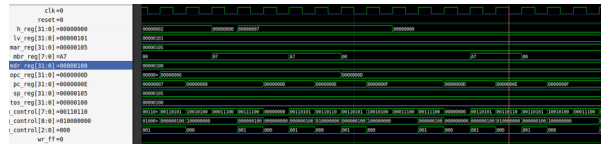


Figura 5.11: Simulazione ifeq

5.0.2 Esercizio 9 b

Il nostro obiettivo è quello di modificare il codice operativo dell'operazione `ISUB`. Facendo riferimento al codice analizzato nell'esercizio 9a, 5.1, abbiamo pensato di modificare il tipo di operazione aritmetica da sottrazione in addizione:

```
isub = 0x5C:
    MAR = SP = SP - 1; rd
    H = TOS
    MDR = TOS = MDR + H; wr; goto main
```

Figura 5.12: Isub modificata

La modifica consiste semplicemente nel sostituire il segno `-` nel segno `+`. Per testare il nuovo codice operativo utilizziamo il seguente programma:

```

1  .main
2  .var
3  a
4  .endvar
5  BIPUSH 0xE
6  BIPUSH 0xA
7  ISUB
8  ISTORE a
9  HALT
10 .endmethod

```

Come possiamo notare, una volta rigenerata la control store e fatto eseguire il programma, a riga numero 94, che sarebbe quella relativa al terzo comando (5.1), i bit relativi al controllo dell'ALU hanno subito una modifica. In particolar modo, possiamo notare che i bit dell'ALU sono: 00111100, che codificano l'operazione di addizione.

```
141 94 => "000000110000001111000010000101000000",
```

Figura 5.13: Microistruzione modificata

Possiamo verificare la correttezza del programma anche tramite l'output della simulazione ($0xA + 0xE = 0x18$):

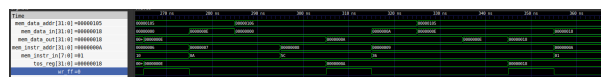


Figura 5.14: Output simulazione